ENGINEERING MANUAL

DebugTUI

用户手册

实现 • 功能 • 架构 • 协议栈
STM32F429 • THA6206 • Bao 调试适配



工程版本 1.3.1；工程配置使用 version = 3

分析基线 1.3.0 多 ELF 实板基线 + 1.3.1 镜像编辑修复

修订范围 Bao/VM 常驻多 ELF 增补；其余历史章节保留日期

修订日期 2026-10-28（Asia/Singapore）

适用平台 当前发布及本地验证：Windows x64

文档文件 `docs/usermanual.tex` / `docs/usermanual.pdf`

阅读说明与证据范围

本手册面向使用者、芯片适配人员和维护者。内容以本地工程的 Rust 实现、TOML 配置、板级脚本、测试脚本和实际日志为依据，覆盖从终端操作到探针、芯片的完整链路。第三方 OpenOCD/GDB 二进制按可见版本、配置及接口分析，不将二进制分发目录误当作 DebugTUI 自研代码。

本次以 2026-10-28 的 1.3.0 安装版多 ELF 实板验收为基线，补充 Bao 与每核 VM 的配置、操作、限制，并纳入 1.3.1 的镜像编辑反馈修复。旧版“不支持多镜像”的表述已修正。其余章节沿用此前手册的专题和历史记录，并非对 0.9.3 以来所有功能重新作全量审计。远端发布状态以 GitHub Release 为准；工程行为以当前代码为准，实板结论以报告中记录的 EXE 哈希、固件和日期为准，不把 1.3.0 的 EXE 验收数字改记为 1.3.1 的新实板结果。

证据类别	本手册的使用方式
当前源码	功能、默认值、路由和约束的首要依据；与旧 README 冲突时采用代码事实。
源码验证	Bao 多 ELF 增补按 1.3.0 核对，安装版 Rust 库测试 606 项、集成 7 项通过，原有 3 项忽略单列；旧章节继续保留各自版本，见第22节。
本地实板	保留 STM32、THA6206 历史证据；新增 2026-10-28 已安装 1.3.0 的 THA6206 双核 Bao/VM 与真实终端 74 项通过记录，见第13.11节。本次编辑和编译手册未再次操作板卡。
仓库历史说明	历史说明须与本机可见原始记录、产物和日期对应；具体证据见各适配章节，不把配置配方当作实板结果。
外部资料	核对 MI/RSP、探针协议、CoreSight/ADI、Trace 和芯片接口；通用架构能力不等于本地已实现或已验证。

本次文档修订的工程根为 `D:/LiBo/Files/DATA/FreeRTOS/vscodegdb/debugtui`。STM32 历史记录来自 `G:/Data/GitFiles/DebugTUI` 与关联固件 `G:/Data/GitFiles/Keil/STM32_CubeIDE/FreeRTOS_Project`，这些旧路径表示当时的测试环境。THA6206/Bao 当前参考目录见第12节，均不随 DebugTUI 分发。迁移时须核对实际路径、工具版本与同次构建产物。

本地 J-Link 长会话曾在约 52–53 秒出现底层访问失效；用户怀疑所用探针为 clone，根因尚未确认，见第21.3节。下载成功与进度显示正常也是两个独立结论；匿名 tuple 解析、SVD 默认端序和资料差异见第23节。

建议首次使用先阅读操作与协议栈；理解探针和片上调试时接着看第8、9节，再进入工具配置和第11节的 STM32F429 实操。THA6206 的 GHS 单核/多核见第12节，Bao Hypervisor、Guest 符号和双 VM 调试见第13节。移植、自动化分别见第20、19节。图中箭头表达请求、数据或触发关系，具体含义由标注说明；均不是电气时序比例图。

目录

阅读说明与证据范围 i

1 产品定位与功能边界 1
1.1 哪些信息由谁负责 . . . 1
1.2 未提供或不能泛化的能力 . . . 1
1.3 v0.8.3 至 v0.9.3 的修订 . . . 2
1.4 1.3.0：Bao 与 VM 常驻多 ELF 增补 . . . 2

2 安装、首次启动与日常工作流 3
2.1 准备材料 . . . 3
2.2 内置工具与工程发现（0.10.0-readonly.5） . . . 3
2.3 首次启动 . . . 4
2.4 项目发现、选择与相对路径 . . . 4
2.5 Examples：从示例建立可修改的配置 . . . 5
2.6 Setup 字段说明与操作入口 . . . 5
2.7 按钮、焦点与保存行为 . . . 7
2.8 一次调试的操作顺序 . . . 7

3 工作台、快捷键与控制台 8
3.1 原始 GDB 命令与工作台命令 . . . 9
3.2 Files 与 Symbols 模糊搜索 . . . 9
3.3 源码选区、复制与加入 Watch . . . 9

4 断点与数据观察点 10
4.1 建立、禁用、删除与编辑 . . . 10
4.2 恢复和失败行为 . . . 11

5 变量、寄存器、内存与 SVD 12
5.1 Watch 和 Locals . . . 12
5.2 数值格式与 CPU 寄存器 . . . 12
5.3 SVD 外设视图 . . . 12

6 运行时内存读取与数据新鲜度 13
6.1 逐项设置方法 . . . 13
6.2 必须理解的地址和一致性限制 . . . 13

7 完整嵌入式调试系统与协议栈 14

8 探针、软硬件协议与物理连接 15
8.1 先区分产品、主机协议和目标接口 . . . 15
8.2 USB 驱动、探针固件与 Server 如何配合 . . . 16
8.3 SWD、JTAG 与复位信号 . . . 17
8.4 STM32F429 接线与电气边界 . . . 17

9 ARM 调试架构：CoreSight、DAP 与 Trace 19
9.1 三条路径：访问、触发、追踪 . . . 19
9.2 DAP、DP、AP 和片上总线 . . . 19
9.3 一次内存访问在 DP/AP 内部如何完成 . . . 20

9.4 Cortex-M4 的停核、断点与观察点 . . . 21
9.5 F429 的两个软件 target 与一个 AP0 . . . 22
9.6 CTI、CTM 与硬件交叉触发 . . . 22
9.7 ETM、ITM、DWT、ATB 与 Trace 接收端 . . . 23
9.8 将通用架构落到 STM32F429 与本工程 . . . 24

10 tools：GDB 与 OpenOCD 25
10.1 三个 profile 的差异 . . . 25
10.2 GDB 最小运行环境 . . . 25
10.3 OpenOCD 服务和目标动作 . . . 26
10.4 自行安装的 J-Link Server . . . 27
10.5 手动启动与所有权 . . . 28

11 STM32F429 本地实板配置与操作 28
11.1 确认本机实际型号和固件 . . . 28
11.2 可复制的项目配置 . . . 29
11.3 板级 OpenOCD 脚本 . . . 30
11.4 两个命名内存通道 . . . 31
11.5 从连接到运行的实操 . . . 31

12 THA6206：GHS 单核与多核适配 32
12.1 参考工程、路径与证据边界 . . . 32
12.2 与 STM32F429 适配的关键差异 . . . 33
12.3 AP、CPU target、CTI 和存储地址 . . . 33
12.4 工具环境、构建和下载 . . . 34
12.5 单独调试 core0 与 core1 . . . 35
12.6 本地板级复位恢复与 CSE 位修正 . . . 35
12.7 0.9.0：本地芯片目录与统一核心选择 . . . 37
12.8 0.8.6 单核与多核实板结果 . . . 40
12.9 双核会话组：启动顺序和控制语义 . . . 41
12.10 断点资源与运行中 Watch . . . 42
12.11 验收记录与故障定位 . . . 43

13 THA6206 上的 Bao Hypervisor 调试 44
13.1 先区分物理核、VM 与 GDB 会话 . . . 44
13.2 EL2、XIP 与 Guest RAM 布局 . . . 44
13.3 构建、烧录与源码映射 . . . 45
13.4 连接与热复位：先处理启动屏障 . . . 47
13.5 同时保留 Bao 与 Guest 的 C 符号 . . . 47
13.6 Symbol images 与每核镜像配置 . . . 48
13.7 源码、栈、Watch 与断点的镜像归属 . . . 50
13.8 在 Bao 中查看挂起 Guest 的保存上下文 . . . 51
13.9 从 HVC 返回 Guest 与运行中取消 . . . 52
13.10 从旧逐核符号配置迁移及能力边界 . . . 52
13.11 1.3.0 安装版的 THA6206 实板验收 . . . 53
13.12 运行时观察：状态块、Timer、IPC 与 UART . . . 53
13.13 双 VM 实板功能与隔离测试证据 . . . 55

13.14 9 月 29 日编译下载故障与历史证据 . . . 55
13.15 后续适配边界与故障定位 . . . 56

14 项目与环境配置参考 57
14.1 覆盖、路径和保存规则 . . . 57
14.2 核心字段 . . . 57
14.3 多核和总线字段 . . . 59

15 实现架构、模块与并发关系 60
15.1 架构分层 . . . 60
15.2 离线源码目录扫描与统一映射 . . . 61
15.3 线程和数据结构 . . . 61
15.4 UI 的按需工作与资源边界 . . . 62

16 MI、RSP、TCL 的报文与流转 63
16.1 GDB/MI2：token 与异步记录 . . . 63
16.2 RSP：由 GDB 和 Server 实现 . . . 63
16.3 TCL RPC：显式 target 与错误封装 . . . 63
16.4 一次“停止并查看变量”的时序 . . . 64

17 连接、执行、暂停与退出状态机 65
17.1 连接流程 . . . 65
17.2 稳定状态和暂态 . . . 66
17.3 暂停竞态的处理 . . . 66
17.4 Build/Download 与退出流程 . . . 66

18 多核工作区与跨核断点 67
18.1 协调器的响应和失败语义 . . . 68
18.2 关联断点不是相同地址推测 . . . 68
18.3 sync、CTI 和 AP 适配边界 . . . 69

19 Headless JSONL 自动化接口 69
19.1 启动、请求与事件约定 . . . 69
19.2 方法参考 . . . 70
19.3 客户端实现注意事项 . . . 72

20 将 DebugTUI 适配到其他芯片 72

21 日志、诊断与常见故障 73
21.1 日志组织 . . . 73
21.2 按故障层次查找 . . . 73
21.3 案例：GDB + J-Link Server 在约 52–53 秒后失效 . . . 75

22 构建、测试、打包与证据矩阵 77
22.1 开发环境和测试分层 . . . 77
22.2 近期新增测试入口与验收方法 . . . 78
22.3 源码测试与历史证据复核 . . . 79
22.4 分发和许可证 . . . 81

23 已知问题、实现限制与维护建议 81

23.1 已确认的下载进度解析缺陷 82
23.2 当前代码与旧说明的差异 82
23.3 实现资源和可观测性限制 82

A 全工程源文件与职责索引 84
A.1 其他目录的工程角色 86

B 命令行速查与术语 87

C 外部参考与文档复现 88
C.1 从 LaTeX 重建 PDF 89

D 流水线产物的本地源码映射 90
D.1 适用场景与现有能力 90
D.2 推荐方式：在 Setup 选择目录层级 90
D.3 步骤一：准备同次构建的镜像、符号和源码 91
D.4 步骤二：查出 ELF 中的旧路径 91
D.5 步骤三：修改工程 TOML 92
D.6 步骤四：烧录配套 HEX，重新启动并验收 92
D.7 故障排查与验收边界 93
D.8 流水线长期规范与参考 93

1 产品定位与功能边界

DebugTUI 是基于 GDB/MI2 的原生终端调试工作台。Ratatui 绘制字符单元格，Crossterm 接收终端事件，Rust 会话核心负责命令、异步状态和进程生命周期。TUI 与 JSON Lines 自动化入口共用核心；运行时无需常驻 Node、Python 或浏览器。npm 是一种分发方式，不是调试引擎。

能力	用户可获得的行为
执行控制	Connect/Reconnect/Disconnect、Run、Continue、Pause、Reset、Step In/Over/Out、指令单步。Reset/Download 由环境动作决定。
代码定位	源文件列表、标签、搜索、PC 定位、栈帧选择、反汇编、路径映射；源码选区、复制和加入当前核 Watch。源码是只读浏览。
数据检查	系统寄存器、Watch、Locals、Memory、CMSIS-SVD 外设/寄存器/位域、独立进制设置。
断点	普通/硬件/临时代码断点，写/读/读写数据观察点，启停、条件、忽略次数、命中数、跨重连恢复。
实时观察	可见项选择显式内存通道；OpenOCD TCL 能力允许时在运行中读内存；另有旧式 ELF 原始全局量 Live Watch。
多核与多 ELF	每核独立物理 GDB，会话组控制、断点联停、单步隔离、共享复位；1.3.0 可为每核常驻 Bao 与所属 VM 符号，见第13.5节。
工程操作	Setup 项目/芯片/核心选择、示例、ELF 路径映射、字段帮助；Build/Download 外部命令、环境引用、偏好保存、日志和清理。
自动化	标准输入/输出 JSONL、脚本串行执行、结构化快照、响应、日志与实时采样事件。

1.1 哪些信息由谁负责

GDB 解释 ELF/DWARF、表达式、栈和 CPU 寄存器。OpenOCD 或 J-Link GDB Server 负责目标访问、断点资源和 Flash 算法。探针承载 USB 到 SWD/JTAG 的转换；芯片提供调试端口及调试资源。DebugTUI 不实现 SWD 波形，不直接解析 DWARF，不内置某颗芯片的寄存器白名单。

SVD 是寄存器描述，不是 Flash 算法，也不建立探针连接。旧式 Live Watch 自行读取 ELF 符号表，仅识别有限的原始全局对象；它不能替代 GDB 类型系统。

1.2 未提供或不能泛化的能力

当前没有专用 FreeRTOS 任务浏览器、RTT/SWO/ETM 跟踪面板、内置源码编辑器、DAP（Debug Adapter Protocol）服务或多人共享调试会话。能调试 FreeRTOS 固件，不等于能枚举和切换全部 RTOS 任务。软件多核协调不保证硬件同时停止或周期级同步。基础配置仍共享 GDB 工具和默认程序/描述；1.3.0 可用 `[[images]]` 为不同核心绑定额外 ELF、独立 Source root 和映射。镜像预先常驻不等于运行中热替换 ELF，也不等于支持所有异构 CPU、迁移 vCPU 或 SMP Guest。

实现依据：`Cargo.toml`；`src/main.rs`；`src/session.rs`；`src/coordinator.rs`；`src/ui.rs`。

1.3 v0.8.3 至 v0.9.3 的修订

本手册以 v0.8.3 标签为比较起点，覆盖至 0.9.3 本地源码和资料。旧的直接单核/显式 cores 配置继续兼容；启用 Chip/Core 选择时使用 version = 3。安装版本通过 EXE 的 –version 核对；实际发布附件和 Latest 状态以 GitHub Release 为准。

变更范围	本手册的对应内容与边界
STM32 工具配置	补充 CMSIS-DAP/OpenOCD 配方；硬件脚本可用环境变量选择 profile，不能把换 profile 等同于完成实板验收。
CI 与芯片适配资料	保留并整合流水线 ELF 源码映射、THA6206 GHS 单核/多核、Bao Hypervisor/Guest 调试步骤；这些资料已在此前提交中加入。
0.8.4 启动配置	首页 Exit、相对 Project 路径、Projects 选择、Examples 示例、加载后刷新配置、按字段显示多行帮助及窄屏布局。
0.8.4 源码操作	鼠标/键盘选区、系统剪贴板复制、加入当前核 Watch、长行横向滚动；保留只读源码及断点栏操作。
0.8.5 搜索与退出	Files 文件模糊筛选、Symbols 函数/变量/类型索引；远程 resume 采用 Continue 后 Disconnect，并核对 GDB 正常退出。
0.8.6 工程配置	Setup 只编辑工程字段及工具环境引用；兼容旧内嵌覆盖和单核/多核配置。三个输入区使用显著边框与聚焦提示。
0.8.7/0.8.8 源码重映射	Setup 离线读取 ELF 源文件目录，选择旧路径前缀映射至 Source root；关闭时保留规则并禁用目录选择，兼容正反斜杠。
0.9.0 芯片与核心	用户本地芯片目录、TUI 新增芯片、核心子集选择、后端匹配、逐核端口和启动模板、按芯片/核心保存 Watch 与断点。
0.9.1 保存与可读性	删除 Save to project 开关，统一 Save config；Start 自动保存。工具栏与标签页增加边界、底色和选中高亮。
0.9.2/0.9.3 界面	真实圆角边框及内部填色、深蓝灰配色、宽屏共享调试操作栏；Setup 默认聚焦 Project，上下箭头只在可用配置项之间循环。
Bao 外部适配更新	迁移至芯片/核心选择，Run 与整片复位分离；WSL 构建、产物校验清单、烧录退出码和日志修正，见第13.3节。
本地 THA6206 板级修正	说明外部参考工程 OpenOCD 的 DBGRESET、复位顺序和 EDRCR.CSE 位修正，单列验证范围；该脚本不包含在 DebugTUI 程序包中。

1.4 1.3.0：Bao 与 VM 常驻多 ELF 增补

每核在连接前加载 Bao 与对应 Guest ELF；真实停止后识别执行镜像，Source、Files、Symbols、反汇编、Watch 与断点保留镜像归属。源码浏览不会重连、复位、下载或改变物理执行位置；运行中不为识别镜像额外暂停。有效诊断快照还可只读查看挂起 Guest 的源码、寄存器、栈和局部变量，并在完整 HVC32 边界返回 Guest。配置、命令、状态与限制集中在第13.5至13.11节，日常入口为 B 的

debug-multi-images.toml。

2 安装、首次启动与日常工作流

2.1 准备材料

当前发布目标为 Windows x64。运行调试至少需要 DebugTUI EXE、匹配目标架构的 GDB；远程嵌入式场景还需 GDB Server、探针驱动和板卡。源码调试需要与板上固件匹配的 ELF 和源码；外设视图另需 SVD。工具目录应放在工程或独立工具区，避免随 npm 升级被替换。

本地构建与已有包安装示例：

```
# In the DebugTUI repository:
./scripts/build.ps1 -Release
./target/release/debugtui.exe --version
./target/release/debugtui.exe --demo

# Existing portable executable: run directly.
# Existing npm archive: substitute its actual filename.
npm install -g ./artifacts/debugtui-cli-0.9.3.tgz
debugtui
```

仓库 PUBLISHING.md 描述了 GitHub Release URL 安装流程；本文不将本地包版本等同于远端 Latest。主程序包不包含 tools 和 SVD，交付时须分别准备。安装后用户芯片目录独立保存在用户配置区；若 npm 未执行 postinstall，可运行 `debugtui --init-profiles` 完成初始化，不必修改全局 npm 策略。已有目录会校验但不会覆盖，详见第12.7节。

2.2 内置工具与工程发现（0.10.0-readonly.5）

npm 包和便携 ZIP 已内置 GDB/OpenOCD、芯片与探针配置、SVD。工程使用 `[tools] profile="builtin:arm-openocd"; probe="cmsis-dap"` 可在 Setup 的 Probe 字段改为 jlink、stlink 或继承默认值。Chip config 对应 `tools.chip\_profile`，可指定工程相对的自定义芯片 TOML。内置模式按工程指定文件、用户 `profiles/chips/<chip>.toml`、包内默认文件的顺序选择；用户文件不会写入 npm 安装目录。自定义继承可使用 `extends="builtin:devices/tha6206.toml"`，板级 cfg/SVD 相对该文件。

Tools / profile 和 SVD file 按 F2 可选择包内文件或浏览外部文件，列表显示实际安装路径。内置 SVD 引用为 `builtin:svd/STM32F429.svd`；Automatic 跟随芯片，Disabled 显式禁用。旧 profile 下切换 Probe 会提供兼容工具选择，选择内置 profile 后应用，Esc 取消保留原配置。切换时只迁移与包内文件完全相同的原始 SVD 副本；客户修改保留，Ctrl+S 才保存工程。

无参数启动从工作目录优先加载 `debug.toml`，否则检索调试工程 TOML；唯一候选直接加载，多个候选显示列表，无候选则创建最简通用配置并进入 Setup。`--init-project` 只查找或创建配置后退出。已有工程保持其原配置，旧的外部 profile 与 `.vscode` 方式继续兼容。新工程无需复制工具或写入 npm 安装路径。

2.3 首次启动

无参数执行 `debugtui` 进入 Setup，不启动 GDB 或占用探针。依次检查 Project、Tools/profile、Program/ELF、Source root、SVD、Build/Download 和退出策略。GDB、目标连接、服务启动及工具超时在引用的 debug-env.toml 中维护。默认聚焦 Project；上下箭头只在可用配置项之间循环，跳过禁用项和顶部按钮。Tab、Shift+Tab 或鼠标可选择顶部按钮；点击 Start debugging，或使用 Ctrl+R、Ctrl+Enter、F5 启动。Save 只保存；Workspace/Esc 回到原工作区并保留草稿。**首页可直接点击 Exit 或按 Ctrl+Q 退出，无需先启动调试**；已有会话仍按正常退出流程清理。

```
debugtui --project ./firmware --setup
debugtui --project ./firmware/debug.toml
debugtui --gdb C:/MinGW/bin/gdb.exe --local --elf ./app.exe
debugtui --gdb C:/tools/arm-none-eabi-gdb.exe `
  --connect 127.0.0.1:3333 --elf ./build/app.elf
```

`--connect` 和 `--local` 禁用自有 service 启动，但不会清除已选 profile 的其他连接动作。若只想建立通用连接，使用明确的 GDB/目标参数且不加载芯片 profile。只有工程中的 `tools/debug-env.toml` 在适当条件下会被发现，TUI 安装目录不作为隐式工具搜索根。

2.4 项目发现、选择与相对路径

没有指定项目时，默认路径是启动目录下的 `./debug.toml`。**进入 Setup 不会自动创建这个文件**：没有现成配置时只是建立内存草稿，点击 Save config/Ctrl+S 或启动调试时才保存。

启动目录情况	默认行为及可执行操作
存在 `debug.toml`	载入默认项目并显示其配置；用 Projects/F3 换到其他调试项目。
没有默认文件，但有项目 TOML	首次空草稿启动时打开候选列表，选择后重新解析并刷新 Setup 各项。返回已编辑的草稿时不强制再次选择。
没有项目 TOML	显示 `./debug.toml` 草稿；可直接填写，也可先用 Examples/F4 套用示例。
配置在其他目录	在 Project 字段按 F2 浏览目录及 TOML 文件，或直接编辑路径；加载失败显示错误并保留此前的有效草稿。

Projects 列表筛选调试项目配置，排除 `Cargo.toml` 和 `debug-env*.toml` 等工具配置；并非所有 TOML 都是可启动项目。`debug-*.toml` 即使损坏也可出现在列表，选择时会报告解析错误。F2 是文件浏览，F3 是项目选择，两者用途不同。

Project 字段以**启动工作目录**为相对路径基准。例如 `./debug.toml` 或 `./examples/debug.toml`；跨 Windows 盘符等无法相对表示的路径仍保留绝对路径。选择项目后，ELF、tools/profile、Source root、SVD、日志等项目资源的相对路径则按**所选 TOML 所在目录**解析。不能因为 Project 显示相对路径，就把全部字段都理解为相对启动目录。

2.5 Examples：从示例建立可修改的配置

在 Setup 点击 Examples 或按 F4，选中示例后按 Enter 应用。该操作替换当前配置草稿并立即刷新字段，**不会写文件、连接探针或执行 Build/Download**；检查并修改后再保存或启动。需要保留尚未保存的自定义内容时，应先保存再切换示例。

示例	填入的主要配置及需要自行补充的内容
单核工程	`./build/firmware.elf`；自行选择 ARM、RISC-V 或其他目标的工具 profile，示例不内嵌工具参数。
本地程序	Windows 程序示例为 `./build/app.exe`；选择声明 `target.mode='local'` 的工具 profile。
双核工程	core0/core1 分别连接 3333/3334，默认 Scope All 和联停；共享 ELF，需选用兼容的工具环境并核对逐核配置。
本地工具配置	若项目下有 `debug-env.toml`、`.vscode/debug-env.toml` 或 `tools/debug-env.toml`，提供引用选项，不改写 profile。

通用示例使用 `./build/firmware.elf`、Source root = .、detach 退出策略和 `debug_log` 日志目录。示例不生成 GDB、target、service 或工具超时字段；保留已选 profile 和旧工程覆盖值。SVD 与构建/下载命令按工程补齐。双核示例只有在 Server 确实暴露两个对应核心端口时才能使用。

2.6 Setup 字段说明与操作入口

选中字段后，表单下方显示对应的用途、填写格式、路径基准和空值影响；长说明随宽度换行。窄屏下顶部按钮换行，字段列表可滚动。45×12 是布局回归用例，实际配置建议至少 80×24，完整工作台建议 120×36。

字段	示例或入口	用途与填写规则
Project	`./debug.toml`；F2/F3	项目配置的读写目标，按启动目录解析。选择已有文件会更新整张配置表；新路径在保存前只是草稿。
Tools / profile	`.vscode/debug-env.toml`；F2	引用工具/服务/动作配置，相对项目目录。留空使用独立 GDB 配置，不自动获得芯片复位和下载能力。
Chip	Enter；目录选择	本地芯片能力声明；选择型号或新增客户型号。Legacy 保留原配置模式。
Debug cores	Enter；核心复选	选择该芯片支持的物理核心子集；至少一个。未选择 Chip 时灰显。配置示例见第12.7节。
Program / ELF	`build/firmware.elf`；F2	与目标固件匹配的符号文件，相对项目目录；不是自动烧录命令。无有效调试信息时无法完整进行 C 源码调试。
Symbol images	Enter；镜像编辑器	1.3.0 的多 ELF 工程入口；维护 ID、ELF、核心、域、源码根/映射、偏移和内存通道。Apply 更新草稿，Save 保存，Start 才建立符号会话，见第13.6节。
Source root	.；F2	本地源码定位根及 Build/Download 工作目录；相对项目目录。

字段	示例或入口	用途与填写规则
Build command	`.\build .bat`	可选构建命令；在 Source root 执行。保存、应用示例或单纯启动 Setup 不会执行它。
Download command	`.\flash .bat`	可选下载命令；在 Source root 执行，使用工程自己的烧录脚本或工具。先核对输入产物和目标。
On exit	detach / resume / disconnect	detach 脱离；resume 对远程目标先 Continue 再 Disconnect，对本地进程暂停后 Detach；disconnect 只关闭连接。最终运行状态仍受 Server 和板级配置影响。
Log directory	`debug_log`；F2	可选会话日志目录，相对项目目录；建议排查多核问题时保留各核 MI 与 Server 日志。
SVD file	`device.svd`；F2	可选外设寄存器描述，相对项目目录；留空仍能使用基本调试，不提供相应外设树。
Source remap	Yes / No；Enter	开关源码重映射；启用后离线扫描 ELF，关闭保留规则。
ELF path prefix	Enter	选择 ELF 中与 Source root 对应的目录层级；显示本地匹配预览。Source remap 为 No 时灰显，键盘跳过、鼠标不可选择或修改。详见附录 D。

配置职责：debug.toml 保存程序、源码映射、任务、SVD、Watch/断点、逐核映射、组策略和 `session.on_exit`；debug-env.toml 保存 GDB、OpenOCD/服务、连接默认值、板级动作和 `session.timeout_ms`，也可通过 `[program] svd` 提供芯片 SVD 默认值，其路径相对于 profile 所在目录。工程中的 SVD 配置优先，空字符串表示禁用；ELF 和源码路径仍由工程配置。随附工具需要 0.10.0-readonly.3 或更新版本；`tools/install.ps1` 将 `tools/debug.toml` 安装到工程根目录、其余工具资源安装到 `.vscode/`（旧工程兼容方式）；新工程直接使用 npm 包内工具。SVD 由芯片描述提供，工程不固定覆盖。`session.timeout_ms` 默认 8000 ms 是单条工具命令等待时间，不是工程退出策略。Setup 已移除六个工具编辑项，Tools/profile 只选择引用，On exit 仍是工程字段。旧工程内嵌工具覆盖继续有效；保存不展开环境值、不改共享 profile，并保留单核/多核的后台偏好。

Setup 中的按键/按钮	行为
Projects / F3；Examples / F4	打开项目列表；打开示例列表。选择后回到 Setup 查看更新的字段。
F2；Enter	在支持的路径字段浏览；激活当前按钮、字段或列表项。
Save config / Ctrl+S	保存当前项目配置，不启动或重连调试；遇到外部修改冲突时报告错误，避免覆盖新内容。
Start / Ctrl+R / F5	校验配置，结束旧会话，保存配置并启动新会话。
Workspace / Esc	从表单返回工作区并保留草稿；在编辑器或选择弹窗中先退出当前编辑/弹窗。
Exit / Ctrl+Q	直接结束程序并执行正常清理；无需先进入调试工作区。

实现依据：`src/launch.rs`: Document、Setup、字段提示及相对路径；`src/launch/choices.rs`: 项目发现与示例；`src/ui.rs`: 启动和退出路由。

2.7 按钮、焦点与保存行为

0.9.3 使用深蓝灰分层面板、蓝色焦点和真实圆角边框。按钮、搜索框只填充边框内部，悬停、按下和输入反馈也不会给圆角外围染色；禁用项灰显，核心选择保留独立身份色，工作区背景仅施加轻微核心色调。范围包括 Setup 顶部、Project/运行控制工具栏、核心切换、源码页签、检查器与 Watch/Locals。

普通窗口使用三行完整圆角按钮，短窗口保留单行左右边框及原点击区域。宽屏横向排列 Project 操作、核心选择与 Symbols 搜索，并在源码和 Inspector 上方共用调试操作栏；两侧页签对齐。窄面板使用 Regs / Periph 短标签，内容标题仍显示全称；窄窗口保留换行和紧凑布局。Setup 默认选中 Project，箭头不进入顶部按钮，Tab 仍可访问按钮。

Save config 立即保存当前草稿，不 Build、不 Download，也不重新连接。**Start debugging** 先校验，再关闭旧会话、保存配置并启动新会话；保存错误会阻止新会话启动。下方不再存在 Save to project 开关。Workspace/Esc 保留内存草稿，Exit 退出时未手动保存的 Setup 草稿会放弃；已经运行的会话仍按正常规则清理和保存其 Watch/断点。重映射、Chip/Core 等启动配置须重新 Start 才应用到调试会话。

实现依据：`src/theme.rs`: control/button；`src/ui/render.rs`、`src/ui/cores.rs`、`src/launch.rs`、`src/ui.rs`。

2.8 一次调试的操作顺序

1. 连接，检查 STOPPED/READY 状态及日志，确认固件和 ELF 匹配。
2. 在已暂停目标上添加源码断点、Watch，浏览栈和外设。
3. F5 继续，等待真实停止事件；F6 暂停，F10/F11/Shift+F11 单步。
4. 需要运行时观察时，暂停解析地址，再显式启用允许的内存通道。
5. Build/Download 前核对工程目录和动作；下载在 TUI 中需要确认。
6. 使用 Ctrl+Q 结束，核对断开日志；退出策略优先来自工程 session.on_exit，板上最终状态还取决于服务端行为。

实现依据：`src/cli.rs`: Options；`src/launch.rs`: Document/Setup；`src/ui.rs`: run；`scripts/build.ps1`。

3 工作台、快捷键与控制台

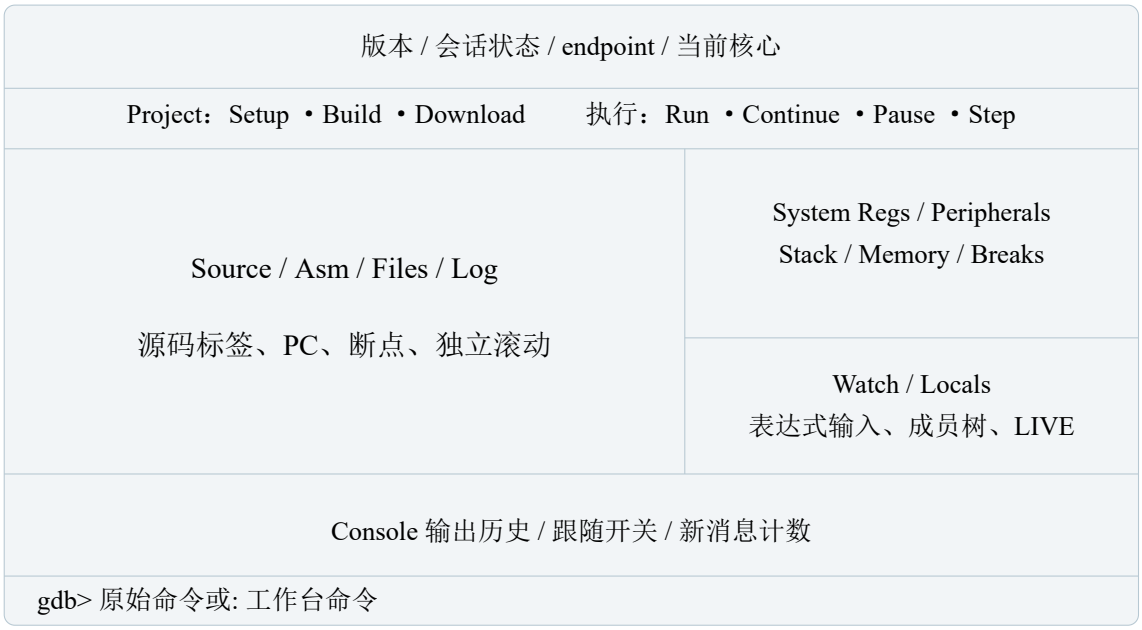


图 1: 工作台逻辑布局示意；面板比例随终端大小调整，并非实板截图。

按键	操作
F2 / Ctrl+P / Ctrl+Q	Setup / Help 命令面板 / 清理并退出
F5 / F6	继续或从 READY 启动 / 暂停；F6 不受源码选区影响
Ctrl+C / Ctrl+Enter	Source 聚焦且有选区时复制 / 加入当前核 Watch；无源码选区时 Ctrl+C 保留暂停用途
F9 / F10 / F11 / Shift+F11	源码断点 / 越过 / 进入 / 跳出
Tab / Shift+Tab	切换视图与键盘焦点；弹窗内切字段
Ctrl+F / Ctrl+O	源码搜索 / 搜索已打开的文件
Ctrl+PgUp / Ctrl+PgDn / Ctrl+W	切源码标签 / 关闭当前源码标签
Ctrl+T	多核工作区切换当前检查核
/ / : / Esc	聚焦控制台 / 输入工作台命令 / 退出输入或关闭弹窗
f / 右键	选中数值的格式菜单；Watch/外设中再按 r 可配置内存通道

三组面板分别保存活动标签；点击右侧标签不切走源码。窄于 100 列时只显示当前焦点所属面板组，建议使用至少 120×36 的终端获得完整工作台。滚轮和滚动条作用于所在视图。Console 向上浏览后停止跟随，End 回到实时输出；输入区保持独立。

实现依据：`src/ui.rs`: PANES、HELP、key、mouse；`src/ui/render.rs`；`src/ui/console.rs`。

3.1 原始 GDB 命令与工作台命令

```
:watch xTickCount
:break Task100ms
:memory $sp 256
:disasm $pc
:frame 1
:refresh

p/x xTickCount
info registers
compare-sections .text
```

冒号命令由 UI 解析为结构化 Request；原始命令通常封装成 `-interpreter-exec console`。常见执行别名如 `c`/`continue`、`s`/`step` 经会话层路由；多核的组控制应优先使用工具栏或结构化方法。原始复杂 GDB 命令仍属当前核，不能假定所有执行别名都遵循 Scope。

Console 与 Watch 各有输入缓冲和历史/补全状态。输入停顿约 150 ms 后最多发送一个补全请求，最多返回 64 项；运行中不查询符号。裸变量前缀走 `-symbol-info-variables`，成员或 GDB 命令走 `-complete`；选中补全不执行表达式。

Source 标签按映射后规范路径去重，每个文件记住滚动和光标。手动打开或关闭源码不切 GDB 栈帧；选择 Stack 行才切上下文。新停止会定位真正的文件和 PC，无源码的停止清空当前源码视图，避免误认旧位置。单文件读取限制为 2 MiB / 30000 行。

3.2 Files 与 Symbols 模糊搜索

Files 页的 Find 框支持点击、Ctrl+F 或 / 开始输入，不区分大小写；先按包含匹配，再按字符顺序模糊匹配。输入 spi 可找到 Spi.c、Spi_Irq.c、espi_hal.c 和 espi_std.c，也可输入路径片段。上下键选择，Enter 或点击结果打开 Source，Ctrl+U 清空；范围为 GDB 提供的 ELF 源文件列表。

顶部 Symbols 框通过点击、Ctrl+K 或 `:symbols` 打开，检索 ELF 中的函数、全局/静态变量和类型。结果显示类别、文件和行号，选中后跳到映射后的本地源码；不切换 GDB 栈帧、PC 或断点。输入停顿 200 ms 后查询，每类最多 200 项；遇到上限需缩小查询，F4 可重试。

新符号查询要求 READY/STOPPED，运行时只可使用已返回的结果，不会隐式暂停目标；切核、重连或换 ELF 时丢弃过期结果。缺少调试信息、位置或对应文件时显示原因，不猜测跳转位置。宏、未编入 ELF 的文件和普通函数局部变量不属于全局索引。Symbols、Files 和 Watch 输入区均显示边框和聚焦状态；Watch 输入框用于添加表达式。

实现依据：`src/search.rs`、`src/session/symbols.rs`、`src/ui/search.rs`；实板终端结果见 `artifacts/regression-0.8.6-20260923/hardware-results.json`。

3.3 源码选区、复制与加入 Watch

0.8.4 本地版本在只读 Source 视图中加入文本选区。**行号/断点栏点击仍切换断点，代码正文点击用于定位和选择文本**；F9 仍对当前源码行操作。源码选区不会修改文件、重新编译或下载固件。

操作	行为与范围
鼠标拖动、Shift+ 点击	建立或扩展选区，支持反向拖动和跨行选择；拖动到边缘可滚动。
双击单词；右键	双击选择标识符；在已有选区内右键保留选区，否则定位单词后打开 Copy / Add to Watch 菜单。
Shift+ 方向键/Home/End	扩展选区；Ctrl+Home/End 到文件首尾，配合 Shift 扩选。Ctrl+A 选择整份已载入源码，Esc 清除选区。
Copy / Ctrl+C	把选区的源码文本写入系统剪贴板，不含行号、断点标记和语法高亮；支持中文、Tab 与多行。
Add to Watch / Ctrl+Enter	把去除首尾空白后的单行表达式加入当前检查核的 Watch。多行或含无效控制字符的文本会被拒绝并提示。
左右移动与横向滚动	长行可随光标/选区横向滚动；行号栏固定，文字位置按显示宽度换算，不把中文或 Tab 当作单字节字符。

Copy 和 Add to Watch 按钮在 Source 顶部空间足够时显示；窄窗口可使用右键菜单或快捷键。快捷键依赖 Source 获得焦点；若终端拦截 Ctrl+Enter 或鼠标事件，可改用可用的按钮/菜单。终端自己的文本选择与 DebugTUI 源码选区是两种不同操作。

加入 Watch 复用已有的表达式求值和错误显示：它使用**当前核心、当前栈帧**，不会把表达式自动加入其他核心。运行中加入不会为求值而自动暂停，显示和更新仍取决于普通 Watch 或显式实时内存通道；表达式不合法时保留错误供修改。单纯移动或复制选区不向 GDB 求值；主动加入 Watch 后则按常规 GDB 表达式规则处理。

当 Source 聚焦且有选区时，Ctrl+C 优先复制，不再表示暂停。需要确定暂停目标时使用 F6 或 Pause。切换核心、切换源码或处理新的停止上下文时会清理选区/菜单，避免把旧选区误用于新上下文。

Windows 使用原生 Unicode 剪贴板接口，不依赖 VS Code、PowerShell 或 Python 剪贴板脚本；复制失败会显示错误。其他平台实现使用系统剪贴板工具，但当前交付验证仍以 Windows x64 为准。这项功能不是源码编辑器。

实现依据：`src/ui/source_text.rs`；`src/ui/highlight.rs`；`src/ui/render.rs`；`src/clipboard.rs`；`src/ui/source_text/tests.rs`。

F5 Continue 与工具栏 Run 不应混用：本地 READY 的 Continue 会启动程序；STM32 profile 的 Run 明确包含 reset halt，可能重启应用。普通 STOPPED 下 F5 仅继续执行。

4 断点与数据观察点

4.1 建立、禁用、删除与编辑

Source 行号点击或 F9 切换源码断点；Breaks 的 + Code（Insert/n）接受函数、`file:line` 或 `*address`。选 Hardware 显式要求硬件断点；Temporary 在命中后删除且不持久化。已有禁用源码断点按 F9 会重新启用。

Breaks 的行首开关或 Space/Enter 只切换启用状态，Delete 才删除。Edit（e/右键）设置条件和忽略次数；忽略 N 表示跳过接下来 N 次命中，UI 显示 GDB 实际命中总数和剩余忽略次数，不自行计

数。条件必须是目标 GDB 能解释的表达式。

类型	MI 命令	语义和限制
代码	`-break-insert`	普通断点，Flash 地址可能由 GDB 自动选择硬件资源。
硬件代码	`-break-insert -h`	不通过可写内存修改指令；数量受目标资源限制。
写观察点	`-break-watch`	值变化语义；相同值写入不一定停下。
读观察点	`-break-watch -r`	需要目标支持读访问捕获。
读写观察点	`-break-watch -a`	需要目标支持访问捕获。

```
:break Task100ms
:data-break write xTickCount
:data-break read *(unsigned int *)&xTickCount
:data-break access *(uint32_t *)0x20000000
:disable 2
:enable 2
:disable all
```

`uint32_t` 等类型必须存在于当前 ELF 调试信息中。地址例子只说明语法，实际访问先确认该地址用途。硬件观察点宽度、对齐和可用数量由 GDB/Server 判断。不要把本板日志中的资源数量当作所有 Cortex-M 的固定配额。

4.2 恢复和失败行为

详细定义保存在项目配置中，含 kind、enabled、condition、ignore_count、temporary 以及可选 group；重连重新创建，GDB 编号可能变化。恢复失败保留可见 `[!]` 记录，可重试或删除。修改已有断点失败会尝试恢复原设置，新增数据断点的附加设置失败会清理已创建项，随后查询真实状态。

```
[[breakpoints]]
location = "Task100ms"
kind = "hardware"
enabled = true
condition = "xTickCount > 1000"
ignore_count = 2
temporary = false
```

旧式 `breakpoints = ["main"]` 仍接受。通过面板修改立即保存；Watch 列表的保存主要在断开/退出时完成，不要把两者的持久化时机混为一谈。临时会话没有项目路径时无法获得同样的磁盘持久化。

实现依据：`src/session/breakpoints.rs`；`src/ui/breakpoints.rs`；`src/config.rs`: BreakpointSpec、save_preferences。

5 变量、寄存器、内存与 SVD

5.1 Watch 和 Locals

Watch 底部输入表达式并按 Enter 或 + Add；删除顶层表达式使用 ×、Delete 或 Del Remove。运行中可增删，新增值等待下一次暂停。删除不会写变量或删除硬件观察点。Locals 跟随选中栈帧，不是所有任务/所有函数局部变量的全局集合。

结构体、数组、指针可以展开；每页 32 个子节点，每根最多 256 节点、8 层，根表达式最多 64 项。暂停刷新时创建短寿命 GDB variable object，只读取展开分支，然后删除根对象及全部子对象。折叠/删除不发目标读取；不依赖 Python pretty printer，不支持变量对象的 GDB 回退到标量求值。

```
object.member
(struct MyType *)0x20000000
*(struct MyType *)0x20000000
((struct MyType *)0x20000000)->member
*(uint32_t *)0x20000000
```

强转无法创造真实类型定义。错误地址、优化掉的变量或缺失调试信息会形成行内错误；更换源码文件也不会改变 ELF 中已有的类型和行号信息。

5.2 数值格式与 CPU 寄存器

Watch、Locals、Memory 字节默认十进制；CPU 寄存器、SVD 寄存器/位域默认十六进制；地址固定十六进制。右键或 f 独立选择二/八/十/十六进制。转换使用最多 128 位整数计算，不写目标，不改 GDB 全局 radix；负数保留负号，浮点/枚举/字符串等保留自然表示。Locals 格式键包含文件和函数，成员格式键包含树路径。

寄存器名称和编号从 GDB 动态取得；`gdb.registers = []` 默认读取所有命名寄存器。配置非空列表可避开复位时不可访问的浮点寄存器组，不存在的名称会报错。专用面板用于读取；需要修改寄存器或内存时使用明确的原始 GDB 命令，修改会影响目标运行。

5.3 SVD 外设视图

`program.svd` 选择 CMSIS-SVD 文件。解析器展开 derivedFrom、数组、cluster 和属性继承，生成设备、外设、寄存器和位域模型；位域从父寄存器值按位偏移/宽度解码，不额外读总线。

默认只在停止、外设面板可见、分组展开时读当前可见寄存器，每个停止代次读一次。只写寄存器跳过；声明 readAction 的寄存器或位域禁止自动刷新，可显式手动读其父寄存器。r / Refresh selected 触发单次读取，错误在行内展示。未声明的硬件副作用不能从 XML 推断。

文件最大 32 MiB，要求字节寻址；读取只接受自然对齐的 8/16/32/64 位。**当前源码在缺失或未知 endian 时默认小端**，与旧说明中的“不猜测”不同；移植时必须核对 SVD。本地 STM32F429 为小端，不受该默认值差异影响。

实现依据：`src/session/watch.rs`；`src/ui/watch.rs`；`src/ui/formats.rs`；`src/svd.rs`: Device::parse；`src/ui/peripherals.rs`。

6 运行时内存读取与数据新鲜度

运行中显示的普通快照是上次停止值。若要在不暂停 CPU 的条件下读 SRAM 或外设，必须选择环境明确声明的 OpenOCD 通道，并理解读总线与 GDB 求表达式的区别。

路径	解析与能力	调度与显示
默认 GDB	停止时由 GDB 获取地址/类型或内存；不允许运行中读取。	停止快照及可见面板按需更新。
逐项 Memory access	先由 GDB 解析可取地址标量，随后按命名 TCL target 读取；支持 8/16/32/64 位、符号整数和 float/double。	UI 调度当前可见项，50–60000 ms，失败至少 1 s 退避；独立缓存标记 LIVE。
旧 live_watch	直接解析小端 ELF32/64 符号表中的唯一全局 8/16/32 位对象；不推断 C 类型。	独立采样线程，运行时采样当前核 Watch 列表；事件标记 raw N-bit。此路径不采用逐项可见性调度。

6.1 逐项设置方法

暂停后添加 Watch 并等其值正常显示；右键数值或 f，再按 r 打开 Memory access / Live refresh。Access 选 `ahb`，Live refresh 选 On，Interval 设为 100，Apply & save。Read once & save 保存策略并单读，不要求循环。关闭面板或折叠成员会停止对应的逐项后台读取。

```
[ui.refresh."single|watch:xTickCount"]
channel = "ahb"
interval_ms = 100
```

channel 为空代表默认 GDB；interval_ms 为 0 关闭循环。多核键由 UI 按核生成，优先通过菜单保存。根结构体策略可由成员继承，成员可单独覆盖。显式逐项策略优先于旧 live_watch，包括关闭或手动设置。

6.2 必须理解的地址和一致性限制

可取地址的标量、指针和成员可解析；临时结果、CPU 寄存器、位域和无地址表达式不能直接总线轮询。指针对象的采样可读到指针值，但已解析的解引用成员不会在运行中自动跟踪指针的新地址，下次暂停才重新解析。64 位值以两次 32 位读组合，不保证原子性。

一次读取成功只代表该次事务成功，不保证多个变量构成同一时刻的快照。即使 CPU 停止，DMA、外设或其他核仍可能修改内存。运行读取频率是尽力而为：串行请求、探针吞吐、前台命令和超时都会降低频率；100 ms 不是固定 10 Hz 的硬实时承诺。移植至有缓存/MMU 的平台还须核对物理地址与核内视图。

实现依据：`src/session/memory.rs`；`src/ui/monitor.rs`: ensure_monitors、monitor_response；`src/live_watch.rs`。

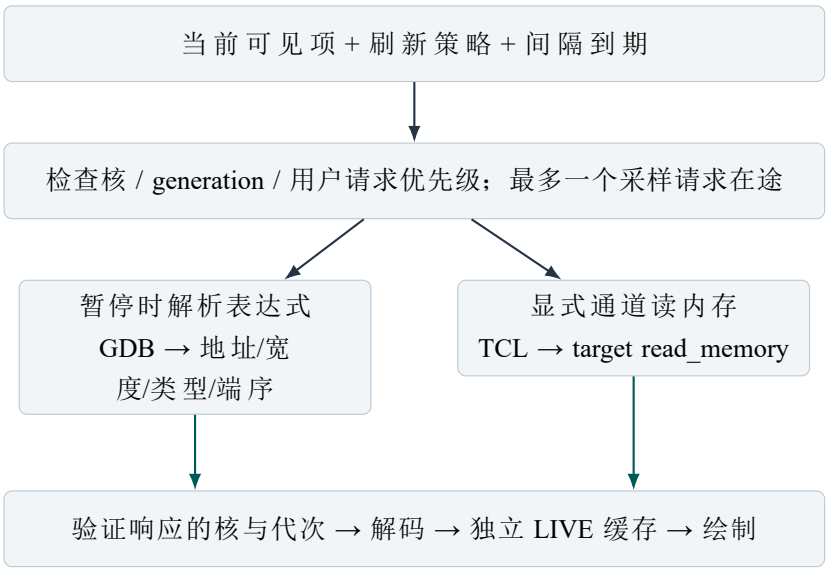


图 2: 逐项内存通道数据流。错核、过期或已删除观察项的响应丢弃。

7 完整嵌入式调试系统与协议栈

前六节描述从工作台发出的操作。本节沿着一次调试请求向下追踪：主机协议把意图交给 Server，Server 通过探针访问芯片；后续两节再展开物理接口和片内路径。这些层次必须一起匹配，才能解释“USB 已识别但不能调试”或“能暂停但没有 Trace”等现象。

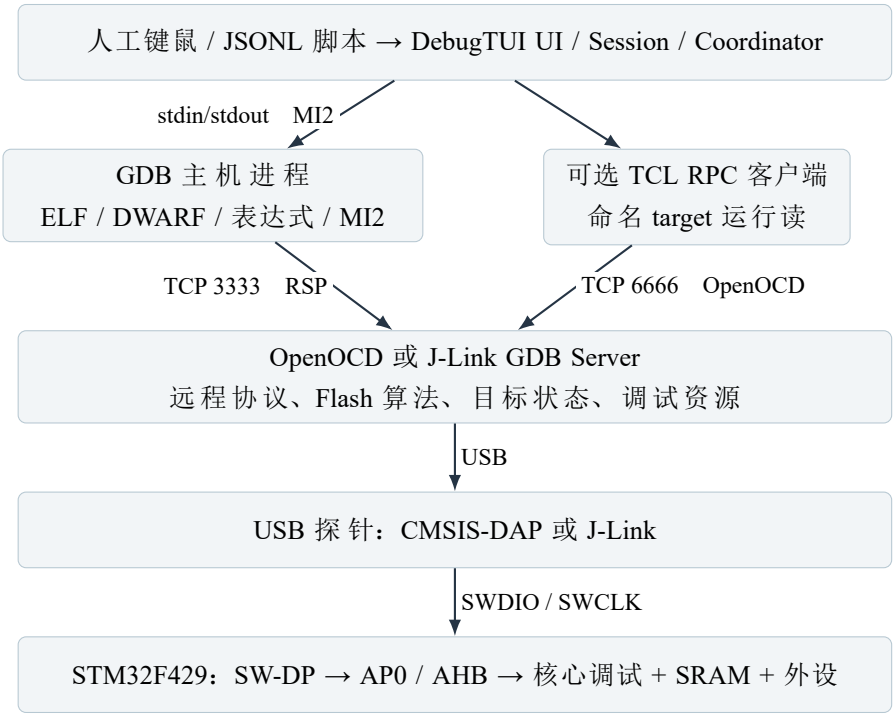


图 3: 本地 STM32F429 调试栈；J-Link Server profile 不提供此处的 TCL 旁路。

层	载体/端点	主要职责
人机/自动化	终端字符、键鼠、JSONL	发出意图、显示结果；两种启动模式不共享同一实时会话。
应用会话	Rust mpsc 通道	Request/Response、快照、日志、错误、取消。
调试器接口	GDB stdin/stdout，MI2	带 token 的命令、结果及异步执行事件。
远程调试	TCP，GDB RSP	GDB 与 Server 交换寄存器、内存、断点和控制请求。
内存旁路	TCP，TCL RPC	DebugTUI 直接请求 OpenOCD 指定 target 读内存。
适配器	USB，探针协议	Server 控制 CMSIS-DAP 或 J-Link；TUI 不实现此层。
目标链路	SWD，SW-DP/MEM-AP	寄存器/总线访问和核心调试；这里的 AP0 属于 F429 配置。

MI 是 GDB 的主机接口，RSP 是 GDB 与 Server 的远程接口，CMSIS-DAP 是探针接口，三者不可互换。OpenOCD 的 TCL 是额外控制入口，不是 MI 或 telnet。GDB/MI 官方说明、RSP 概览和TCL RPC 文档给出各层语义。

实现依据：`src/session.rs`: connect_inner；`src/live_watch.rs`: transact；tools/debug-env.toml 与 tools/devices/ 芯片描述。

8 探针、软硬件协议与物理连接

8.1 先区分产品、主机协议和目标接口

SWD、JTAG、J-Link 和 CMSIS-DAP 经常同时出现在连接说明里，但它们并非四种互斥的线缆协议。J-Link 是探针及其软件产品体系；CMSIS-DAP 定义主机与探针固件之间的标准命令接口；SWD/JTAG 则处于探针与目标芯片之间。一个 CMSIS-DAP 探针或 J-Link 探针都可能支持 SWD 和 JTAG，具体能力取决于型号、固件与驱动。

概念	所处位置	对本工程的含义
GDB/MI、RSP	UI ↔ GDB；GDB ↔ Server	DebugTUI 发 MI；Server 将 RSP 请求转换为目标访问。它们不指定 USB 驱动或板上的引脚。
J-Link	硬件探针、固件、主机库和工具	可由 J-Link GDB Server 或 OpenOCD 的 jlink 驱动使用；两套服务端实现和主机依赖不同。
CMSIS-DAP	主机 ↔ 探针固件	DAP_Info、DAP_Connect、DAP_Transfer 等命令用于查询能力、选择接口和搬运访问；不是 Cortex-M 内部的 AP 编号。
SWD	探针 ↔ SW-DP	SWCLK 与双向 SWDIO 承载调试请求、应答和数据；地、电压参考及复位另计。

概念	所处位置	对本工程的含义
JTAG	探针 ↔ TAP/扫描链	TCK、TMS、TDI、TDO；可有 nTRST。经 TAP 的指令/数据扫描访问调试逻辑，也用于边界扫描；不保证所有 JTAG 工具都懂 ARM 调试。
SWO / SWV	芯片 → 采集端	SWO 是单线 Trace 输出；SWV 常指利用该通路观察事件/变量的工具能力。它们不是 SWDIO 的另一个名称。
CMSIS-SVD	离线描述文件	解释外设寄存器名称、位域及访问属性；不负责 USB、SWD 或 CoreSight 探测。

“DAP”也需按上下文阅读：芯片内的 Debug Access Port 是 ARM 调试访问体系；CMSIS-DAP 是探针协议/固件规范；编辑器中的 Debug Adapter Protocol 又是另一层软件接口。本工程的 `--headless --stdio` 使用自定义 JSONL，而非编辑器 DAP。同类概念还有 ST-Link（ST 的探针及主机协议体系）和 FTDI MPSSE（可由主机驱动产生 JTAG/SWD 所需时序的 USB 引擎）。它们也不能与 SWD/JTAG 直接画等号。OpenOCD 是否支持某个适配器，取决于构建中的驱动、硬件电路和配置；当前 tools 只提供下述三条已配置路径，没有据此声明 ST-Link/FTDI 的本地验收结果。

实现依据：Arm CMSIS-DAP Overview；`src/main.rs`；`tools/bin/openocd/scripts/interface/cmsis-dap.cfg`、`tools/bin/openocd/scripts/interface/jlink.cfg`。

8.2 USB 驱动、探针固件与 Server 如何配合

这里的“驱动”至少有三种含义：Windows 设备驱动负责打开 USB 接口；Server 内的 adapter driver 实现探针命令；芯片/Flash driver 负责核控制和烧写算法。更改 `adapter driver` 不会自动替换 Windows 驱动，也不会让不支持某芯片的 Flash 算法获得支持。

本机路径	主机到探针的软件链	排错重点
GDB + J-Link Server	JLinkGDBServerCL → JLinkARM.dll → SEGGER 支持的 USB 驱动 → J-Link 固件	核对同套 Server/DLL、实际绑定驱动、探针版本。Windows 能枚举 USB 并不表示 DLL 能打开它。
GDB + OpenOCD/J-Link	OpenOCD jlink adapter → libjaylink/libusb → USB → J-Link 固件	本地分发静态链接 libjaylink。Windows 接口须匹配该 USB 后端；与上一行切换时要核对驱动绑定。
GDB + OpenOCD/CMSIS-DAP	OpenOCD cmsis-dap adapter → HID 或 USB bulk 后端 → CMSIS-DAP 固件	本地历史日志明确是 HID；不能仅从固件字符串“2.0”推断已经使用 bulk。

CMSIS-DAP v1 通常使用 USB HID 报告，Windows 可利用通用 HID 驱动；v2 配置使用 USB bulk，Windows 常经 WinUSB，命令/响应可按包批量搬运，并可选配独立 SWO 流端点。**USB 类、命令协**

议版本、产品固件版本是三个维度。设备是否提供 HID/bulk、是否实现 SWO、最大包长和速率，都应从接口描述符与实际能力查询确认。固件菜单写有 CMSIS-DAP，不等于所有可选功能均可用。

OpenOCD 的 `cmsis-dap backend auto/hid/usb_bulk` 决定主机后端，`transport select swd` 决定目标侧传输；`adapter speed 1000` 请求的是目标调试时钟，单位 kHz，不是 USB 速率或 CPU 主频。最新上游文档可能还包含本地分发未验证的其他后端，不应直接抄入项目。参考 Arm USB 配置说明和 OpenOCD adapter 文档。

本机曾因 J-Link 接口绑定为 WinUSB/libwdi 而让 SEGGER Server 报 `Could not connect to J-Link`。这是启动阶段无法打开探针的问题，和第21.3节“已经工作约 53 秒后失效”的问题应分开记录。核对设备管理器中的接口、驱动提供者、探针序号与 Server 输出，避免多个服务同时占用同一探针。若是复合 USB 设备，还需辨认调试接口与串口接口。

实现依据：`tools/bin/openocd/PROVENANCE.txt`；`tools/debug-env.toml` 与 `tools/devices/` 芯片描述；本地 0.8.0 驱动诊断及 DAP 实板报告。

8.3 SWD、JTAG 与复位信号

SWD 通过一次请求选择 DP/AP、读写方向和寄存器地址，再进行总线方向切换、ACK 与数据传输；Server/探针处理 OK、WAIT、FAULT 等响应、重试和协议错误。WAIT 不等于目标必然死机，FAULT 也不必然表示 USB 已断。实际故障需与电源、地址、访问权限及 DP 错误状态一起解释。

JTAG 使用 TAP 状态机，通过 IR/DR 扫描选择指令和移入/移出数据；多器件扫描链还需正确的 TAP 顺序、IR 长度及旁路配置。芯片内多核不等于板上必须有多个 TAP。SWJ-DP 可将共享引脚切换到 SWD 或 JTAG 模式；SWD 的新版本还存在多点连接能力，但本地 F429 profile 只使用单个 DAP，未启用或验证多点模式。

NRST/SRST 是目标系统复位，nTRST 是 JTAG TAP 复位，SWD line reset 则恢复传输状态；三者不能互换。`SYSRESETREQ` 是芯片内部的系统复位请求机制，板级“连接时保持复位”还取决于 NRST 接线、探针电路和 Server 配置。不能因为 profile 中有 `reset halt`，就假定它一定拉动了物理 NRST。

8.4 STM32F429 接线与电气边界

STM32F429 本地 profile 使用 SWD。调试信号与 UART 日志是不同链路；当前工程未提供 UART 接收面板，应用串口输出不会因为连接 GDB 自动出现在 Console。Console 的 target 通道来自 GDB/Server 输出，不能当作任意串口的镜像。

信号	方向/作用	接线与确认
VTref / Vtarget	目标 → 探针参考	使用板卡实际调试电平；参考电压不等于探针必然给板供电。
GND	公共参考	板卡与探针共地，缩短回流路径。
SWCLK	探针 → 芯片	时钟，STM32F429 常用 PA14/SWCLK；profile 配置 1 MHz 或 4 MHz。
SWDIO	双向	调试数据，STM32F429 常用 PA13/SWDIO；固件重配引脚可导致失联。
NRST	复位控制	建议板级引出；是否采用硬件复位取决于 reset_config，不能只看接线。
SWO	可选跟踪输出	STM32F429 的 PB3 可用于 SWO；本手册 profile 未启用专用 SWO 采集。

引脚功能应核对具体封装、原理图和 ST 手册；这里不推定某块板的接插件针序。调试电压、板卡供电和连接器定义以实物为准。

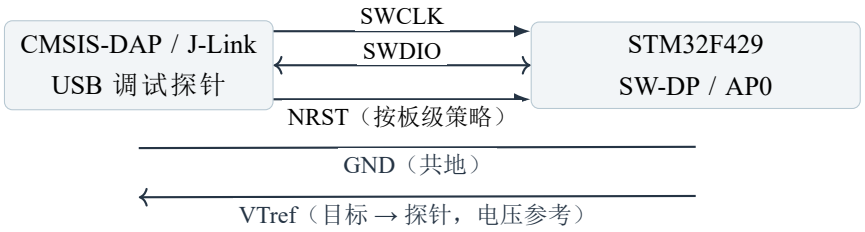


图 4: 信号关系示意。调试线中承载的访问并不等同于 CPU 执行的指令流。

F429 的 Cortex-M4、Flash、SRAM 和外设经芯片内部调试/总线路径访问。CPU target 与独立 mem_ap target 是同一颗芯片的两个软件访问入口，**不是两个 CPU，也不是两个物理 AP**。本地两者共用 DAP 的 AP0。DP ID 和芯片 ID 也不同：本地日志 DPIDR 为 `0x2ba01477`，DBGMCU 设备 ID 为 `0x20016419`。

连接不是完全无副作用的观察：本地 OpenOCD 的 stm32f4x.cfg 在 examine-end 设置低功耗调试位和看门狗暂停位；下载还会使用 SRAM 工作区。需要复现运行故障时，应把调试环境对时序和寄存器的影响纳入判断。

实现依据：`tools/bin/openocd/scripts/target/stm32f4x.cfg`: examine-end、work-area；`artifacts/dap-ui-20260920-2307/raw-openocd.stderr.log`。引脚语义另参 ST RM0090 / 数据手册。

9 ARM 调试架构：CoreSight、DAP 与 Trace

本节把探针送入芯片的访问与实际工作台功能联系起来。采用 STM32F429/Cortex-M4 所在的 ADIv5 调试模型作为主线，再解释多核 SoC 中常见的 APB-AP、CTI 和 Trace 组件。ADI 的其他版本、Cortex-A/R 或厂商集成不必具有相同寄存器、AP 编号或访问限制。

9.1 三条路径：访问、触发、追踪

CoreSight 是片上调试与追踪架构及组件体系，不是某种 USB 探针协议。它规定可发现的组件、寄存器接口以及 Trace/触发互连；具体芯片只集成其中一部分。**DAP 提供访问入口**，用于到达调试组件及系统资源；严格按 CoreSight 可见组件的定义，DAP 本身不等同于一个普通的 CoreSight 组件。

把系统分成三条路径更容易理解本工程的边界：**调试访问路径**读写寄存器、内存并控制 CPU；**交叉触发路径**传播停止、重启或采集触发事件；**Trace 路径**传送运行期间生成的追踪数据。它们可以协作，但访问成功不代表触发路由已配置，更不代表能够采集 Trace。

路径	典型组成	DebugTUI 当前参与方式
访问/配置	SWD/JTAG → DP → AP → 内存/调试寄存器	通过 GDB/Server；运行读可走 OpenOCD TCL。
事件触发	核/ETM 事件 → CTI → CTM → CTI → 动作	仅可透传显式 sync.open 配置；组控制另由软件协调。
Trace 数据	ITM/DWT/ETM → ATB/链路 → 缓冲或输出 → 采集器	未实现采集、解码或时间线 UI；与 LIVE 轮询不同。

实现依据：Arm CoreSight Architecture v2.0；`src/coordinator.rs`；`src/session/memory.rs`。

9.2 DAP、DP、AP 和片上总线

DAP（Debug Access Port）在这里指由外部 Debug Port 和若干 Access Port 组成的访问设施。**DP 面向外部链路**：SW-DP 接受 SWD，JTAG-DP 接受 JTAG；SWJ-DP 可选择二者。DP 提供访问状态、错误处理和调试电源请求等入口。**AP 面向片内资源**：Server 选择 AP 后，利用其接口访问某条总线或其他调试结构。

MEM-AP 是内存映射访问端口的通用模型。AHB-AP、APB-AP、AXI-AP 的名称说明其片内一侧连接的总线；JTAG-AP 则用于到达内部 JTAG 链，不是“外部改用 JTAG 时必然用到的 AP”。AP 编号不是 CPU 编号，也不是厂商间通用的用途枚举。

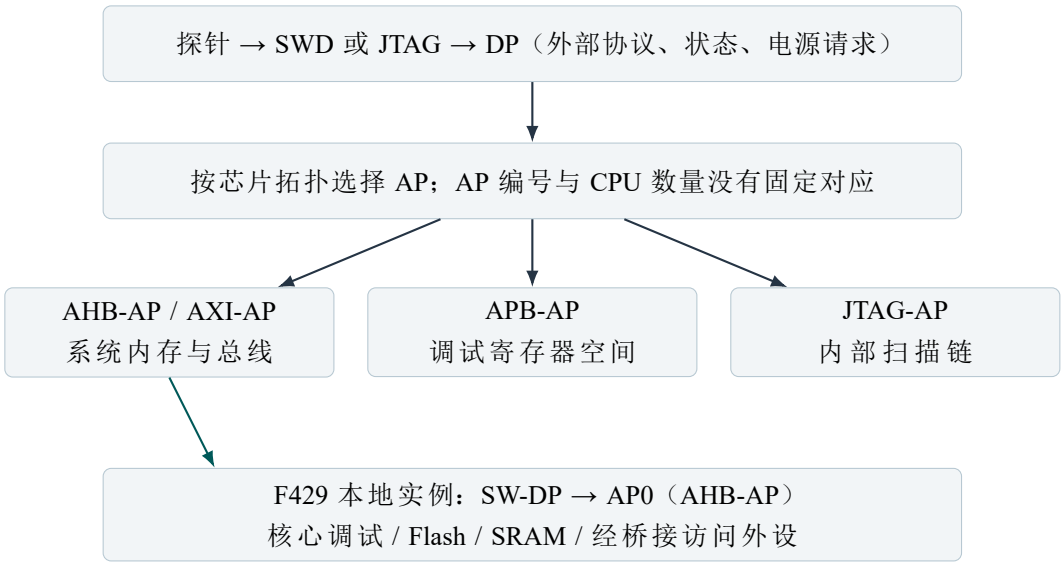


图 5: 通用 DAP 访问分类与 F429 实例。上方各 AP 为架构示意，不表示 F429 实现了三种 AP。

端口	常见用途	适配时必须确认
AHB-AP	作为 AHB 访问主设备访问系统地址空间；Cortex-M 中常见	可达地址、传输宽度、总线时钟、访问保护。
APB-AP	访问调试 APB 上的核调试、CTI、ETM 等寄存器	调试地址空间和上电状态；不假定覆盖全部系统 RAM。
AXI-AP	访问采用 AXI 的系统地址空间	位宽、地址范围、权限及缓存一致性属性。
JTAG-AP	由 DAP 转到芯片内部 JTAG 结构	内部 TAP、链路控制和特定目标支持。

名称相似也不能直接等同：STM32 的 RCC/GPIO 等外设可能位于系统 APB 总线上，但外部调试可由 AHB-AP 经 AHB-to-APB 桥到达。**“读取 APB 外设”不意味着必须存在独立 APB-AP**。在本地配置中再创建一个名为 `stm32f4x.apb` 的软件 target，也不会凭空增加硬件端口。

实现依据：Arm CoreSight basics Part 2；`tools/openocd/stm32f429.cfg`。

9.3 一次内存访问在 DP/AP 内部如何完成

以 ADIv5 MEM-AP 的概念流程说明：Server 首先识别 DP，并确认调试/系统电源请求得到响应；选择正确 AP 与寄存器 bank；设置 `CSW` 中的访问尺寸等属性，把目标地址写入 `TAR`，再通过 `DRW` 发起数据传输。AP 读存在流水/延后返回语义，需要按协议完成后续取数，例如通过 DP 的 `RDBUFF` 获取最终读结果；不能把“发出 AP 读请求”当作已经拿到该地址的值。

这些步骤由 Server、适配器库与探针固件配合完成，并可能批量化或缓存配置。DebugTUI 只提交高层内存请求，不在 Rust UI 中实现 CSW/TAR/DRW，也不能从一次 MI 响应反推出发生了多少个 SWD 时钟周期。

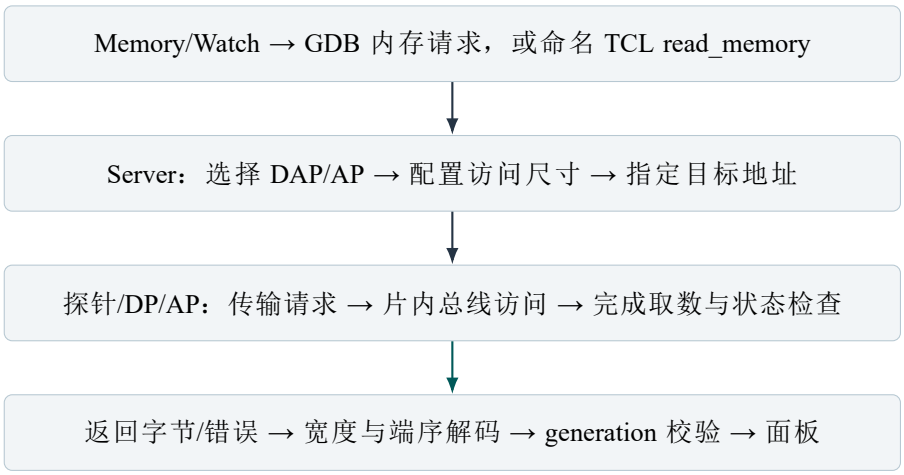


图 6: 内存请求的逻辑完成路径。批量读取、AP 流水和主机请求并非一一对应。

DPIDR 说明调试端口，AP 的 IDR 说明端口类别；MEM-AP 的 BASE 和 CoreSight ROM table 可引导工具发现内存映射组件，再利用组件/外设标识确认类型。它们与 STM32 的 DBGMCU_IDCODE、CPU 的 CPUID 不是同一个身份。软件 target 名、AP 编号、组件地址也不能相互替代。ROM table 有条目并不自动证明对应组件当前已上电、已解锁或允许访问。

发生 WAIT/FAULT 时，应区分外部传输响应和片内总线完成状态；DP sticky error 会影响后续访问，恢复由了解该协议的 Server 处理。若端口仍能识别而系统地址不可读，电源/时钟域、地址映射、复位、保护状态和 AP 路由都是可能原因。提高 TUI 的等待时间不能修复错误的硬件路由。

实现依据：Arm ADIv5.0–ADIv5.2 Architecture，MEM-AP 与 DP 章节；src/session/memory.rs。

9.4 Cortex-M4 的停核、断点与观察点

DAP 使寄存器可达；核调试逻辑决定如何暂停、单步和转移核心寄存器。Cortex-M4 常见的 DHCSR 用于调试控制/状态，DCRSR/DCRDR 配合选择和传输核心寄存器，DEMCR 包含 vector catch 及追踪相关控制。GDB 中看到的 R0、PC、SP、xPSR 不应简单理解为同名普通 RAM 地址。

资源	工作职责	在 DebugTUI 中的表现
Core debug	暂停、继续、单步、核心寄存器传输	Pause/Step 和 System Regs；完成状态由 Server/GDB 确认。
FPB	Flash Patch and Breakpoint，指令地址匹配等	硬件代码断点；本机 Server 报告 6 个硬件断点资源。软件/Flash 断点策略仍取决于服务端。
DWT	Data Watchpoint and Trace，地址比较、计数及可选事件	数据观察点通常消耗比较器；本地报告 4 个观察点资源。Watch 表达式列表本身不分配 DWT。
ITM / ETM	软件插桩事件 / 执行流追踪来源	当前没有对应的采集解码面板；详见第9.7节。

DWT 可以同时承担观察点与追踪事件相关任务，因此资源配置可能互相影响；“芯片有四个比

较器”不代表每种组合都有四个任意条件槽位。停止后的寄存器快照还需使用正确线程/栈上下文。FreeRTOS 调试中看到 PSP/MSP 的区别，不等于 TUI 已实现 RTOS 任务枚举。

实现依据：Arm Cortex-M4 TRM；本地 OpenOCD 探测日志；`src/session.rs`、`src/session/breakpoints.rs`。

9.5 F429 的两个软件 target 与一个 AP0

在本地板级脚本中，`stm32f4x.cpu` 是 Cortex-M CPU target；`stm32f4x.ahb` 是独立的 `mem_ap` target。两者引用同一个 `stm32f4x.dap`，且 AP 均为 0。前者提供停核、寄存器、断点和 GDB 服务，后者给 TCL 运行时内存读提供独立入口，并关闭自己的 GDB 端口。

因此，`channel="ahb"` 是 TUI 命名通道到 OpenOCD target 的映射，不是“切换到第二颗 CPU”。读取 RCC 外设走 AHB-AP 到芯片总线，读取 RAM 也走同一实际 AP；是否能在运行中访问、读到的数据是否一致，仍由目标状态和总线系统决定。`while_running=true` 只声明并开放已有能力，不会解除芯片保护或给 AP 上电。

`targets stm32f4x.cpu` 在本地脚本初始化时设置默认 CPU；实时读取使用 `stm32f4x.ahb read_memory ...`，不不断切换全局 target。CPU 与 MEM-AP 访问共享物理资源，故高频轮询可能增加总线和探针负载。对更复杂的 A/R 多核 SoC，核内缓存、MMU、调试 APB、电源域和独立 AP 还会增加限制，不能直接套用 F429 的地址与 AP0。

实现依据：`tools/openocd/stm32f429.cfg` 和 `tools/openocd/probes/`；`tools/debug-env.toml`；`src/live_watch.rs`: transact。

9.6 CTI、CTM 与硬件交叉触发

CTI（Cross Trigger Interface）把本地触发输入映射到触发通道，再把选中的通道映射到本地输出；CTM（Cross Trigger Matrix）在多个 CTI 之间传播通道事件。ECT（Embedded Cross Trigger）是这套交叉触发设施的统称。例如核 0 进入调试态，经路由请求核 1 停止，或令 Trace 采集器在某个事件附近保留数据窗口。

本工程的软件联停：核 0 的 MI 停止事件 → Coordinator → 核 1 的 GDB Pause



下行是芯片硬件路径，需核对事件编号、方向、时钟/电源、使能及确认条件。

图 7: 软件联停与硬件交叉触发各有独立路径；通道号不是 AP 号或 TUI 核索引。

适配 CTI 时先从芯片 TRM/集成资料确认 CTI 基址、触发输入输出接线、通道数、时钟与权限，再配置输入映射、通道门控和输出映射；对保持型事件还须遵循清除/确认规则。错误映射可能导致事件无法传播或反复触发。硬件联停延迟受实际拓扑影响，也不能笼统承诺所有核同周期停止。

本工程 `[sync]` 中的 `method="cti"` 是兼容配置名，与 `method="tcl"` 一样执行 `sync.open` 中明确给出的 TCL 命令。它不发现 CTI、不自动分配通道、不生成寄存器值。`halt_peers` 是收到停止事件后的软件协调；`startup_order` 是会话启动顺序；二者都不会建立 CTI 硬件连接。本地单核 F429 profile 未配置 CTI，不据此推断芯片中存在某组可用 CTI。

实现依据：Arm CoreSight basics Part 1：Cross Triggering；`src/coordinator.rs`、`src/coordinator/control.rs`；`docs/examples/multicore-access.md`。

9.7 ETM、ITM、DWT、ATB 与 Trace 接收端

调试快照回答“现在停在哪里”，Trace 则尝试保存“到达这里之前发生了什么”。硬件 Trace 由事件源持续生成编码流；主机随后结合正确的程序镜像和解码规则还原执行信息。它不等同于 TUI 每 100 ms 读一次变量，也不等同于 GDB 日志或 UART printf。

组件	作用	常见误解与限制
ETM	Embedded Trace Macrocell；观察处理器执行，生成压缩追踪流	ETM-M4 提供指令追踪，不能据此宣称记录了所有内存数据访问。是否存在及资源数量取决于芯片。
ITM	Instrumentation Trace Macrocell；接收软件 stimulus 写入及相关硬件事件	常用于轻量软件事件；插桩本身有成本，需要正确时钟、使能与输出配置。
DWT	比较器、周期计数、采样或异常等事件来源	可为 ITM/Trace 提供事件；并非一条无限带宽、无丢失的全内存访问记录。
ATB	AMBA Trace Bus；在片上传送带来源标识的 Trace 数据	不是 APB 控制总线；经 APB 可配置组件，但 Trace 字节走另一条数据路径。
Funnel / Replicator	多路汇聚 / 一路分发到多个去向	需要有效来源 ID、路由和带宽；不负责解释指令含义。
TPIU	Trace Port Interface Unit；格式化并输出到外部采集口	是输出设施，不是存储全部历史的内存；支持的串行/并行模式与芯片实现有关。
ETB / TMC	ETB 片上 Trace 缓冲；TMC 可采用 ETF 缓冲或 ETR 路由到系统内存等配置	是通用 CoreSight 示例，本地 F429 手册不假定这些组件存在；容量、溢出和停止条件必须明确。
外部采集器/解码器	接收 SWO 或并行 Trace，保存并解码	普通 SWD 调试能力不保证拥有 Trace 引脚、采样带宽、软件许可或对应解码器。

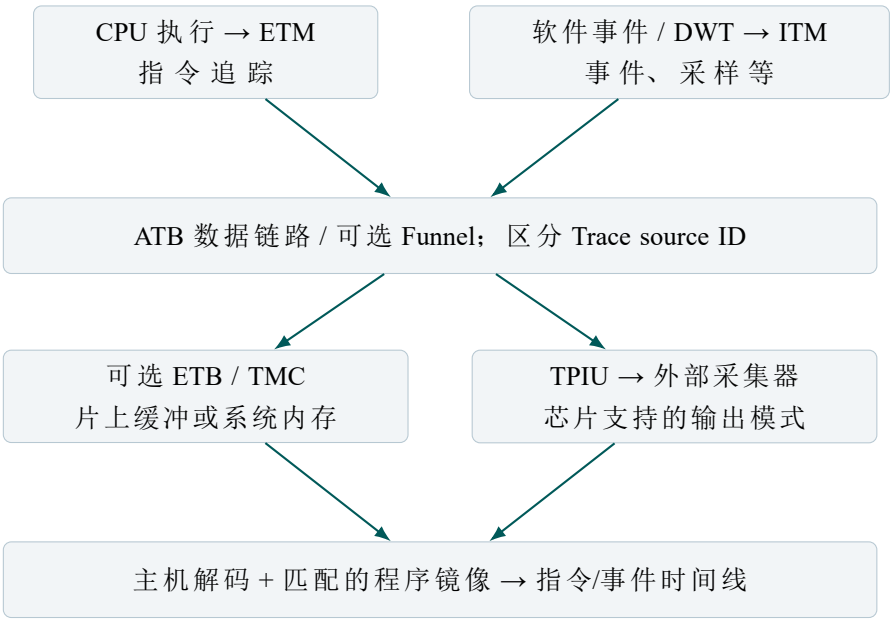


图 8: 通用 Trace 数据流。图中可选缓冲与输出不是 F429 的硬件清单，也不是当前 DebugTUI 的实现。

Funnel/Replicator、缓冲、TPIU 组成数据通路；CTI/CTM 传递触发事件；DAP/AP 访问配置寄存器。这三条路径在采集任务中协作：配置来源和路由、启动接收端、使能来源、运行到触发条件、停止并排空缓冲，再解码。不同实现的启动/停止顺序须遵循对应手册，不能用一串通用地址写操作代替。

Trace 数据量可能超过引脚、USB 或主机处理能力。采样频率、来源筛选、缓冲容量、溢出标志、时间戳来源与时钟变化都影响可解释性。缺失区间不能用“图上连续”来掩盖；多个来源即使有时间戳，也需确认同一时间基准。ETM 解码依赖正确的执行镜像；固件重新下载后继续使用旧 ELF，可能得到错误的源码或执行路径。

实现依据：Arm ETM-M4 TRM；Arm CoreSight basics Part 3；Arm TMC TRM。

9.8 将通用架构落到 STM32F429 与本工程

STM32F429 的具体调试能力应从 RM0090 的 Debug support 章节及所用封装数据手册核对。PB3 可复用 JTDO/TRACESWO，因此本芯片异步 SWO 与 JTAG TDO 的复用关系必须考虑；SWO 并非额外占用 SWDIO。并行 Trace 使用 TRACECLK/TRACED 等独立信号，是否引出及能否使用取决于封装、板卡布线和其他外设复用。

若从当前 SWD 改为 JTAG，F429 的相关复用信号为 PA13/JTMS、PA14/JTCK、PA15/JTDI、PB3/JTDO、PB4/NJTRST。并行 Trace 的常见分配为 PE2/TRACECLK、PE3–PE6/TRACED0–3；这些是芯片信号映射，不能当成本机板卡连接器已经布线的证明。例如 ST 的 STM32429I-EVAL 板手册还列出了与存储器信号复用时的板级连接条件，本地测试板应查自己的原理图。

对 Cortex-M4/F429 的使用，SWO 主要用于 ITM/DWT 串行事件通路；ETM 执行追踪应核对并行 Trace 端口和具有相应采集能力的设备，不能把“探针支持 SWO”推导成“能完整采集 ETM”。SWO 接收时钟/波特率也不是 SWD 频率；修改系统时钟后，采集端需要匹配实际 Trace 时钟。参考 ST RM0090，第 38 章与具体器件数据手册。

当前工程能力	与架构的实际对应
暂停、单步、断点、寄存器	经 GDB/Server 使用 CPU 调试与 FPB/DWT；已包含本地实板证据。
ahb 运行读	经 TCL 命名 target 访问 AP0；离散内存采样，不保存所有中间变化。
SVD Peripherals	以 XML 描述解释内存映射寄存器；不自动发现 CoreSight 拓扑。
多核联停 / sync.open	软件组协调 / 显式环境命令入口；需要板级 CTI 验证才能声明硬件联动。
SWO / ETM / RTT	当前没有专用采集/解码 UI。RTT 是目标 RAM 缓冲的数据通路，也不是 ETM 或 SWO 的别名。

实际移植应先验证调试访问，再验证运行时读取或 CTI，最后单独验收 Trace 的完整采集链。即使外部工具能显示 SWV，也不意味着 DebugTUI 已接入该数据源。后续工具配置只建立本工程已实现的 GDB/TCL 路径，避免把芯片可选能力混入现有 profile 的承诺。

10 tools：GDB 与 OpenOCD

10.1 三个 profile 的差异

下表均为仓库实际配置。三者复用 ARM GDB、GDB 端口 3333 和 TCL 6666，telnet 禁用，仅监听本机。同一端口不能由两个服务同时占用。SEGGER J-Link GDB Server 不再随附，见第10.4节。

项目	OpenOCD/J-Link	OpenOCD/CMSIS-DAP	OpenOCD/ST-Link
profile 文件	`debug-env.toml`	`debug-env.toml`	`debug-env.toml`
服务程序	openocd	openocd	openocd
探针驱动	OpenOCD jlink	OpenOCD cmsis-dap	OpenOCD st-link（dapdirect）
芯片/接口	stm32f4x / SWD	stm32f4x / SWD	stm32f4x / SWD
速度	1000 kHz	1000 kHz	1000 kHz
TCL 实时读	ahb / core-tcl	ahb / core-tcl	ahb / core-tcl
退出前动作	monitor resume	monitor resume	monitor resume
本地证据	0.8.0/F429 运行读取等实板通过	2026-09-20 最新本地实板资料完整	仅配置解析，尚未上板

10.2 GDB 最小运行环境

仓库 GDB 的工具链发行标识为 GNU Tools for STM32 13.3.rel1.20240926-1715。调试器版本为 GDB 14.2.90.20240526-git。profile 不只指定 EXE，还设置 data-directory、debug-file-directory、auto-load 路径，移除继承的 PYTHONHOME/PYTHONPATH；这些安排用于该工具分发，不代表任意 GDB 都必须采用相同目录结构。

```
[gdb]
executable = "./bin/gdb/bin/arm-none-eabi-gdb.exe"
args = [
  "--data-directory=${profile_dir}/bin/gdb/arm-none-eabi/share/gdb",
  "-iex", "set debug-file-directory ${profile_dir}/bin/gdb/lib/debug",
  "-iex", "set auto-load scripts-directory ${profile_dir}/bin/gdb/arm-none-eabi/share/gdb/auto-
    load",
  "-iex", "set auto-load safe-path ${profile_dir}/bin/gdb/arm-none-eabi/share/gdb/auto-load"
]
unset_env = ["PYTHONHOME", "PYTHONPATH"]
init = ["set remotetimeout 5", "set tcp connect-timeout 5"]
```

核心追加 `-nx -q --interpreter=mi2`，关闭分页与交互确认，尝试开启 mi-async。GDB 自身资源或 DLL 问题应先在独立命令行复现；TUI 不补造目标能力。

10.3 OpenOCD 服务和目标动作

仓库分发 DebugTUI 补丁版 OpenOCD：上游 `d3ebb8d` 加 `third_party/openocd-adapter` 的 ARMv8-R 寄存器适配，版本串为 `0.12.0+dev-gd3ebb8d-dirty`（2026-10-06）。它保留上游调试功能并提供 R52 寄存器专用协议，STM32 与 R52 工程共用；只完成构建、协议命令和配置解析的软件验证，未连接探针或板卡。配套 libusb、HIDAPI DLL 和精简后的 upstream scripts（STM32 系列 target、J-Link/CMSIS-DAP/ST-Link 接口与通用辅助脚本，共 40 个，内容未修改）已纳入工具校验清单；GPL 对应源码随 v0.10.0-readonly.1 Release 提供。其他芯片的脚本需从对应 OpenOCD 版本或厂商获取。

```
[service]
command = "${tools_dir}/bin/openocd/bin/openocd.exe"
args = ["-s", "${tools_dir}/bin/openocd/scripts",
        "-f", "${probe_config}",
        "-f", "${tools_dir}/openocd/stm32f429.cfg"]
ready = ["Listening on port 3333 for gdb connections"]
timeout_ms = 15000

[target]
mode = "extended-remote"
endpoint = "127.0.0.1:3333"
after_connect = ["monitor halt", "maintenance flush register-cache"]

[actions]
restart = ["monitor reset halt", "maintenance flush register-cache"]
run = ["monitor reset halt", "maintenance flush register-cache",
       "-exec-continue"]
download = ["-target-download", "monitor reset halt",
            "maintenance flush register-cache"]
before_disconnect = ["monitor resume"]

[session]
on_exit = "detach"
```

这是 STM32 芯片描述的配置示意；公共入口为 `.vscode/debug-env.toml`，芯片参数放在 `.vscode/devices/stm32f429.toml`。三种探针共用 `.vscode/openocd/stm32f429.cfg`，只修改公共入口的 `probe.config` 选择 `openocd/probes/` 下的驱动。实际逐核端口和 ready 由芯片描述的 core_targets 提供。ready 是输出子串就绪条件，不代表符号已加载或应用完成启动；之后仍需 GDB 连接和实际状态查询。

10.4 自行安装的 J-Link Server

tools 不再包含 SEGGER J-Link 软件：其许可证要求每次再分发都事先取得 SEGGER 书面授权。需要 SEGGER Server 时，从 SEGGER 安装 J-Link Software，把 `docs/examples/jlink-gdb-server.toml` 复制到 tools 目录，并把 `service.command` 改为本机安装路径。该 Server 没有 OpenOCD TCL 端口，Memory access、Live refresh 和寄存器专用读取不可用。

本地 J-Link Server 路径有普通调试成功记录，也有约 52–53 秒后的长会话访问失效记录。使用前应同时阅读第21.3节；短程连接成功不能代替长会话稳定性验证。

```
[service]
command = "C:/Program Files/SEGGER/JLink/JLinkGDBServerCL.exe"
args = ["-device", "STM32F429IG", "-if", "SWD",
        "-speed", "4000", "-port", "3333", "-select", "USB",
        "-localhostonly", "-nogui", "-halt", "-singlerun"]
ready = ["Waiting for GDB connection", "Connected to target"]
timeout_ms = 15000
```

该环境的 restart 依次执行 reset、halt 和刷新寄存器缓存。run 在此基础上继续，download 首先 `-target-download`。`-singlerun` 使 Server 在客户端断开后自行结束，外部服务也可能因此退出，不意味着 TUI 杀死了它。参数语义见SEGGER 官方 GDB Server 文档；本地旧版使用 `-select USB`，不要直接照搬新版本专用参数。

10.5 手动启动与所有权

```
# Installed project root, terminal 1:
./.vscode/bin/openocd/bin/openocd.exe `
  -s ./.vscode/bin/openocd/scripts `
  -f ./.vscode/openocd/probes/cmsis-dap.cfg `
  -f ./.vscode/openocd/stm32f429.cfg

# Terminal 2, retaining this profile's GDB arguments and actions:
debugtui --environment ./.vscode/debug-env.toml `
  --chip stm32f429 --cores 0 `
  --connect 127.0.0.1:3333 --elf <matching.elf>
```

外部启动的服务由启动者管理。TUI 的工作区重建、任务执行或退出只终止自己启动的服务/GDB/子任务。OpenOCD 在 GDB 断开后继续运行，由启动者用 Ctrl+C 结束。

实现依据：`tools/debug-env.toml`；`tools/devices/stm32f429.toml`；`docs/examples/jlink-gdb-server.toml`；`tools/bin/openocd/PROVENANCE.txt`。

11 STM32F429 本地实板配置与操作

11.1 确认本机实际型号和固件

本机服务配置指定 STM32F429IG，关联固件链接脚本为 `STM32F429IGTX_FLASH.ld`。历史 OpenOCD 实测报告 Cortex-M4 r0p1、1024 KiB Flash、6 个代码断点资源与 4 个观察点资源。这些是此板此环境的报告值，不是 DebugTUI 写死的限制。

项目	核对结果
芯片基础	STM32F429 系列 Cortex-M4 + FPU；官方系列最高 180 MHz。本手册不以此推定该固件的实际运行频率。
主 Flash	链接 ORIGIN=0x08000000，LENGTH=1024K。
主 SRAM	链接 ORIGIN=0x20000000，LENGTH=192K；栈顶 0x20030000。
CCM RAM	链接 ORIGIN=0x10000000，LENGTH=64K。声明区域不代表全部被使用或可经任意 AP 路径访问。
程序文件	`build/FreeRTOS_STM32F429.elf`，不要误选历史 `Debug/FreeRTOS_Project.elf`。
编译参数	`-mcpu=cortex-m4 -mthumb -mfpu=fpv4-sp-d16 -mfloat-abi=hard -O0 -g3`。
RTOS 配置	FreeRTOS 抢占式；tick 1000 Hz，heap 配置 15360 字节。
本地 DAP 探针	历史报告 ATK-FS-HID-CMSIS-DAP，固件 2.0.0，SWD 1000 kHz。

芯片系列概况参ST STM32F429IG 产品资料；本表容量取自本地链接脚本和历史设备识别，不将系列“最大容量”误写为此板容量。主 SRAM 与 CCM 应分别看待，不应把它们当作从 0x20000000 开始连续的 256 KiB。

11.2 可复制的项目配置

以下配置以固件工程目录为根，先安装工具到 `.vscode`，在公共入口选择 CMSIS-DAP。芯片选择为 STM32F429/Core0；这是手册示例，本文未改写用户工程。

```
version = 3
watch = ["xTickCount", "uwTick"]
breakpoints = ["main"]


[tools]
profile = "./.vscode/debug-env.toml"
[debug]
chip = "stm32f429"
cores = [0]
[program]
elf = "build/FreeRTOS_STM32F429.elf"
source_root = "."
# SVD follows Chip from .vscode/svd/; no fixed project override.
[tasks]
build = "mingw32-make -j8"
download = ""
timeout_ms = 300000
[session]
log_dir = "debug-logs"
on_exit = "detach"
timeout_ms = 8000
```

`download = ""` 使下载回退至 profile 的 GDB action。旧工程如果仍保留启动 J-Link 的外部下载命令，它会优先于新的 CMSIS-DAP profile，必须检查清楚。Build 依赖工程自己的 GCC/make 和 PATH；tools 的最小包只含调试器/Server，不包含完整固件构建工具链。该 Makefile 使用 which、ls 等工具查找 GCC，构建环境还需提供对应命令或适配工程脚本。

11.3 板级 OpenOCD 脚本

以下为共用 `.vscode/openocd/stm32f429.cfg`；探针驱动先由独立的 `openocd/probes/*.cfg` 加载：

```
bindto 127.0.0.1
gdb port 3333
tcl port 6666
telnet port disabled
transport select swd
source [find target/stm32f4x.cfg]
adapter speed 1000
target create stm32f4x.ahb mem_ap \
  -dap stm32f4x.dap -ap-num 0 -endian little
stm32f4x.ahb configure -gdb-port disabled
targets stm32f4x.cpu
```

`stm32f4x.cfg` 建立 DAP、cortex_m CPU target、stm32f2x Flash 驱动、OTP bank 和 32 KiB SRAM 工作区。该 Flash 驱动名用于 STM32F4 是脚本本来的选择，并非配置误写为 F2。下载可使用工作区 `0x20000000`、大小 `0x8000`，且 backup=0；不要期待下载前 RAM 状态被保留。

脚本的默认软复位路径是 SYSRESETREQ；实际 NRST 行为仍须按 reset_config 和板卡连线核对。examine-end 会设置 `DBGMCU_CR` 低功耗调试位和 `DBGMCU_APB1_FZ` 看门狗停止位。`adapter speed 1000` 写在 target 文件之后，覆盖其默认速度。`targets stm32f4x.cpu` 只在板级初始化中选择 CPU；TUI 后续内存轮询不会不断切换全局 target。

11.4 两个命名内存通道

```
[[memory_access]]
id = "ahb"
label = "AHB MEM-AP 0"
tcl_endpoint = "127.0.0.1:6666"
target = "stm32f4x.ahb"
while_running = true

[[memory_access]]
id = "core-tcl"
label = "Core target (stopped)"
tcl_endpoint = "127.0.0.1:6666"
target = "stm32f4x.cpu"
while_running = false
```

此处 while_running 是配置能力声明，TUI 会按它放行或拒绝；声明为 true 并不能让不支持总线访问的硬件变得可用。未指定 channel 的 GDB 路径始终停止限定。

11.5 从连接到运行的实操

1. 在 Setup 选择上述配置，连接后检查 CPU 识别与 STOPPED。若只出现 Server 监听端口，尚不能断言整个会话成功。
2. Console 执行 `compare-sections .text`，核对匹配后才解释源码/变量。历史曾遇到旧 Debug ELF 与板上固件不同，换成 build ELF 后 matched。
3. 在 `Task100ms` 或实际 `user_task.c` 的调用行设置断点。F5 命中后检查 Stack/Locals；Step In 可进入 `Led_Task`，Step Out 返回调用者，Step Over 越过调用。具体行号随固件版本变化。
4. 添加 `xTickCount`、`uwTick`；先在停止时解析，再对 xTickCount 选 ahb、100 ms 并继续。LIVE 数值应变化，停止代次不会因读总线递增。
5. 暂停并 Refresh，将 GDB、core-tcl、ahb 的稳定 RAM 值作对照。对正在自主变化的外设或 DMA 数据，不要求两次读绝对相同。
6. 如需烧录，先 Build，确认 ELF 和 profile 后点击 Download 并确认；完成后用 `compare-sections` 复核全部可加载段。
7. Ctrl+Q 退出；本 profile 在退出前 resume、detach，目标预期恢复运行，仍应以实际设备状态为准。

已用于对照的寄存器	地址	读取解释
RCC.CR / RCC.CFGR	0x40023800 / 0x40023808	时钟控制/配置；时钟调整时数值可变化。
GPIOB.MODER	0x40020400	GPIO 模式，适合核对 SVD 位域与原始读。
DBGMCU.IDCODE	0xE0042000	本地历史读 0x20016419；不能与 DPIDR 混淆。

SVD 文件版本 1.9，含 84 个外设描述，原文件 2118594 字节。其描述覆盖范围不等于全部寄存器均已实板读写验证。`xTickCount` 的地址应由当前 ELF 求得，不能固定抄录某次测试的地址。

实现依据：关联固件 `debug.toml`、`Makefile`、`STM32F429IGTX_FLASH.ld`、`Core/Inc/FreeRTOSConfig.h`、`BSP/USER_TASK/Src/user_task.c`；`resources/svd/stm32/catalog.json`；第22节证据表。

12 THA6206：GHS 单核与多核适配

12.1 参考工程、路径与证据边界

本节沿用 STM32 章节的“工具、连接、下载、执行、数据、验收”顺序，但采用 THA6206 的板级脚本。芯片为双 Cortex-R52，调试配置使用 Armv8-R、AArch32；不能直接替换 STM32 profile 的芯片名称后使用。

本文用以下根目录简称，后续路径均相对对应根目录。本机实际 MCAL 目录名称为 `THA6XXX_MC_AS440`，用户描述的 `THA6XXX/_MC/_AS440` 嵌套路径未找到，应以实际存在目录为准。

根目录简称	本机路径
T：DebugTUI	`D:/LiBo/Files/DATA/FreeRTOS/vscodegdb/debugtui`
M：GHS/MCAL 工程	`D:/LiBo/Files/DATA/FreeRTOS/vscodegdb/THA6XXX_MC_AS440`
B：Bao 工程	`D:/LiBo/Files/DATA/FreeRTOS/BAOHyp/bao-hypervisor`
S：Bao 引用的 SDK	`D:/LiBo/Files/DATA/FreeRTOS/BAOHyp/THA6XXX_MC_AS440`
A：Bao 实板记录	`D:/LiBo/Files/DATA/FreeRTOS/BAOHyp/analysis`

M 和 S 是两个独立目录，不应因为名字相同就视为同一份工具或固件。B 的 profile 通过相对路径引用 S；迁移时如改用 M，需要同时核对 GDB、OpenOCD、SVD、下载器及工作目录。M 基于 `50ff448`，B 基于 `b25150e` 的 `dev/tha6xxx-init` 分支；两者均存在本地修改，底层移植分析来自 2026-09-21 可见工作树；本节的芯片目录和 Bao 启动/构建更新另按 2026-10-02 当前文件核对。不能只 checkout 这两个提交就假定得到全部示例。

此前章节资料包含配置静态核对、历史实板和离线 ELF/GDB 检查；2026-09-23 的 MCAL 实板回归见第12.8节。它实际下载了配套 HEX，不覆盖 Bao 固件。9 月 29 日 Bao smoke 单/双 GDB 与 10 月 28 日 1.3.0 的 dual 多 ELF 分别验收；最新范围见第13.11节。

12.2 与 STM32F429 适配的关键差异

项目	STM32F429 参考方案	THA6206 参考方案
处理器与启动	单 Cortex-M4；向量表、MSP、Thumb	双 Cortex-R52；AArch32，需理解 HYP/SVC、共享初始化及从核等待。
工具链	Arm GNU 与对应 STM32 ELF	MCAL 使用 GHS；Bao 使用 GNU。不能把 GHS 的 DWARF workaround 无条件套到 Bao。
OpenOCD	STM32 target/Flash 驱动	厂商分发的 0.12.0+dev、`armv8r` target、专用复位脚本；不能假设任意上游包等价。
访问路径	CPU target 与 AHB-AP0	CPU 调试经 AP1；独立 `AHB_3` 经 AP3 读总线。
下载	可由 GDB load 配合 Flash bank	现有 cfg 无 PFlash bank；外部 `jtag.exe --flash` 下载 HEX。
执行与复位	一条 GDB 会话控制一个核	单核入口与共享 Server 的双 GDB 入口并存；`chipreset` 是整片复位。
运行时观察	明确配置的 MEM-AP 通道	GMU/外设用 AP3；物理地址、别名、MPU 和缓存语义需要单独核对。

历史测试工具包括 GHS 2023.1.4、Arm GDB 16.3、厂商 OpenOCD 和 CMSIS-DAP，板级脚本设适配器时钟 5000 kHz。实际启动时应检查 Console 的版本、DPIDR、核识别和端口信息，而不只检查可执行文件存在。

12.3 AP、CPU target、CTI 和存储地址

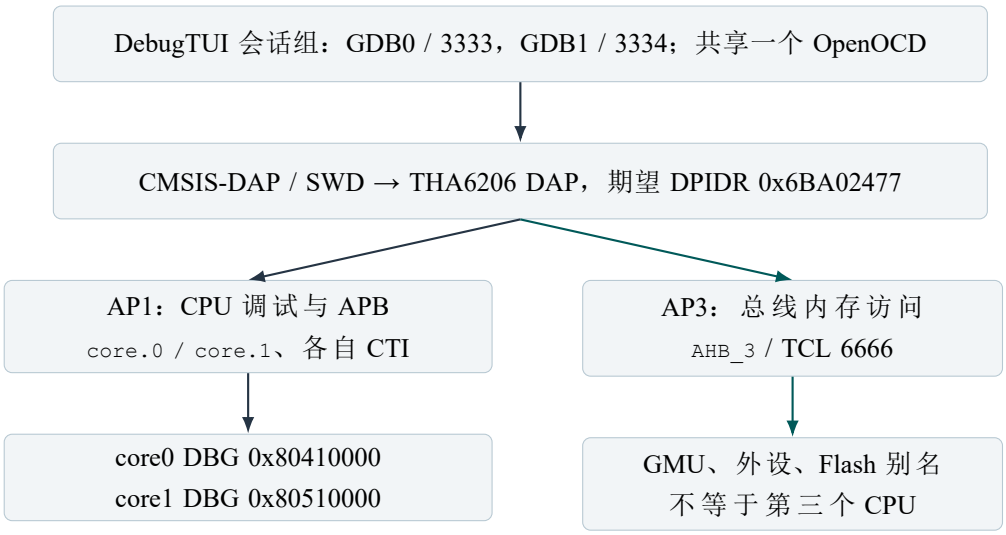


图 9: THA6206 的执行控制与后台读内存走不同 target。

`.vscode/openocd.cfg` 创建 `core.0`、`core.1` 两个 `armv8r` target，均绑定 AP1。CTI 基址分别为 `0x80420000`、`0x80520000`；另建 `APB_1` 和 `AHB_3` MEM-AP target。配置中 `DEBUGCORE` 是位掩码：1 选择 core0，2 选择 core1，3 选择两个核；未选核使用 defer-examine，并非从 target 列表删除。因此 core1

的 GDB 端口仍是 3334。

CTI 对象参与 Armv8 target 的运行控制，但仅创建并绑定 CTI，并不证明已经设置跨核硬件同步路由。当前 TUI 的组控制、断点联停属于软件协调，不能据此声称两核在同一周期停止。

区域/用途	本地工程地址	调试时的含义
CPU PFlash0 / 1	0x08000000 / 0x08300000，各 3 MiB	ELF 执行/源码断点地址；当前 GDB cfg 把整个 6 MiB 区间声明为只读。
AHB PFlash0 / 1 别名	0x90400000 / 0x90700000	总线侧验证镜像时使用；不可拿来作为 CPU C 函数断点地址。
GMU RAM	0x10000000，256 KiB	普通 RAM 地址；设备别名基址为 0x82000000。
Flash 控制器	0xC0100000	控制寄存器，不是固件镜像起始地址。

TCM 还有核私有地址与全局别名。对 core0/core1 的私有 TCM，不能仅因表达式地址相同就默认 AP3 读到了当前核的数据。运行中 CPU cache 内的新值也不一定立即等于总线看到的值；访问通道选对不代表获得了缓存一致性快照。

实现依据：M：`.vscode/openocd.cfg`、`.vscode/gdb.cfg`、`DemoWorkspace/CodeProject/LinkFiles/Ghs/LinkerGhsPf.ld`；OpenOCD 官方 Armv8/CTI 命令说明。

12.4 工具环境、构建和下载

M 的四个工程入口分别引用工具环境：debug.toml 对应 debug-env.toml，debug-core0.toml 对应 debug-env-core0.toml，debug-core1.toml 对应 debug-env-core1-standalone.toml，debug-multi.toml 对应 debug-env-multi.toml；这些 profile 都在.vscode 下。另保留 debug-env-core1.toml 用于附加外部服务。工程 ELF 为 `DemoWorkspace/CodeProject/Debug/THA6206_Demo_Prj.elf`，Source root 为工程根，SVD 为 `.vscode/THA6206/tha6206.svd`。GDB 环境载入 `gdb.cfg`，设置 `armv8-r`、可访问存储范围和超时；`maintenance set dwarf unwind off` 是当前 GHS 调试兼容配置。

```
# Run these commands from M.
.\DemoWorkspace\CodeProject\Build.bat ghs rebuild tha6206
.\.vscode\Load.bat tha6206

debugtui --project .\debug-core0.toml
# Alternative entries, one debug workspace at a time:
debugtui --project .\debug-core1.toml
debugtui --project .\debug-multi.toml
```

`Load.bat` 使用 Debug 目录的 `THA6206_Demo_Prj.hex`，调用厂商下载器烧录并执行系统复位。TUI Build/Download 调用相同的外部任务，先释放已有 GDB 和其拥有的 Server，再执行任务；原来已连接的工作区在成功后重新连接。不要同时运行多个入口争用同一探针/端口，也不要把 Console 中的 `load` 当作这个 HEX 下载流程。

工具 profile 显式列出 `r0–r12`、`sp/lr/pc/cpsr`，避免初始化早期请求不适用的 VFP 寄存器。若 DSCR/sticky abort、读寄存器或复位失败仍出现，应保留原始日志排查；减少请求范围不是忽略真实错误。SVD 能显示外设名不代表对应硬件已初始化。

12.5 单独调试 core0 与 core1

配置	core0：`debug-core0.toml`	core1：`debug-core1.toml`
Server 选择	DEBUGCORE=1	DEBUGCORE=2
GDB 端口	127.0.0.1:3333	127.0.0.1:3334
Run 动作	chipreset、刷新寄存器缓存、临时断点 `_main`、继续	仅继续当前 core1
Restart 动作	chipreset、刷新寄存器缓存	空；不主动发整片复位
前提	用作主核启动/初始化入口	已有有效固件，core0 已完成必要共享初始化并释放从核

1. 先确认板上 HEX 与当前 ELF 同一次构建。启动 core0 入口，检查 core0 状态和 PC；Run 应到达 `_main`，再验证所需 C 函数的断点、栈和单步。
2. 测 core1 前，让固件按板级启动流程进入可运行状态。仅以 DEBUGCORE=2 连接，并不能替代 core0 的时钟、共享 RAM 和多核屏障初始化。
3. 用 core1 入口连接 3334，在 `DemoApp_MainCore1` 等实际从核函数设置断点并继续。若停在等待循环，应先核对主核状态与同步变量，不应立即归因于 TUI。
4. 单核入口只表示一个 GDB 会话。退出时 detach 不保证一个原本暂停的核自动继续；需要固件保持运行时，应先明确 Continue 并确认目标状态。

`chipreset` 通过 AP3 发调试模块复位请求，通过 AP1 访问 EDECR/EDPRCR 发应用复位请求，并包含重新 examine 和累计调试错误清理。不要把它替换为未经验证的通用 `reset halt`，也不要复制 core0 的复位动作到只想附加观察 core1 的入口。

12.6 本地板级复位恢复与 CSE 位修正

2026-09-22 核对的 M 下 `.vscode/openocd.cfg` 包含以下适配及中文注释。它是**外部参考工程的板级修改**，不是 DebugTUI Rust 代码或主程序包的内置复位实现；复制 TUI EXE 不会自动更新该文件。

步骤/修改	作用与实际范围
增加 `dbgreset`	经 AHB_3 向 `0xC00008A0` 写 `0x1`。本地 SVD 对应 MAINRESET.DBGRSTCON.DBGRST（bit0），写 1 触发并自清零；该请求不受 DEBUGCORE 单核选择限制。
首次执行标志	每次加载 cfg 初始化 `_dbgreset_done=0`，首次 chipreset 执行 dbgreset 并等待 300 ms，再按 DEBUGCORE 掩码 arp_examine。只重连 GDB 不清除此标志。
删除复位前 resume	不再要求先成功恢复所有 halted 核，避免调试状态异常时 resume 报错阻断后续恢复。
保留应用复位序列	APB_1 写 core0 EDECR/RCE、EDPRCR，再关闭 RCE；增加 100 ms 等待后，遍历全部核，对 running 核执行 halt。
修正 EDRCR.CSE	每核 DBGBASE + `0x090` 写 `0x4`，对应 CSE bit2；旧 `0x1` 写入的是保留位。此循环遍历全部核，不按 DEBUGCORE 掩码过滤。
恢复 target 选择	正常结束后恢复调用方原先选中的 OpenOCD target；不代表恢复各核原运行状态或刷新每个 GDB 的缓存。

两个寄存器的写入值不能混淆：DBGRSTCON 的 `0x1` 仍正确；应改为 `0x4` 的是 EDRCR.CSE。Cortex-R52 TRM（100026_0104_00_en，11.6.2 / Table 11-8）规定 CSE 为 bit2，EDRCR[1:0] 为 RES0。CSE 清除 EDSCR 的 TXU、RXO、ERR，在 Debug 状态还清除 ITO；它不是清空 DTR 接收缓冲区的命令，不能据此保证消除 DTR_RX_FULL。

```
# Debug reset: MAINRESET.DBGRSTCON.DBGRST is bit 0.
catch {AHB_3 mww 0xC00008A0 0x1}

# Clear sticky debug errors: EDRCR.CSE is bit 2.
# core0: 0x80410090; core1: 0x80510090.
for {set _core 0} {$_core < $_cores} {incr _core} {
    set _dbgbase [lindex $DBGBASE $_core]
    catch {APB_1 mww [expr {$_dbgbase + 0x090}] 0x4}
}
```

上面仅摘录两个不同用途的写操作，不能替代完整 cfg。当前脚本仍以固定延时等待，没有显式轮询 EDPRSR.SR/HALTED 完成状态；catch 允许流程继续，但不证明写入成功，也不消除 Server 已输出的错误。应用复位和调试复位的影响应按芯片定义判断，不能由 DEBUGCORE=1/2 推导为单核隔离复位。

本地离线 Tcl 检查覆盖 DEBUGCORE=1/2/3、首次与再次 chipreset、掩码 examine、CSE 写值和 target 恢复。2026-09-23 使用修正后脚本完成单核/多核实板回归，见第12.8节；未做 CSE 新旧值的硬件 A/B 试验，不能仅据功能通过证明复位过程中每次写入都被确认。

实现依据：M：`.vscode/openocd.cfg`、`.vscode/THA6206/tha6206.svd`；Arm Cortex-R52 TRM 100026_0104_00_en，11.6.2；Arm CSAL 寄存器定义。

12.7 0.9.0：本地芯片目录与统一核心选择

用户芯片目录位于 `%LOCALAPPDATA%/debugtui/profiles/devices.toml`。npm 安装时初始化；独立 EXE 首次启动也会初始化。已有文件不覆盖，客户新增的芯片在升级或卸载后保留。`debugtui --init-profiles` 可补建或检查，`DEBUGTUI_CONFIG_DIR` 可指定配置根目录。默认声明 THA6104 的 core0、THA6206 的 core0/1、THA6412 的 core0/1/2/3 与 STM32F429 的 core0；目录是能力声明，不是硬件自动识别。

Setup 在 Tools / profile 后提供 Chip 与 Debug cores。Chip 按 Enter 打开目录，选择芯片后用 Space 或鼠标勾选核心；A 全选、N 清空、Enter 应用，至少保留一个。Ctrl+S 保存项目，Start 才按新选择启动会话。Chip 页面 N / Add chip 可新增名称、核心编号和 backend，例如 s32k144、0、generic，Ctrl+S / Save 追加目录。重复名称或无效编号拒绝保存，Esc 取消。核心编号支持 0–31，必须唯一。未选芯片时 Debug cores 灰显；L / Legacy 保留旧项目语义。

```
# debug.toml
version = 3
[tools]
profile = ".vscode/debug-env-chip.toml"
[debug]
chip = "tha6206"
cores = [0, 1]
```

工具环境片段（另一个文件）：

```
# debug-env.toml
backend = "tha6"
[core_targets."0"]
endpoint = "127.0.0.1:3333"
ready = "Listening on port 3333 for gdb connections"
[core_targets."1"]
endpoint = "127.0.0.1:3334"
ready = "Listening on port 3334 for gdb connections"
```

backend 仅选择匹配的工具环境；增加芯片目录条目不会安装 S32K 驱动或烧录算法。单一后端使用环境根字段，多后端可用 `[backends.tha6.gdb]`、`[backends.stm32f4.service]` 等分组覆盖公共工具配置。generic 可复用未声明 backend 的传统环境。缺少匹配后端、选了不支持的核心、端口缺失或重复均在启动前报告。

所选每核建立独立 GDB，共享一项服务。core1 单核仍命名 core.1 并连接其配置端口。项目 `[[cores]]` 可作为逐核模板，仅实例化勾选核心，保留初始化和启动顺序；不勾选的核不会创建 GDB 会话。Watch 和断点写入 `core_preferences.<chip>."core.N"`，切换芯片或子集时保留其他配置。共享 Reset 所属核心未选中时停用该组命令，不改派到其他核。Scope All 的软件组运行、暂停与现有逐核单步机制保持不变，不承诺硬件同步锁步。

发行包 `profiles/tha6-project.toml.example` 与 `profiles/tha6-environment.toml.example` 是一组完整 THA MCAL 示例，分别复制到 M 的 `debug-chip.toml` 与 `.vscode/debug-env-chip.toml`。用

`debugtui --project debug-chip.toml --setup` 打开。`${chip}`、`${chip_upper}` 模板切换程序、SVD、Build/Download 参数；`${core_mask}` 为所选核心位掩码，`${available_core_mask}` 为全部支持核心位掩码，`${unselected_core_ids}` 为未选编号列表。替换仅限启动相关字段，不处理 Watch、断点表达式或源码映射。`${profile_dir}` 保持原工具目录语义。

该示例为 MCAL 启动屏障 examine 全部支持核，但只创建所选核的 GDB。包含 core0 时先连接其他核，再通过 core0 做一次 chipreset 并恢复未选辅助核；仅 core1 等不含 core0 的组合为附加模式，要求 core0 已初始化。芯片切换不会自动编译或烧录，必须使用所选芯片配套 ELF/HEX/SVD。Bao 应保留自身启动策略，不能直接使用 MCAL 的入口和复位钩子。

此版本验证包含目录持久化、TUI 添加/勾选、真实原生 GDB 的 10 种单核/多核组合；原生 GDB 测试不是所有芯片的实板验证。THA6104/6412、STM32 和 S32K 的实际连接能力仍取决于工具适配和对应硬件验证。

实现依据：`src/devices.rs`、`src/launch/devices.rs`、`profiles/`、`scripts/test-devices.ps1`、`scripts/test-devices-gdb.cjs`。

四类文件的职责与最小迁移步骤

文件	保存内容、作用阶段与边界
用户 devices.toml	型号、支持的物理 core ID、backend 名称。启动前供 Setup 列表和合法性检查使用，不探测芯片，不启动 OpenOCD。目录本身的 version=1 与项目 version=3 无关。
项目 debug.toml	本次所选 chip/cores、ELF、源码、任务、逐核启动模板及偏好。用于复现工程调试方案，不把环境展开值写回共享 profile。
工具 debug-env.toml	GDB/Server 路径、公共动作、backend 分组、core_targets 端口/就绪信息。backend 是配置匹配键，不是动态插件、OpenOCD 驱动名或芯片自动识别结果。
板级 openocd.cfg	创建实际 target、AP、CTI、GDB 端口及复位操作。工具环境通过参数传 CHIPNAME/DEBUGCORE 等变量；变量含义由该脚本决定，不由 TUI 通用化。

默认目录条目的形式如下；S32K 条目是用户可追加的例子，并非随包默认值：

```
# User profiles/devices.toml, NOT project debug.toml
version = 1
[devices.tha6206]
cores = [0, 1]
backend = "tha6"
[devices.stm32f429]
cores = [0]
backend = "stm32f4"
# Example user addition; matching tools must be supplied separately.
[devices.s32k144]
cores = [0]
backend = "generic"
```

目录名称区分实际键名；新增名称按不区分大小写拒绝重复。名称与 backend 允许 1–64 个字母、数字、点、下划线或连字符。TUI 当前提供新增和选择，编辑/删除既有条目需直接维护目录文件。追加过程持有跨进程锁，读入最新内容后用同目录临时文件发布；锁等待约 2 秒，失败给出路径，不盲目覆盖。已有目录不会因升级自动合入新默认型号，缺少的型号可在 TUI 追加。

1. 旧工程可保持 Legacy；无需为了升级 EXE 强制改写全部工程。启用 Chip 时，Setup 将项目迁移为 version=3，保留已有配置字段；名称匹配的核模板及顶层继承仍提供初始 Watch/断点，运行期保存再写入芯片偏好。不自动重命名旧核心，也不替换用户的工具路径和启动脚本。
2. 确认目录的 backend 与环境匹配。例如 Bao 旧环境缺少根字段 `backend="tha6"`，选择 tha6206 会在启动前报告 requires backend；补齐声明还须提供每核端口/动作，不能通过改成 generic 掩盖不匹配。
3. 统一 THA 示例按名称 `core.0` 至 `core.3` 选择模板；旧式自定义名称不会自动按字符串尾部推断物理核，应按示例整理。endpoint 优先取项目核模板，缺省再取环境 core_targets；仅单选 core0 时允许回退 target.endpoint。
4. 选择 Chip，再选 Debug cores，核对展开后的 ELF/SVD/命令。Save config 保存；Start 按这些设置启动。切换芯片不自动编译下载，也不证明板卡与选择相符。

STM32F429 可继续使用原单核工程；如希望也从 Chip 统一选择，保留其原 program/tasks/profile，仅增加：

```
# Project: merge into existing tables.
version = 3
[debug]
chip = "stm32f429"
cores = [0]
# Its debug-env.toml root includes backend = "stm32f4".
# An existing [target].endpoint can serve single core0.
```

它只生成一个 core.0，不需要伪造第二个 target。S32K144 采用同样方式，但用户必须另备兼容的 GDB、Server target/Flash 配置；选择 generic 仅允许复用未声明 backend 的传统环境，不能赋予硬件支持。

同一个环境文件需要同时支持 THA 与 STM32 时，可将各自的工具表放入 `[backends.tha6.*]` 和 `[backends.stm32f4.*]`，公共表提供默认值。所选分组覆盖公共表，项目字段仍具有更高优先级。分组提供 core_targets 时整张表替换根映射，因此应列出该后端需要的全部端口。单环境只调 THA 系列时，根字段 backend=tha6 已足够；不要复制多套仅核心子集不同的工程文件。

从选择到会话的解析设计

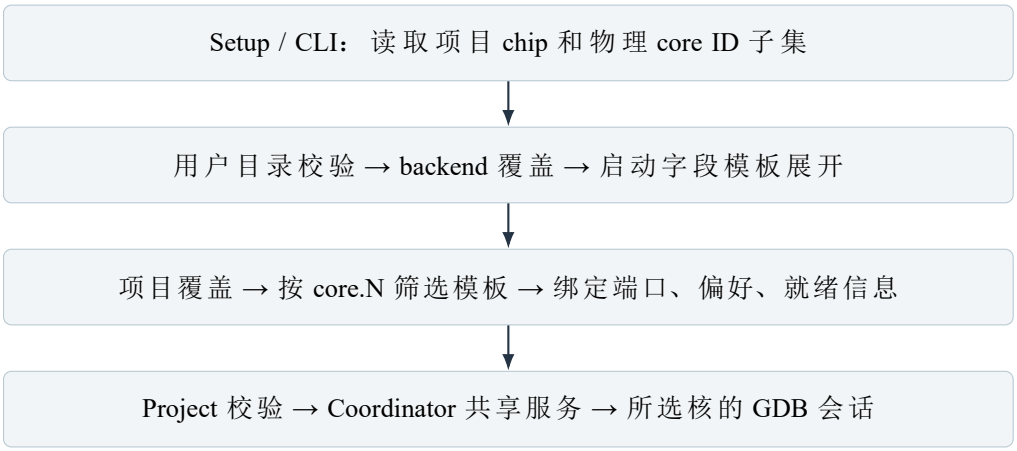


图 10: 启动前解析与运行期协调分离。目录中的 core ID 是物理编号。

所选核至少一个且不能重复，必须在型号声明中；远程端口不能重复。配置逐核 ready 时，每个所选核都须提供就绪标记。core1-only 在界面和偏好中仍称 core.1，但其工作区 index 可以是 0。内部索引用于当前会话路由，不能拿来生成板级 DEBUGCORE 掩码。

Watch/断点按 chip 加 core.N 存储；切换选择不丢失未选核定义。该隔离不应泛化为所有 UI 偏好：当前 formats/refresh 仍使用既有核心/对象键，应在换芯片时核对通道及寄存器读取策略。共享 Reset 的指定核未选中时只清除共享 restart 动作，不会自动给该核创建会话；各自的板级动作仍必须与目标启动条件匹配。

实现依据：`src/devices.rs`: resolve/Plan::apply；`src/config.rs`: from_document/save_preferences；`src/launch/devices.rs`；`profiles/devices.toml`。

12.8 0.8.6 单核与多核实板结果

2026-09-23 使用已安装的 v0.8.6、工程 GDB/OpenOCD 和实际 THA6206，完成 16 个功能组及其中 21 个退出/重连场景。先发现板上 Flash 与 ELF 不匹配，再通过 Load.bat 下载现有配套 HEX；下载后及测试结束时，向量表和代码段都与 ELF 匹配。没有重新编译当前应用或修改源码。

core0/core1 独立调试、双核 All/Core 范围、按序启动、断点联停、单核/多核断点转换、单步、Watch 树、运行中 Live Watch、内存与外设、Files/Symbols 跳转及 CI 源码映射均有通过记录。core1 独立调试期间 core0 计数器持续增长。硬件代码断点实测两核同时各插入 8 个，各自第 9 个被拒绝，清理后恢复正常；OpenOCD 报告每核另有 8 个数据观察点，但本轮未做数据观察点耗尽试验。

退出矩阵覆盖 core0/core1/multi、detach/resume/disconnect、暂停/运行，以及 remote 模式的 resume。对外部 OpenOCD 的附加会话退出不会关闭原服务。原工程 Watch/断点分别加载正确，构建/下载失败保持断开且可手动重连；成功构建生命周期使用 echo 任务，实际固件下载另做实测。

修正了两类外部工程工具配置：默认 Run 显式使用 chipreset、先放行 core1 再继续 core0，避免 GDB exec-run 走不支持的通用 reset；默认和外部 core1 环境统一自动读取 17 个通用寄存器，避免复位入口 VFP 未启用时的 DSCR.ERR。配置内有原因注释，这些工程 profile 不随 TUI 包分发。

残留限制：首次 dbgreset 仍可打印写内存失败，后续 examine 和执行恢复通过不代表该写入确认；GHS 深层栈存在 pc 0x80、未知帧及栈读取调试错误，不保证完整调用栈可靠。多核联停为软件协调，本轮 peer STOPPED 日志样本延迟为 124–266 ms，不是 CTI 同步停机。测试结束时两核暂停在复位入口，调试进程与端口清理；未做物理拔插、断电和多小时压力测试。

实现依据：`TESTING.md`；`artifacts/regression-0.8.6-20260923/report.json`、`manifest-after.json`、`log-observations.json` 及各功能组 result.json。artifacts 为本地证据目录，不进入源码或安装包。

12.9 双核会话组：启动顺序和控制语义

M 的 `debug-multi.toml` 已含两个 `[[cores]]`。以下节选保留启动/复位要点；其余 program、tools、tasks、GDB 配置沿用原文件，不要另建缺失工具路径的孤立片段。

```
[actions]
restart = []

[multicore]
scope = "all"
halt_peers = true
restart_core = "core.0"
restart = ["monitor chipreset"]

[[cores]]
name = "core.0"
endpoint = "127.0.0.1:3333"
startup_order = 1
after_connect = ["monitor chipreset"]
run = ["tbreak _main", "-exec-continue"]

[[cores]]
name = "core.1"
endpoint = "127.0.0.1:3334"
startup_order = 0
run = ["-exec-continue"]
```

`startup_order` 数值小者先连接、先 Run。本例 core1 先继续并等待，core0 随后运行初始化，符合参考固件的 `MULTICORE_SYNCVAR` 握手。core0 连接动作会复位整片，协调器在全部连接完成后同步各 GDB 的真实状态。默认选中 core0 不表示 core0 必须先启动。

操作	当前行为和验证方法
Run	当前界面的默认 Run 按组启动；JSONL 明确指定单核 scope 可只执行该核动作。Run 是否复位取决于动作，不能与 Continue 混用。
Continue / Pause	Scope=All 时对组内各核执行；Scope=Core 时针对选中核。用实际逐核状态确认，不能只看顶部一处 RUNNING。
Step / Next / Finish	单步选中核；All 范围下先暂停其他核，使非选中核不在后台继续。
断点联停	`halt_peers=true` 时，某核断点停止会触发同组其他核运行状态的协调停止。存在通信延迟。
Reset Chip	在 `restart_core` 执行一次共享复位，然后同步所有核；即使当前 Scope=Core，整片复位也不能当作单核复位。

单步跨越等待另一核应答的循环，可能因对端被暂停而无法前进。此时选择适当的 Continue 范围或临时关闭联停，不是反复单步等待协议自动完成。原始 `monitor resume` 改变的是 OpenOCD target，绕过会话组控制；日常组调试优先使用 TUI 控件。

12.10 断点资源与运行中 Watch

源码行号/断点栏点击新建断点仍只放到当前核；0.8.4 的代码正文点击用于选区。要两核都在同一代码位置停下，在 Breaks 选中持久代码/硬件断点，打开 Cores（快捷键 c），勾选另一核，Ctrl+Enter 应用。当前核须保留；临时断点和数据观察点不支持这种关联。两核的 GDB 断点编号可以不同，TUI 保存的是组关系。

历史 Server 日志分别报告 core0、core1 各有 8 个硬件断点、8 个硬件观察点。因此这里的容量是 **per-core**，不是全芯片共用 8 个。一个双核硬件断点通常占每核一个比较资源；Flash 上的普通断点也可能需要硬件资源，临时断点、条件实现和调试服务器保留用途会影响可用数量，不能把“8”当作始终还能添加的数量。

M 的旧式 `[live_watch]` 使用当前 MCAL ELF、TCL 6666、`bus_target="AHB_3"`，间隔 200 ms，可用于 `uart_cnt` 这类可解析的全局原始对象。看到 Console 的 `[live]` 增长后，还要检查 Watch 同名项的数值、LIVE 来源和时间，不能以 Console 代替面板验收。

新式逐项内存通道可用于 Watch、Memory 和外设可见项。`while_running=true` 只是允许在运行时请求；项目还要选择通道及非零刷新周期。`interval_ms=0` 表示手动刷新。局部变量、栈、寄存器或复杂表达式需要暂停后由 GDB 求值；总线读不能自动获得它们的当前上下文。

12.11 验收记录与故障定位

2026-09-20 历史实板证据（T 下）	可支持的结论
`artifacts/tha6206-multicore-fix-20260920/hardware-release-safe-regs/result.json`	passed=true；122 个请求、27 个检查；核入口/断点、All/Core 运行控制、单步对端 PC 不变、复位后双核 PC、重连和 Live Watch。
`artifacts/tha6206-live-watch-20260920/hardware-final/result.json`	passed=true；17 个请求，覆盖运行中的双核 Watch 采样变化及停止代次保持。
`artifacts/multicore-breakpoints-20260920/hardware-release/result.json`	passed=true；61 个请求；逐核实际命中、关联断点联停/删除及恢复运行。

配套 events/trace 与 M 下的报告保留上下文。复现时核对选中核、endpoint、PC/停止原因、对端状态和数据来源；结果仅适用于记录中的探针、芯片及固件。

2026-09-22 的 `artifacts/source-selection-20260922/result.json` 另记录了 core0、core1、多核三种入口的源码选区、复制动作路由、真实 GDB 加入 Watch 及逐核 Watch 隔离检查通过。测试使用真实源码和 GDB，会话绘制由 Ratatui TestBackend 驱动；不是人工在终端逐项点击的记录。原生 Windows 剪贴板读回另外验证，测试后恢复原剪贴板。

同组附加 Reset/Run 检查未在 8 秒内到达 `_main`，出现等待组停止超时和 DSCR.ERR；另行 compare-sections EX_CODE 返回 target memory fault。因此只能确认上述源码操作用例通过，不能声称当前固件完整启动链路通过。此次没有下载固件，也未以这些记录完成新 CSE 写值的实板验收。

现象	优先检查
core1 连不上/一直等待	DEBUGCORE=2/3、3334、主核初始化与从核释放；不要改连 APB/AHB target 的端口。
复位后 PC/寄存器像旧值	芯片复位是否成功、其他 GDB 是否重新同步、寄存器缓存是否刷新。
Flash 下载失败	是否误用 load；实际执行的 HEX、下载器、工作目录与退出码；是否仍有进程占探针。
Console 有变化而 Watch 没变	是旧 Live Watch 还是逐项通道；符号是否同一对象，通道是否 ahb，间隔是否为零，采样错误是否被忽略。
C 行号错位或打不开	HEX/ELF/源码提交是否匹配；Source root 与 source_map 是否正确，参见附录D。

实现依据：M：debug-core0.toml、debug-core1.toml、debug-multi.toml、.vscode/debug-env.toml、.vscode/Load.bat；T：src/coordinator.rs、src/coordinator/control.rs、src/coordinator/breakpoints.rs。

13 THA6206 上的 Bao Hypervisor 调试

13.1 先区分物理核、VM 与 GDB 会话

Bao 为静态划分资源的 Hypervisor；本地 THA6xxx 移植选择 armv8-r / aarch32 / cortex_r52、GICv3 和 MPU 路径。CPU0 为启动主核，平台描述包含两个物理核。本节针对 B 的本地移植，不代表上游 Bao 已发布相同平台支持。

入口	固件组织	当前 DebugTUI 连接
debug.toml	tha6206-smoke；一个单 vCPU Guest，affinity=1，位于 CPU0	格式 3；当前本地选择 [0,1]，Setup 可改为 [0]、[1] 或 [0,1]。
debug-dual.toml	tha6206-dual；两个 VM，每个一 vCPU，分别 affinity=1/2	格式 3；当前选择 [0]，已有两核模板，可在 Setup 选择双核。
debug-multi-images.toml	tha6206-dual；Bao + VM0/VM1 诊断构建	1.3.0 推荐入口：core.0 常驻 Bao+VM0，core.1 常驻 Bao+VM1；配置及实板验证见第13.5节。

双 VM 不是一个具有两个 vCPU 的 SMP Guest；两个物理核在运行，也不自动意味着出现两个 TUI 核标签。只观察 VM0 时，单 GDB 完全可以启动两核固件；要独立查看 VM1 的 PC、栈、C 断点，则需要 core1 的调试连接。

前两行的核心选择是 2026-10-02 文件快照；多 ELF 行按 2026-10-28 的参考工程核对。用户后续保存会改变所选核心；固件类型由 ELF 和 tasks 的 Config 决定，不能由核心标签数推断。参考项目属于外部 Bao 工作树，安装 DebugTUI 不会自动在任意目录生成这份项目。

当前启动流程依赖平台的双核同步。不要为了“只调一个 Guest”直接把 platform.cpu_num 改为 1：现有初始化调用关系中这可能绕开必需的 SoC 初始化。单 vCPU 配置与减少物理 CPU 数不是同一个改动。

实现依据：B：src/platform/tha6xxx/platform.mk、src/platform/tha6xxx/platform.c、src/platform/tha6xxx/tha6xxx_desc.c、configs/tha6206-smoke/config.c、configs/tha6206-dual/config.c；Bao 上游项目。

13.2 EL2、XIP 与 Guest RAM 布局

Bao 代码在 PFlash0 的 0x08000000 XIP 执行；已初始化数据的加载镜像在 Flash，运行地址在 GMU，BSS 和运行状态也在 GMU。Guest 二进制通过 VM_IMAGE 嵌入 Bao 镜像，由安装流程复制到配置指定的 RAM 执行地址。.vm_images 的 Flash 地址不是 Guest C 代码的运行地址。

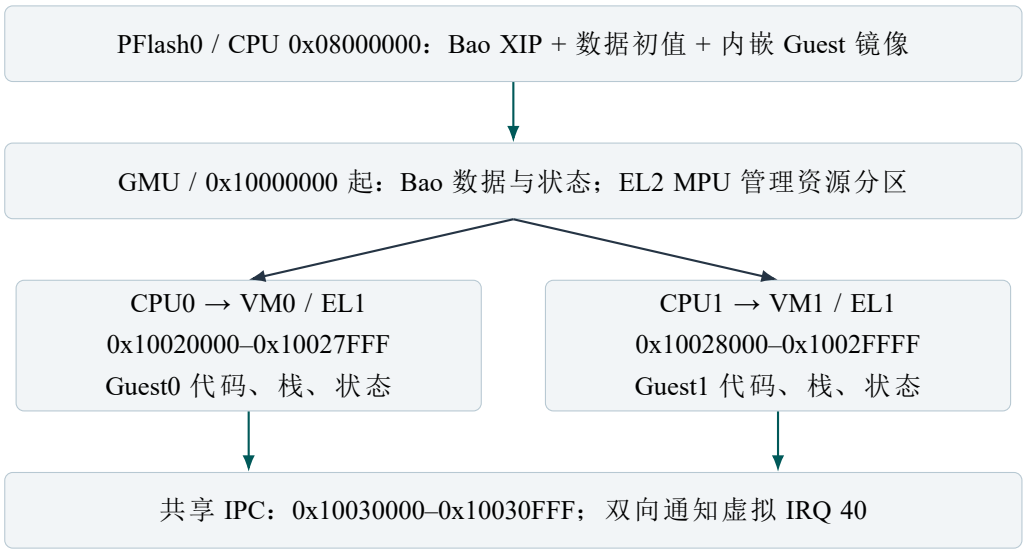


图 11: dual 配置的运行地址。smoke 使用另一套布局，不能与本图叠加。

配置/资源	地址/大小	用途
smoke Guest	0x10030000 / 64 KiB	单 vCPU；状态块 0x1003F000，SVC 栈顶 0x1003E000。
dual VM0 / VM1	0x10020000 / 0x10028000，各 32 KiB	状态块分别 0x10027000 / 0x1002F000；同一份 C 源码按 VM_ID 构建两份 ELF。
dual 共享 RAM	0x10030000 / 4 KiB	双向序号、payload/checksum、ack/epoch 和通知。
ASI0 UART	0xD0080000 / 4 KiB	dual 中映射给 VM0；VM1 不映射。Bao 自身也使用诊断串口。
GIC / Timer	GICD 0xF0000000；GICR 0xF0100000	平台描述的计数器频率 25 MHz；dual 验证使用物理定时器 PPI 30。

这里采用 MPU 隔离与固定物理地址布局，不应照搬 MMU Guest 的虚拟地址重定位步骤。CPU 处于 Bao HYP 时 CPSR mode 低五位为 `0x1A`，进入 Guest SVC 时为 `0x13`；其他 CPSR 位随中断/条件标志改变。只比较完整 CPSR 是否恒定并不能判断切换是否正确。

已有 MCAL 裸机固件可能假定从 EL2 启动并直接初始化全局时钟、RAM、其他核。将其用作 Guest 需要 EL1 启动代码、banked stacks、向量表、数据/BSS 初始化、中断/定时器适配和设备授权；不能只修改入口地址或把裸机 HEX 嵌进 Bao 就当作完成移植。

13.3 构建、烧录与源码映射

以下命令在 B 根目录执行。PowerShell 包装脚本默认使用 `Ubuntu-24.04D`，支持 -WslDistribution 选择发行版，-Jobs 默认 2、范围 1–32；迁移前核对发行版名称和 GNU 工具链。构建主入口是 B 的 Makefile/src/configs，而不是旁边的 bao-demos 工作树。

```
# Smoke image, then the dual-VM image (choose one pair).
powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\tha6206-build.ps1
powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\tha6206-download.ps1

powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\tha6206-build.ps1 -Config tha6206
    -dual
powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\tha6206-download.ps1 -Config
    tha6206-dual

debugtui --project .\debug.toml
# Or use the dual firmware project and select Debug cores in Setup:
debugtui --project .\debug-dual.toml
```

脚本先构建 Guest ELF/BIN，再调用 make，参数为 `PLATFORM=tha6xxx`、所选 CONFIG、`CROSS_COMPILE=arm-none-eabi-` 与 `DEBUG=y`。输出为 `bin/tha6xxx/<config>/bao.elf`、`bao.bin`、`bao.hex`；Guest ELF 在 `build/tha6xxx/<config>/guest/`。`VM_IMAGE` 的 incbin 依赖不一定被构建系统自动追踪，因此包装脚本在 Guest 重建后更新配置源文件时间，使内嵌镜像重新打包。

Build 在开始时删除旧 `bao.sha256`，仅当 Guest、Bao ELF/BIN/HEX 全部成功后生成三份产物的 SHA-256 清单。失败/取消后旧 ELF 或 HEX 可能仍在磁盘，但因没有成功清单而不能通过本地下载入口。Download 校验同一 Config 的 ELF、BIN、HEX 和清单，任一缺失或哈希不符即报错；校验通过才调用 S 的厂商下载器。烧录器返回 0 后才系统复位，烧录失败不会继续复位。不要混用 smoke 的 ELF 与 dual 的 HEX，也不要拿 Bao 的 BIN 直接按 Guest 的 RAM 地址整体烧写。TUI 的任务 timeout 为 300000 ms；Setup 中更换主 ELF 时也必须检查 build/download 参数是否同步。

无需占用探针的产物检查可单独执行：

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\tha6206-download.ps1 -Config
    tha6206-smoke -VerifyOnly
# Optional build parameters (use your installed distribution):
powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\tha6206-build.ps1 -
    WslDistribution Ubuntu-24.04D -Jobs 2
```

VerifyOnly 只比对磁盘文件，不读回 Flash，也不验证源码提交。流水线交付应提供同次构建 ELF/BIN/HEX 及匹配清单；没有可信配套产物时先解决产物问题，不用删掉校验来绕过。该校验是 Bao 工程脚本的约定，DebugTUI 通用 Download 不自动为其他工程生成清单。

构建使用 `wsl --cd` 直接传入工程目录，WSL 和 PowerShell 输出均设为 UTF-8，保留原始错误。下载器 stdout/stderr 同时写入 `debug_log/flash-<timestamp>.log`；Windows PowerShell 的原生 stderr 可能表现为 ErrorRecord，故脚本以实际原生退出码判断成功，明确读取全局 LASTEXITCODE，避免函数局部同名变量遮蔽。日志会记录镜像、哈希、命令、烧录和复位各阶段退出码。

现有 profile 用 `set substitute-path /mnt/d/ D:/` 解决本机 WSL 路径。此手写 GDB init 映射不受 Setup 的 Source remap 开关管理。迁移工程或使用流水线产物时，建议通过 Setup 选择旧目录，并清理冲突的手写 substitute-path；也可在项目 TOML 顶层增加明确前缀映射，供 GDB 与 TUI 一起使用；Source root 只声明本地源码根，不能替代绝对前缀映射。

```
# Merge into existing tables; do not duplicate [program].
[program]
source_root = "."
# Keep elf and svd from the selected Bao project.

[[source_map]]
from = "/mnt/d/LiBo/Files/DATA/FreeRTOS/BAOHyp/bao-hypervisor"
to = "."
```

`from` 必须以该份 ELF 的 DWARF 为准。流水线另有源码根时，用真实构建前缀替换；使用同次构建的 Bao ELF、Guest ELF、HEX 和对应源码提交。完整流程见附录D。

实现依据：B：`scripts/tha6206-build.ps1`、`scripts/tha6206-build.sh`、`scripts/tha6206-download.ps1`、`scripts/tha6206/debug-env.toml`。

13.4 连接与热复位：先处理启动屏障

当前 Bao profile 声明 backend=tha6、core0/1 的 3333/3334 端口，Server 仍以 DEBUGCORE=3 建立两个 target。项目的 [debug].cores 决定创建哪些 GDB；没有选中 GDB 不等于物理核不参与 Bao 初始化。

1. 包含 core0 时，先连接已选的 core1（startup_order=0），core0 最后连接（=1）。core0 的 after_connect 执行一次 chipreset，选回 core0，写 `set {unsigned int}&_barrier = 0`。
2. 清屏障后，只对未选中的辅助核执行 Server resume。core0-only 会释放 core1，双 GDB 时由协调器控制已选 core1，避免偷偷恢复另一 GDB 管理的核。最后选回 core0 并刷新寄存器缓存。
3. Run 的逐核动作只负责进入执行：core0 建 init 临时硬件断点后 Continue，core1 刷新缓存后 Continue；**Run 不再隐含整片复位**。重做启动实验用 Reset Chip，再 Run All；Continue All 从当前状态继续。
4. 共享 Reset 仅通过指定的 core.0 执行一次 chipreset、清屏障及恢复未选辅助核，之后协调器同步已选核。core1-only 没有复位拥有者，禁用这组 Reset 动作，是附加到已初始化平台的模式。

本地 chipreset 保留 GMU，前一次的 `_barrier=2` 可能让 core1 过早越过屏障；必须先复位、清屏障，再释放辅助核。地址通过本次 Bao ELF 中的 `&_barrier` 解析，不抄录固定数值。只发 monitor chipreset 并不能代替上述完整流程。

core1-only 在 Setup 直接选 [1]，无需再复制配置并手改端口。前提是 core0 已完成平台初始化；新启动的 Server 也可能影响现有运行状态，须检查该板级脚本。暂停附加核后再添加 Guest1 符号。该模式的配置可加载和选择测试不等于其完整 Build/Download 冷启动已验收。退出后的核状态仍取决于 `on_exit`、Server 与板级行为。

实现依据：B：`debug.toml`、`debug-dual.toml`、`scripts/tha6206/debug-env.toml`，2026-10-02 文件快照。

13.5 同时保留 Bao 与 Guest 的 C 符号

1.3.0 已提供常驻多 ELF 调试。主 `program.elf` 保留 `bao.elf`，另用 `[[images]]` 声明 Bao、VM0 和 VM1。每个物理核的 GDB 在连接目标前装入主 ELF 与该核绑定的额外符号；底层使用 GDB 的

`add-symbol-file` 能力。Guest 的 incbin 只嵌入程序字节，不会把其 DWARF 自动并入 Bao，所以仍须提供各 Guest ELF。

真实停止后，DebugTUI 用 PC、ELF 可执行 section 和停止前程序 CPSR 识别执行镜像；选中调用者帧时另识别帧的归属。Bao 与 VM 的符号持续保留，跨 HVC/虚拟中断时无需先移除一个 ELF 再加载另一个。**浏览另一个镜像只改变显示，不使 CPU 跳转，也不隐式暂停、复位、下载或重连。**运行中显示最后已知身份，不提供逐指令 Trace。

以下为本地已安装版本的日常入口，在 B 根目录执行：

```
debugtui --version
# debugtui 1.3.1
debugtui --project .\debug-multi-images.toml
```

直接运行 `debugtui` 仍读取当前目录默认项目 `debug.toml` 并进入 Setup；默认文件若是旧单 ELF 配置，不会自动变成多 ELF 工程。可在 Setup 的 Project 行选择 `debug-multi-images.toml`，也可使用上面的显式命令。PowerShell 独立终端即可使用，无需 VS Code、外部 Python 服务或另外安装辅助 GDB。

13.6 Symbol images 与每核镜像配置

在 Setup 选 **Symbol images**，Enter 打开镜像编辑器。先选 Image，维护 ID、ELF file、Source root、Cores、Program domain、Section offset、Source maps 和 Global memory channel；Add/Remove 管理条目，Apply to draft 更新工程草稿，Save config 保存，Start 应用到新的符号会话。文本字段 Enter 后进入编辑，1.3.1 显示 Editing 标题和插入光标，长路径窗口跟随光标；Enter 确认，Esc 撤销，Ctrl+U 清空。编辑本身不连接目标、不下载固件；高级 section 地址与 saved_context 诊断声明在 TOML 中维护，编辑器会保留它们。

以下片段应**合并到已有、可正常连接的 Bao 工程**，不是完整板级 profile。本地 `debug-multi-images.toml` 已包含端口、启动动作、AHB 通道、Build/Download 和下文的 Guest 诊断配置。已有同名 TOML 表不可重复追加。

```
version = 3

[tools]
profile = "builtin:arm-openocd"
probe = "cmsis-dap"

[debug]
chip = "tha6206"
cores = [0, 1]

[program]
elf = "bin/tha6xxx/tha6206-dual/bao.elf"
source_root = "."
```

```
[[images]]
id = "bao"
elf = "bin/tha6xxx/tha6206-dual/bao.elf"
source_root = "."
domain = "hyp"
memory_channel = "ahb"

[[images]]
id = "vm0"
elf = "build/tha6xxx/tha6206-dual/guest/guest0.elf"
source_root = "."
cores = ["core.0"]
domain = "guest"
memory_channel = "ahb"

[[images]]
id = "vm1"
elf = "build/tha6xxx/tha6206-dual/guest/guest1.elf"
source_root = "."
cores = ["core.1"]
domain = "guest"
memory_channel = "ahb"

[[source_map]]
from = "/mnt/d/"
to = "D:/"

[source_remap]
enabled = true
```

Bao 未限制 cores，适用于本项目全部核；VM0/VM1 分别只属于 core.0/core.1。镜像 ID 必须唯一，`program.elf` 与主镜像 Bao 对应；旧单 ELF 工程可不声明 images。这里的核心名是会话名，与 `[debug].cores` 的物理索引分开。双 VM 不是 SMP Guest，也没有因此支持迁移/复用 vCPU 的自动调度发现。

ELF、Source root 和映射目标的相对路径以工程 TOML 为基准。参考 Guest 按 RAM 运行地址链接，`offset` 默认 0，不填 Flash 的嵌入位置；其他布局可显式配置 offset/sections，主 ELF 保持其链接地址。先核对同次构建的 Bao ELF、Guest ELF、HEX、源码提交及 SHA 清单，源码映射不能修正错误的固件或重定位。

镜像还可独立声明 `[[images.source_map]]`，位置放在所属 `[[images]]` 后、下一个镜像前。相同远端前缀可分别映射至不同本地目录；同前缀优先使用镜像规则，再使用公共规则，按最长前缀匹配。Source remap 为 No 时两类规则均停用。正斜杠、反斜杠和混合路径均可匹配，包括带盘符的 Windows 路径。命中映射但本地文件缺失时明确报错，不借用别的镜像同名文件；相对 DWARF 文件名用本 ELF 编译目录定位，不能唯一定位时不猜测。

13.7 源码、栈、Watch 与断点的镜像归属

入口	操作与含义
`:images`	查看当前核的常驻镜像、内容版本及识别状态。`:core core.0` / `:core core.1` 或 Ctrl+T 选择物理核；Scope 是运行控制范围。
Files / Symbols	各 ELF 独立查询，结果带 `@bao:` / `@vm0:` 归属；Symbols 可输入 `@vm0:guest_main` 限定镜像。
Source / `:open`	`:open @vm0:configs/tha6206-dual/guest.c` 显式浏览 VM0 源码，标签显示镜像并独立记住位置；不改变物理选中帧。标题区分 Exec、Browse 和 Frame。
Asm / `:disasm`	`:disasm @vm0:guest_main` 浏览该 ELF 的指令并标明来源；运行中默认 PC 是 last-known，ELF 指令不证明板上字节一致。
Watch / Locals	普通名称默认属于物理选中帧镜像；`@bao:debug_domain_id`、`@vm0:debug_domain_id` 显式指定全局变量。成员和数组保留相同归属。
Breaks / `:break`	`:break @vm0:guest_main` 从 VM0 ELF 解析精确地址，在当前物理核建立硬件代码断点；断点编辑器也有 Image 项。持久定义保存 ID 与原名称，重连后重解析。

Files、Symbols 与 ELF 反汇编在运行中也可使用，不访问物理调试端口。停止时 Asm 默认使用当前镜像的物理 GDB；显式其他镜像使用离线 ELF。辅助 GDB 只同步 gdb.init 中独立的架构声明，不执行其中的复位、下载或连接动作。参考 Bao 的 `set architecture armv8-r` 用于正确解码 HVC、ERET 等指令。

多个 objfile 的同名全局可能被 GDB 绑定到主 ELF，MPU 也可能阻止当前 CPU 读取另一域。1.3.0 用单 ELF 的辅助 GDB 提供类型，再经镜像 `memory_channel` 读取实板内存；为空时可沿用 `saved_context.channel`，缺少可用通道时报错。THA6206 参考值为工程/芯片 Memory channels 中的 `ahb`。当前隔离求值限 ARM32 小端 DWARF 的只读 C/C++ 子集。

当前执行镜像的表达式可组合局部值、整数寄存器和全局值，例如 `input + debug_domain_id`。寄存器来自真实物理顶层，辅助展开到调用者时核对帧地址，不修改物理 GDB 选中帧。显式其他镜像只读其全局作用域，不能借用执行镜像的局部变量或寄存器；挂起 Guest 的局部值须进入下节的 Guest 视图。浮点寄存器、GDB 便捷变量及隔离表达式赋值尚不支持，不可恢复的值显示 unavailable 或 `<optimized out>`。

多镜像代码断点默认用硬件，避免 Bao 启动复制 Guest 时覆盖软件断点。资源属于实际核心/后端，由后端报告耗尽；多 ELF 不会增加硬件槽位。地址重叠时无法把单一硬件地址比较器冒充镜像专属断点。命名数据断点限当前物理帧镜像；条件可用局部参数或明确类型地址，命名全局条件因归属不可靠而拒绝。多核关联仍需显式设置 Cores，并核对每核的符号地址。

启用已有 `[live_watch]` 后，8/16/32 位全局标量可通过声明 `while_running=true` 的通道轮询。显式 `@vm0:` 变量绑定 VM0；普通名称绑定最后一次停止镜像，显示 last-known，数值仍从内存采样。切核、停止代次或 ELF 内容改变会丢弃旧响应；这不是连续执行轨迹，也不是跨核原子快照。

13.8 在 Bao 中查看挂起 Guest 的保存上下文

物理 CPU 停在 Bao 异常处理时，当前寄存器和栈属于 EL2，GDB 同时有 Guest 符号也不会自动恢复 Guest SP/LR 和调用栈。需要 Bao 提供真实、有效的保存上下文。本地参考工程使用诊断构建：

```
pwsh -NoProfile -File scripts/tha6206-build.ps1 `
  -Config tha6206-dual -DebugGuestContext
pwsh -NoProfile -File scripts/tha6206-download.ps1 `
  -Config tha6206-dual
debugtui --project .\debug-multi-images.toml
```

`-DebugGuestContext` 在 VM_EXIT 保存完成后捕获真实 Guest r0–r12、banked SP/LR、返回 PC、保存 CPSR 和 CPU/VM/vCPU 身份，VM_ENTRY 前使快照失效。`bao_v1` ABI 为 112 字节；不是从 Bao 当前栈猜测 Guest 寄存器，也不补造原先未保存的 x[13]。该选项增加诊断 RAM 和异常路径指令，普通发布固件可不启用。参考汇编另提供只在调试节的 CFI，物理栈在不可恢复的异常边界结束，该 CFI 本身不改变装载程序字节。

以下子表放在 `id="vm0"` 的镜像条目中、下一个 `[[images]]` 之前；本地参考项目已配置。memory 是允许读取的 Guest 私有及共享区，不能为绕过校验任意放宽。

```
[images.saved_context]
provider = "bao_v1"
symbol = "bao_debug_guest_contexts"
index = 0
cpu = 0
vm = 0
vcpu = 0
channel = "ahb"
memory = [
  { start = 0x10020000, size = 0x8000 },
  { start = 0x10030000, size = 0x1000 },
]
```

VM1 的 index/cpu/vm 均为 1，**vcpu 仍为 0**；其私有 RAM 起点改为 `0x10028000`，大小仍为 `0x8000`，共享区相同。这是当前 dual 固件布局，其他固件须按自己的 ABI、VM 配置和 ELF 核对。

停在对应核有效的 HVC/IRQ 保存窗口内，输入 `:guest vm0`；只有一个候选时可用 `:guest`。Guest 视图独立显示保存 PC/SP/CPSR、源码、调用栈及局部变量。上下箭头或鼠标选择 Guest 逻辑帧，PgUp/PgDn 浏览源码，Esc 返回物理 Bao 视图。物理 GDB 帧选择保持不变，切核、继续执行或停止代次变化后旧视图失效。

附带 GDB 没有 Python；DebugTUI 启动同工具链辅助 GDB，经本机只读 RSP 提供真实保存寄存器，经配置 AHB 通道读取实板内存。它不连接物理 GDB 端口，不接受继续、单步、写入和断点命令。缺少有效快照、保存尚未完成/已开始恢复、身份不符、SP 越界、ELF 改变时拒绝解析；优化消除的局部和未捕获的浮点上下文不补造。

13.9 从 HVC 返回 Guest 与运行中取消

在完整保存的 HVC32 边界输入 `:guest-return vm0`，或在 Guest 视图按 r。DebugTUI 为**当前物理核**建立自有临时硬件返回断点后继续；真实用户断点、异常或显式 Pause 优先，操作中断时保留真正停止原因，清理自己的临时资源，不自动越过其他停止。这不是所有 IRQ/异常边界通用的自动单步跨域。

`:guest-return-cancel` 可在运行中取消，不额外暂停或继续。GDB all-stop 运行时不能正常发送删除远端断点请求，DebugTUI 先核对当前 GDB 连接、OpenOCD 核端口和 TCL 路由，再禁用自有记录、删除唯一自有的物理断点并读回核对；若与用户断点共址，保留物理位置及用户条件。禁用记录在下一次真实停止时删除，结果中 `logical_cleanup_pending` 表示仍待这一步收尾。无法证明路由或所有权时明确报错，不隐式 halt。测试用 `-DebugReturnTests` 仅制造确定的等待窗口和 UDF 负例，**日常仅需 -DebugGuestContext**。

部分 GDB 在硬件断点耗尽时先回复 running 再报告插入失败，且没有 stopped 通知。1.3.0 对可信后端错误作只读线程状态核对，确认实际已停才恢复 STOPPED，并只清理本操作的临时资源。普通 Continue 也采用此恢复路径，不额外复位或重连。

13.10 从旧逐核符号配置迁移及能力边界

旧版可在各核 init 手工执行 `add-symbol-file`，这只能补充 GDB 符号，不提供可靠的镜像归属和挂起 Guest 视图。1.3.0 推荐改用 images，并从该项目的逐核 init 移除重复的 Guest add-symbol-file，保留原有板级 startup_order、after_connect 和 run。单 ELF 工程不必迁移，原有 Build/Download 成功后重连、重载符号的流程仍适用。

参考多 ELF 项目先连接 core1，再由 core0 统一复位并清 `_barrier`。组控制设置为 `scope="all"` 和 `halt_peers=false`。启动动作与跨域源码显示不同，只有明确的运行控制、Build/Download 或重连会执行相应动作。需要单核隔离验证时显式用 `:scope core`，不要用浏览镜像代替选择物理核心。

情况	1.3.0 行为/限制
运行中新增或替换 ELF	本版镜像在连接前常驻；未提供运行中热增/热换 objfile。修改声明后须明确建立新的符号会话，不宣称维持原运行状态完成热替换。
unknown / ambiguous	无法唯一确认镜像时不猜测，不展示旧局部值或假物理栈；仍可显式浏览 ELF 文件。
同路径 ELF 改变	标记 stale，停止使用旧地址/类型求值和写入预览；先确认板上固件匹配，再明确重启符号会话。
跨 HVC/虚拟 IRQ	正常停止位置属于 Bao/VM 时自动识别源码；挂起 Guest 的栈/局部只在有效快照窗口可恢复，原生异常栈不能保证无限展开。
多个域的变量写入	隔离表达式目前只读，不借错误 objfile 执行赋值；命名全局断点条件也不宣称隔离求值。
运行中观察	最后已知身份与内存采样，不是 Trace；AHB 与 CPU 缓存的一致性由平台决定，多个读取不是原子快照。

Headless 提供 `images`、`guest_context`、`return_guest` 和 `cancel_guest_return`。例如 guest_context 参数为 `{"image":"vm0","level":1, "expressions":["handled","tick"]}`；结果包括物理停止上下文、镜像版本、保存序列、只读来源和不可用值。方法的异步完成仍须遵守 JSONL 响应与目标状态规则。

13.11 1.3.0 安装版的 THA6206 实板验收

2026-10-28 使用 Release 构建并通过完整 npm 包全局安装的 1.3.0，在 THA6206 双核板执行下表回归。测试过程使用两条物理 GDB 和一个共享 OpenOCD；连续跨域流程核对物理进程不变。普通诊断固件用于主要功能，取消/异常控制负例使用独立测试固件，结束后已恢复普通 `-DebugGuestContext` 固件。本次编辑手册只引用已有原始证据，不重新烧录。

类别	结果	实际覆盖
资源、符号、边界	27/27	硬件断点耗尽恢复；同名函数/全局/类型；帧归属及边界。
双核跨域主流程	13/13	Bao、VM0、VM1，20 次 HVC、虚拟 IRQ、混合表达式与物理进程连续性。
导航与运行中 Watch	10/10	各镜像文件/符号/反汇编；标量采样的镜像、核心及版本绑定。
真实终端与全局入口	12/12	ConPTY 操作 9 项；直接调用已安装 debugtui 的版本、真实子进程与 VM 源码浏览 3 项。
两核控制负例	12/12	每核 6 项：运行中取消、用户共址条件断点、显式 Pause/重试、实际 UDF 异常优先。
实板/终端合计	74/74	0 失败、0 跳过；不把软件模拟项计入实板。
软件/分发/本机	10 / 14 / 21	分别为模拟多镜像 10 项、分发 14 项、本机 GDB 21 组；全部通过，独立于实板计数。
Rust Release	606 + 7	库测试通过 606 项、集成通过 7 项；原有 3 项忽略，不改记为通过。

原有忽略项涉及 Windows 剪贴板写入、缺失 STM32 UI replay 及需要显式环境的 THA6206 源码选区测试；新实板/TUI 回归按自己的报告验收，不用新结果覆盖历史未执行项。已安装 EXE 的 SHA-256 为 `e18469afc910c8ffdea4e40a5af615b936265d7588efce364721ab11bd98979a`。这些结果证明当前最小 dual Guest 参考场景，不等于通用 RTOS/SMP/迁移 vCPU 的验收。

实现依据：T：`artifacts/validation-1.3.0.json` 及其中列出的原始 report.json、固件摘要、终端/进程证据；`docs/bao-multi-elf.md`、`docs/bao-multi-elf-development.md`；B：`debug-multi-images.toml`、`scripts/tha6206-build.ps1`。实现：`src/images.rs`、`src/session/images.rs`、`src/session/image_values.rs`、`src/session/guest/`、`src/launch/images.rs`、`src/ui/guest/`。

13.12 运行时观察：状态块、Timer、IPC 与 UART

Bao profile 已配置 `[[memory_access]] id="ahb"`，TCL 6666，target=`AHB_3`，允许运行中读取。CPU 在启用 MPU 后可能无法直接读某些 RCC/PORT 地址，而 AP3 仍可观察总线侧数据；不要为让 GDB

读寄存器方便而随意扩大 Guest 的设备权限。

现有配置里的外设刷新项 `interval_ms=0` 均为手动刷新。要持续看状态，可在 Memory 显示对应状态块并选择 ahb、200 ms，或在 Watch 对可解析地址的标量选择相同通道。运行前先在暂停状态核对地址/宽度。Locals、调用栈和 CPU 寄存器仍按停止状态解释。

smoke 地址	内容	预期解释
0x1003F000	magic	0x42414F47，表示 Guest 已写状态。
0x1003F004	CPSR	历史 0x1D3：SVC，IRQ/FIQ 关闭。
0x1003F008	vMPIDR	历史 0xC0000000；不能据虚拟 MPIDR 判断物理核编号。
0x1003F00C	CNTFRQ	25000000。
0x1003F010 / 14	fault / heartbeat	fault=0，心跳在运行中递增。

smoke 是启动和 EL1 执行的最小探针，不能用于证明 IRQ、调度器或 FreeRTOS 移植已完成。

dual 状态块偏移	字段	读取含义（每项 32 位）
0x00 / 04 / 08	magic / vm_id / phase	magic=0x42414F32；vm_id=0/1；健康运行 phase=4。
0x18 / 1C / 20	heartbeat / timer / ipc	心跳、Timer IRQ、IPC IRQ 计数；运行中应随采样变化。
0x24 / 28 / 2C	sent / received / errors	IPC 往返及校验错误；健康测试 errors=0。
0x30 / 34	uart_chars / last_irq	VM0 可发 UART，VM1 不应借此拥有设备；last_irq 按实际事件变化。
0x38 / 3C / 40 / 44	fault / fault_pc / status / addr	Guest 自身异常记录；EL2 截获的非法访问未必进入 Guest handler。
0x48 / 4C / 50 / 54	command / probe_started / returned / observed	故障注入协议；写 command 会驱动 Guest 执行访问，不是普通观测。
0x58 / 5C / 60	period / hvc_ret / guard	VM0/1 周期 2500000/5000000；guard=0x600DCAFE+VM_ID。
0x70 / 74	unexpected_irq / notify_errors	健康功能测试应为 0。

VM0 的基址为 `0x10027000`，VM1 为 `0x1002F000`。例如 VM0 timer 位于 `0x1002701C`，VM1 timer 位于 `0x1002F01C`。25 MHz 下两 VM 定时器周期分别 100/200 ms；IPC 通知使用 HVC `0x86000001` 和虚拟 IRQ 40。应同时检查收发计数、校验/通知错误以及对端响应，不能只凭心跳判定 IPC 正常。

串口是独立观测通道。当前 ASI0 配置使用 200 MHz 时钟、115200 baud，历史分频寄存器为 IBRR=108、FBRR=8。COM3 是历史测试端口，换机器应枚举实际串口；TUI 的 GDB Console 本身不是自动接收 UART 的终端。

13.13 双 VM 实板功能与隔离测试证据

A 下 `tha6206-dual-board/verification.json` 记录 passed=true、66 项检查。配套包含功能采样、UART 原始数据、Flash 对照、逐条故障注入和构建产物哈希。当前 dual ELF/BIN/HEX 的 SHA-256 已与该目录的 `artifact-sha256.json` 对照一致；这限定了记录对应的产物，不自动证明整个当前工作树已经重新构建。

记录中的检查	可以据此说明什么
Flash 比对	`flash-verify.log` 经 AHB_3 在 0x90400000 对照 Bao BIN，记录 59064 字节 verified。这里是 PFlash0 的总线别名。
健康运行	一次采样 VM0/1 Timer=1381/690，IPC=1372/1372；errors、fault、unexpected_irq、notify_errors 均为 0。计数为历史快照，不能当作新测试必须相等的常数。
实际双核执行	两物理核 PC 分别位于 0x100204D8 / 0x1002849C，CPSR=0x60000153，落在两份 Guest 的运行区间。
UART	VM0 77 字节、VM1 0 字节，原始串口记录可复核；不等于 RX IRQ/DMA 或长期串口压力测试。
七项非法访问	VM0/1 互读写对方私有区四项，VM1 读 UART 一项，VM0/1 读 Bao RAM 两项；均检查 EL2 异常与对端活动。

隔离检查不是让调试器自己读受保护地址。测试通过 AHB 写目标 VM 状态块的 command，让 **Guest 的指令**发起越权访问，再检查 HSR/HDFAR、probe 是否返回、guard 是否被改及另一核的心跳/Timer。调试 AP 可读内存是调试通路能力，不能据此判定 Guest MPU 隔离失效。

上述非法访问的 HSR.EC 为 `0x24`，表示来自较低异常级的数据中止；写访问还检查 WnR。互访 guard 地址为 `0x1002F060` / `0x10027060`，VM1 非法 UART 地址为 `0xD0080010`，Bao RAM 探针为 `0x10000000`。HDFAR 应对应发起访问的地址。故障核停留在 EL2 错误循环；另一核仍可运行，但对故障 VM 的 IPC 请求不再得到正常回应。因此这是异常拦截及对端继续执行证据，不是自动恢复或服务可用性的证明。

每项负向测试从健康状态重启，避免前一项异常污染下一项。目录中保留的 `pre-reset-fix` 失败记录用于解释屏障修正，不计入最终七项通过结果。smoke 的另组记录在 A 的 `tha6206-board-debug/`：2026-09-21 的 TOML 编译/下载、55956 字节 Flash 对照、心跳变化和 67 字节 Bao UART banner，支持最小入口的构建下载链路。

实现依据：A：`tha6206-dual-board/verification.json`、`first-functional.json`、`final-functional.json`、`final-healthy-start.json`、`isolation-*.json`、`artifact-sha256.json`（后五项在同一 dual-board 目录）；`tha6206-board-debug/` 的 2026-09-21 记录。

13.14 9 月 29 日编译下载故障与历史证据

`session-20260929-134940-542.log` 的失败发生在 WSL 启动/路径转换阶段，编译器尚未运行，Download 也未执行。独立 `/bin/true` 同样返回 `Wsl/Service/E_UNEXPECTED`，Windows 同时段有虚拟内存不足记录。确认发行版停止后执行 wsl –shutdown，再启动后恢复；这是当次宿主故障的证据与

恢复步骤，不是所有编译错误的统一根因，也不能在有运行作业时盲目关闭全部 WSL。

另修复了包装脚本吞掉原始 WSL 错误、编码不一致和烧录器退出码的判断。诊断顺序是：先看 Console/会话任务输出定位宿主或编译器，再看 bao.sha256 的产物校验，最后看独立 flash 日志；不要把构建失败后留下的旧镜像继续作为新结果调试。

验证层与固件	已记录结果和范围
隔离脚本夹具	6 项通过：缺失清单、空格/中文路径、ELF 被修改、WSL 原始失败与旧清单失效、原生 stderr 成功、烧录失败不复位；无板卡访问。
Bao smoke / core0	已安装 0.9.0，5 项通过：连接、Build 后符号重载、Download 后重连、全量 Flash 比对、C 源码断点/单步及 Guest 心跳。
Bao smoke / core0+1	同版 0.9.0，5 项通过：双核连接、Build/Download 释放与重连、Flash 比对、Run/单步/心跳；不等于 dual Guest 符号验收。
Flash 回读	56,596 字节全部匹配本次 smoke 构建；SHA-256 见原始报告。该结果不适用于随后其他烧录或另一 Config。
Setup / 0.9.1	安装后的程序以 Bao 隔离配置验证无参数启动、三种核心选择和保存；没有连接或重新烧录板卡。

实现依据：T：artifacts/bao-tasks-1790663763716-5b7a1cdd/report.json；artifacts/bao-workflow-1790663775392-2c3ae114/report.json（core0）；artifacts/bao-workflow-1790663333421-4fde5892/report.json（双核）；artifacts/bao-setup-f2a2b64336c1474b9a201bbb59540787/report.json。

13.15 后续适配边界与故障定位

现象	检查顺序
复位后偶发卡住/串口异常	保留 GMU 的旧 `_barrier`、清零时机、两核启动次序，再查时钟/LLPP/MPU；不要只重连 GDB 掩盖启动条件。
能停 Bao，不能看 Guest C	Guest ELF 是否额外加载、对应 VM 是否正确、.text 是否用 RAM 运行地址、WSL/source_map 是否匹配。
dual 只有一个核标签	检查是否仍使用原始 `debug-dual.toml`；固件有两 VM 不会自动创建 cores 配置。
RCC/PORT 读失败或值不更新	CPU 当前 MPU 权限、ahb 通道、刷新间隔；不要把手动刷新 0 ms 当作实时轮询。
互访异常却 Guest fault=0	检查是否由 EL2 先截获；联合 HSR/HDFAR、EL2 PC 和未返回的 probe 判断。
对端不动	TUI 是否联停、是否正在单步等待 IPC、对端是否故障；区分调试暂停与固件死锁。

本地移植已能支持上述 smoke 与双 VM 功能记录，但仍有明确边界：平台时钟初始化的返回值未完整向上报告；部分等待循环缺少健壮的超时处理；GIC maintenance IRQ 25 在平台描述中仍带待确认

项。应继续验证冷启动/POR、异常恢复、全部内存权限、长时间压力与实际外设组合。

移植 FreeRTOS Guest 时还要验证 EL1 上下文切换、tick 中断、临界区/中断屏蔽、任务栈、设备 IRQ 路由和共享资源策略。当前两个最小 Guest 的 Timer/IPC 通过，不等于 FreeRTOS 调度器、SMP Guest、DMA 隔离或生产安全认证已完成。B 的 `docs/tha6206-bao-porting.tex` 可作底层移植细节参考，但其中较早的未验证项需要结合 2026-09-21 新记录判断，不能用旧快照否定后来的实板证据。

实现依据：B：`src/arch/armv8/cpu.c`、`src/arch/armv8/aarch32/boot.S`、`src/platform/tha6xxx/platform.c`、`src/platform/tha6xxx/tha6xxx_desc.c`、`configs/tha6206-dual/guest.c`；T：`src/config.rs`、`src/session.rs`、`src/coordinator.rs`。

14 项目与环境配置参考

14.1 覆盖、路径和保存规则

配置优先级为：默认值 → tools 公共环境 → 所选 backend 分组 → 项目字段 → CLI。表按字段合并，数组整体替换；项目中写空数组会清空相应环境数组。环境普通配置允许 `gdb`、`target`、`service`、`actions`、`session`、`sync`、`memory_access`、`multicore`，另接受元数据 `backend`、`backends`、`core_targets`，由芯片解析层消费；不接受 `program`、`cores`、`live_watch` 等工程表。大部分结构启用 deny_unknown_fields，字段拼错会报错。

含斜杠的 gdb.executable、service.command 和 cwd 相对定义它们的 TOML 文件；裸命令从 PATH 查找。`${profile_dir}` 只在环境文件展开。program、source_map.to、live_watch.elf、session.log_dir 等最终相对项目目录解析；不要认为环境文件中的所有路径字段都会自动相对 profile。CLI 路径相对启动时 cwd 处理。

0.8.4 的 Setup 将 Project 字段优先显示为相对启动目录的路径，但内部仍解析为确定的项目位置。资源路径遵守上述原规则；尤其 `gdb.args` 中 `-x` 后的相对文件名由 GDB 工作目录解释，未设置 `gdb.cwd` 时继承 DebugTUI 启动目录。完整操作见第2.4节。

保存 Setup 时保留环境引用和工程显式配置，不把整个展开环境写回项目。运行时偏好写入使用全局互斥锁，将相关列表/UI 表合并到最新文档。检测到外部启动配置冲突时拒绝盲写并要求重新加载；跨进程修改及崩溃期间并非数据库式原子事务。

Examples 是主动替换启动草稿的操作，不能理解为只填补空字段；它不立即写文件。Projects 则载入所选项目并重新派生所有显示字段，错误时保留此前有效草稿。保存始终写所选 Project，不改写引用的 profile；没有 TOML 的目录在 Save config 或 Start 时才产生文件，进入 Setup 本身不保存。

14.2 核心字段

字段	默认/类型	说明
version	支持至 3	Chip/Core 选择要求 3；旧式配置可保留 1/2。旧 [server] 被拒绝并提示迁移。
tools.profile / root	空/路径	profile 优先，root 对应 root/debug-env.toml。

字段	默认/类型	说明
debug.chip / cores	空/编号数组	型号和物理核心编号子集；未选芯片沿用旧模式，选中后校验本地目录和后端。
core_preferences	按芯片/核的表	保存 watch 与 breakpoints；核名称为 core.N，保留未选核的偏好。
gdb.executable	gdb	与目标架构匹配的 MI2 调试器。
gdb.args / init	空数组	程序参数 / GDB 初始化命令。
gdb.cwd / env / unset_env	可选/映射/数组	宿主进程环境；默认 LC_ALL=C。
gdb.registers	空数组	空表示读取所有命名寄存器；非空按名称筛选。
target.mode	remote	remote、extended-remote 或 local。
target.endpoint	空字符串	远程 HOST:PORT；多核可由各 core.endpoint 提供。
target.after_connect	空数组	建立目标连接后执行。
program.elf / svd	空路径	符号/程序与可选外设描述；ELF 可省略。
program.source_root	项目目录	源码定位及外部 tasks 的工作目录。
service	可省略	没有此表时只启动 GDB。
service.enabled	true	false 使用外部服务。
service.command / args	路径/数组	通用进程启动，不以 shell 拼接。
service.cwd / env / unset_env	可选	独立于 GDB 环境。
service.ready	空数组	非空时等待任一就绪片段；空则启动后直接连接。
service.timeout_ms	15000	服务就绪等待上限，必须大于 0。
actions.run	空数组	空时默认 -exec-run；芯片 profile 可声明 reset/continue。
actions.restart / download	空数组	为空时动作不可用，download 可由 tasks 覆盖。
actions.before_disconnect	空数组	停止后、detach/disconnect 前执行。
session.on_exit	detach	detach、resume、disconnect。
session.timeout_ms	8000	普通 MI 请求超时；不是全部链路的唯一超时。
session.log_dir	未配置	配置后创建独立会话日志。
tasks.build / download	空字符串	单行 shell 命令，工作目录固定 source_root。
tasks.timeout_ms	300000	外部任务超时，必须大于 0。
watch / breakpoints	空数组	表达式列表 / 兼容字符串或详细断点对象。
source_map	数组表	from 是编译路径前缀，to 是本地目录。

字段	默认/类型	说明
source_remap	enabled/from/aliases	enabled 为可选布尔值；from 为 ELF 目录前缀，目标跟随 Source root；aliases 保存原路径拼写。
ui.animations / unicode	subtle / true	off、subtle、full；Unicode 动效字符开关。
ui.formats / refresh	空映射	数值格式 / 按核按项的读取策略。

14.3 多核和总线字段

配置	含义
`backend / backends`	工具环境中的单后端标识/多后端覆盖分组，与用户目录中芯片的 backend 匹配。
`[core_targets."N"]`	工具环境提供该物理核的 endpoint 和可选 ready 字符串；也可放入所选 backend 分组。
`[[cores]]`	name 唯一；远程 endpoint 唯一；startup_order 默认 0。可配 init、after_connect、run、watch、breakpoints。
`[[images]]`	1.3.0 的常驻镜像：唯一 id、elf、可选 cores、domain、source_root/source_map、offset/sections、memory_channel；配置及限制见第13.6节。
`[images.saved_context]`	所属 Guest 的显式快照 ABI、身份、索引、通道和允许内存区；只读 Guest 视图见第13.8节。
`[multicore]`	scope 默认 core；halt_peers 默认 true；共享复位由 restart_core 和 restart 数组定义。
`[[memory_access]]`	唯一 id、label、tcl_endpoint、target、while_running；cores 可选限制通道可用于哪些核。
`[live_watch]`	tcl_endpoint、bus_target、elf、正数 interval_ms；工程配置，非环境表。
`[sync]`	method 接受 tcl、cti 或空；open 为显式 TCL 数组，非空时要求显式 cores；失败中止连接。

核的 watch/breakpoints 未配置表示继承顶层，显式空数组表示不继承。但 **core.after_connect 与 core.run 的空数组在当前 derive_project 中仍保留顶层动作**；要清除应在顶层重新设计动作，不能照搬所有数组的覆盖规则。core.init 追加在共享 gdb.init 后执行。

```
[[source_map]]
from = "D:/old/build/workspace/firmware"
to = "."

[ui]
animations = "subtle"
unicode = true
[ui.formats]
"watch:xTickCount" = "hex"
```

源码映射用于文件定位，不能修复 ELF 与烧录镜像不一致；流水线产物的配置、操作和排查步骤见附录D。

实现依据：`src/config.rs`: Project::from_document/validate、各 Default；`src/cli.rs`: document；`src/launch.rs`: Document::save；`src/coordinator.rs`: derive_project。

15 实现架构、模块与并发关系

15.1 架构分层

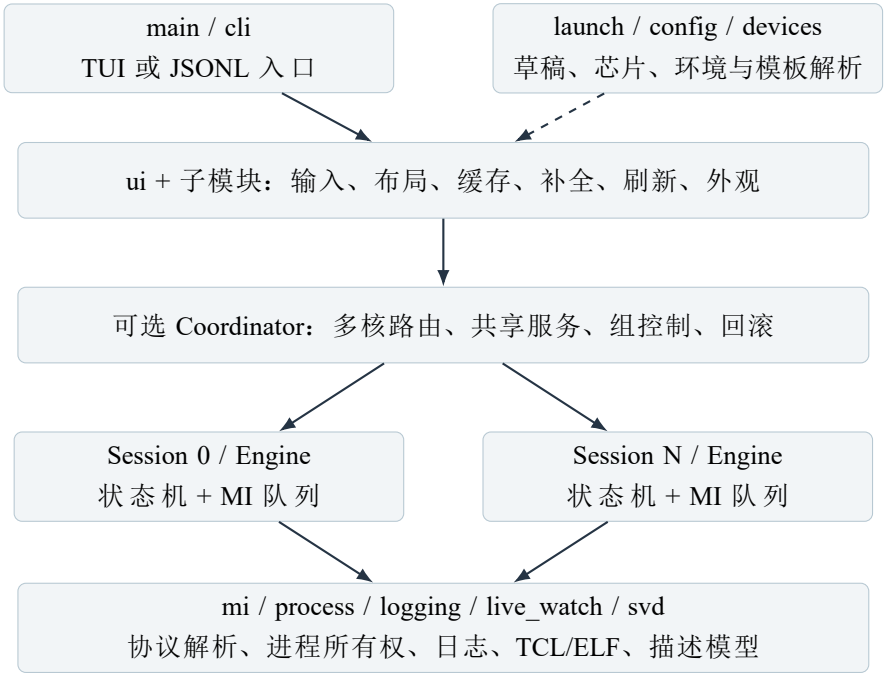


图 12: 代码职责图。Legacy 单核未配置 cores/live_watch/sync 时直接创建 Session；Chip 模式先派生显式 cores。

普通单核仅配置 memory_access 也不增加 Coordinator 线程；逐项 TCL 读仍在 Session 请求路径中执行。显式 cores、live_watch 或 sync 任一存在时走协调层。即使 cores 只有一项，也属于显式核工作区；工作区索引按最终核心数组位置，启动顺序另由 startup_order 排序。Chip 模式用物理编号生成 core.N 名称；例如只选 core1，工作区 index 为 0，但名称仍为 core.1、端口仍取物理核 1 的映射。JSONL 的 core index 不应当作芯片物理编号。

15.2 离线源码目录扫描与统一映射

Setup 重映射通过独立工作线程启动短生命周期 GDB，使用 `-nx -nh`、关闭 auto-load，并先设置 source open off；发送 file-exec-and-symbols、file-list-exec-source-files、gdb-exit。只复用选定 GDB 的可执行文件及环境变量，不带 profile 的 args/init/cwd，不创建 Server 或调试连接；工作目录为项目 TOML 所在目录。这样不会为扫描 ELF 执行旧 profile 中的复位或下载钩子，普通调试会话的加载流程仍独立存在。

扫描结果去除虚拟编译文件和控制字符项，以目录层级建立候选，统计映射到 Source root 后的文件存在数并显示样例。列表是候选目录，不是 DWARF 类型/调用关系浏览器。取消、重新扫描和退出弹窗会发出取消标记，子进程由拥有者 kill/wait 回收；UI 通过消息接收结果，不阻塞终端输入。

边界	当前实现
GDB 元数据等待	使用 session.timeout_ms 并限制为 1–30 秒；不可达旧网络路径仍可能超时。
输出与列表上限	GDB 输出最多 32 MiB；过滤后的源文件条目与候选目录各最多 50,000。
路径比较	正反斜杠归一化，按完整目录边界匹配；不会将 /ci/app 匹配到 /ci/app2。大小写按原字符串比较，不自动折叠盘符或路径大小写。
映射的消费者	TUI 本地文件定位、GDB substitute-path 使用同一有效规则；对原拼写、正斜杠、反斜杠生成去重别名。
检查的含义	matched 表示本地文件存在，不比较文件内容/Git 提交，不保证行号和固件匹配。

开关未设置时，已有 source_map 或非空 source_remap.from 维持启用，兼容旧工程；显式 false 停用两者但保留内容。from 的目标始终是当前 Source root。手工规则中与选中前缀相同的项被选中映射替代，其余手工规则保持顺序并在前；没有自动“最长前缀优先”排序，应把更具体的手工前缀放前面。用户在 GDB init/Console 自行发出的 substitute-path 不属于这套开关。

实现依据：`src/source.rs`；`src/launch/remap.rs`；`src/config.rs`: effective_source_maps/source_path；`src/session.rs`: connect。

15.3 线程和数据结构

UI 主线程维护 App、面板状态与输入；Session 工作线程拥有 GDB、Server、Snapshot、断点/Watch 定义、按通道 TCP 缓存。GDB stdout 读取线程解析 MI，stderr 和服务输出读取线程采集日志。旧式 Live Watch 配置增加独立采样线程。Windows 下所有自有进程归入 Job Object。

对象	关键内容
Request	id: u64、method: String、params: JSON。
Event	snapshot、response、log、live_watch、exit。
Snapshot	state、stop_reason、frame、stack、locals、watches、registers、breakpoints、files、assembly、memory、generation、async_supported；多核再带 core/cores/control_scope。
Variable / WatchTree	name/value/changed/error，及可选类型、成员路径、子节点、展开和分页标志。
Breakpoint	每核真实 id、location、kind、enabled、condition、ignore_count、hit_count、restore_error；可选 group/cores。
EngineHandle	命令发送端、事件接收端和共享取消标志 Arc<AtomicBool>。

命令通道采用 mpsc，事件为容量 512 的 sync_channel；GDB 解析记录通道容量 256。日志缓冲有界，不能认为在无限输出压力下保存所有历史。Worker 正常约每 20 ms 检查请求，等待 MI 时仍消费异步记录，避免把 `*stopped` 误当作请求结果。

15.4 UI 的按需工作与资源边界

Asm/Memory 根据停止代次、栈帧地址及层级失效，等待前台命令结束才读取；失败后等待显式 refresh 或下次停止，不高频重试。Source 只保留当前文件文本，各标签保存路径/滚动状态。SVD 有独立寄存器缓存，不写 Memory 面板快照；实时采样有独立缓存，不回写普通 GDB Snapshot。

外观模块消费已有事件，不访问 GDB：短动画最高约 25 FPS，运行/忙碌提示约 2.5 Hz，空闲或失焦暂停装饰绘制。效果上限包括 256 个短脉冲、2048 个值历史、3 个实际停止落点；数值变化直接显示最终值，动画不伪造运行进度。

0.8.4 的 `ui/source_text.rs` 保存选区 anchor/caret、横向视口及菜单状态，以 UTF-8 字节位置截取原文，以字符显示宽度换算屏幕坐标。highlight 提供对应文本范围，render 在着色结果上叠加选区并固定行号栏；切换文件、核心及停止上下文时清理相关状态。`clipboard.rs` 只在显式复制动作中写宿主剪贴板，Add to Watch 则沿用现有请求路径，不新增另一套 GDB 求值或多核路由。

`launch/choices.rs` 负责项目筛选和示例生成，应用选择时经原 Document 解析与校验后替换草稿；浏览和选择阶段不创建调试进程。Setup 的直接 Exit 汇入既有退出/清理流程。这些界面功能没有引入新的调试工作线程或第三方运行时依赖。

实现依据：`src/session.rs`: Engine、spawn、run；`src/coordinator.rs`: spawn；`src/ui.rs`: ensure_visible_data；`src/ui/effects.rs`；`src/ui/source_tabs.rs`。

16 MI、RSP、TCL 的报文与流转

16.1 GDB/MI2：token 与异步记录

DebugTUI 给每条 MI 命令加递增 token，写入 GDB stdin。只有 `^` 且 token 匹配的结果记录结束当前请求；其间的流输出、运行/停止通知继续更新会话。`^done` 或 `^running` 是命令结果，不能替代真正的停止通知。MI 语法详见GDB 官方输出语法。

```
17-exec-continue
17^running
*running,thread-id="all"
*stopped,reason="breakpoint-hit",bkptno="1",
  frame={addr="0x08001234",func="Task100ms",line="40"}
```

上例为便于阅读的逻辑报文示意，实际每条 MI 记录为一行；地址/行号不是本次实板采样。解析后的 Record 保存 token、kind、class 和 Value。Value 有 Text、Tuple（键值对向量）与 List，可保留重复键和顺序，处理引号、反斜杠、八进制 UTF-8 转义以及嵌套边界。

标记	类别	核心处理
`^`	结果	token 匹配，error 返回 msg；其余交给调用者。
`*`	执行异步	running/stopped；停止增加 generation、清空过期反汇编/内存。
`=`	通知	breakpoint-* 触发后续刷新。
`+`	状态	尝试处理 download 的 total-sent/total-size；已知匿名 tuple 例外见限制。
`~`	GDB 控制台流	日志通道 gdb。
`@`	目标输出流	日志通道 target；内部 TCL 回显可被过滤为 diagnostic。
`&`	诊断流	diagnostic。

单条 MI 记录限制 1 MiB；遇到超长记录读取线程结束并报告。普通 MI 超时进入 FAULT，要求重连恢复，避免迟到结果误配新请求。

16.2 RSP：由 GDB 和 Server 实现

远程包基本形式为 `$payload#checksum`，存在应答和可协商特性；常见内存、寄存器、继续和停止回复由 GDB/Server 处理。`monitor` 经 GDB 转为远程命令，故 monitor reset 与 monitor resume 的语义由服务器定义。DebugTUI 不自己封装 RSP，也不能从 MI 一条命令推定对应一个 SWD 总线事务。

16.3 TCL RPC：显式 target 与错误封装

OpenOCD RPC 在 TCP 上以字节 `0x1a` 结束命令和响应。客户端对 TCL 单词转义，使用 catch 携带返回码，格式化为 `__DEBUGTUI_RPC__<code>:<value>`；只把 code=0 作为成功，并检查完整终止符、UTF-8、宽度与数量。

```
# Logical payload before the status wrapper:
"stm32f4x.ahb" read_memory 0x20000000 32 1

# Status-bearing response, followed by byte 0x1a:
__DEBUGTUI_RPC__0:1234
```

总线读取地址示例不承诺该处是变量。`target read_memory` 的对象选择语义见OpenOCD target配置文档。每通道按需复用一个 TCP；故障关闭缓存连接，后续重试重新连接。连接/读写超时约 500 ms，事务循环还设 2 s 截止和 8192 字节响应上限，实际可能先触发 socket 超时。

16.4 一次“停止并查看变量”的时序

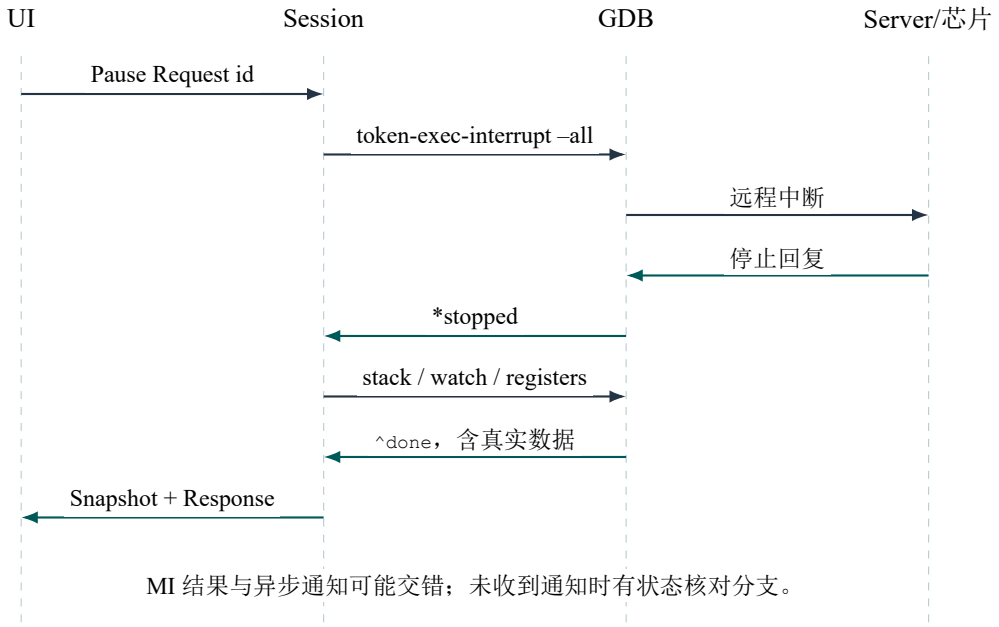


图 13: 正常暂停路径。GDB 的中断确认不等于目标已经停止。

实现依据：`src/mi.rs`；`src/session.rs`: request、record、interrupt_target；`src/live_watch.rs`: connect、word、transact。

17 连接、执行、暂停与退出状态机

17.1 连接流程



图 14: 连接成功由后续查询确认；中间失败清理本次启动的资源，多核回滚已连核。

没有 ELF 可以连接，但源码和表达式信息受限。配置了 Build 且 ELF 尚不存在时允许先进入工作区构建；“能打开工作台”不等于“已连接目标”。Setup 中再次 Start 先清理旧会话并等待退出，再保存和启动新会话；清理失败停止切换。

17.2 稳定状态和暂态

状态	解释与操作
DISCONNECTED	无有效连接；可配置、Build、Connect。
STARTING SERVER / STARTING GDB / CONNECTING	正在准备；不得当作已暂停或可以读寄存器。
READY	GDB 有效，但没有活动 inferior/可用栈帧，例如本机尚未 run 或程序已退出；可设代码断点。
STOPPED	目标停止，可读普通快照、单步、设置数据观察点和执行需停止的动作。
RUNNING	普通面板保留缓存；可 Pause；只有明确能力的通道可运行读。
DOWNLOADING	执行环境下载动作；MI 完成后重新核对目标状态。
DISCONNECTING	清理、detach/disconnect、退出 GDB 和自有服务。
FAULT	GDB 关闭、请求超时等异常；目标状态可能未知，应检查日志再 Reconnect。

17.3 暂停竞态的处理

Pause 先发 `-exec-interrupt --all`，等待真实异步停止。约 250 ms 尚未确认时发 `-thread-info`：全部线程确认为 stopped 才同步停止状态；无 inferior 则进入 READY。目标仍运行时在约 1 s 后最多补发一次 interrupt，等待流程有界，通常使用 5 s 停止等待窗口（具体 MI 请求仍受各自超时影响）。

GDB 报“目标已停止”或“Inferior not executing”时也先核对，不通过 reset 或重连伪造成功。对运行中明确拒绝 thread-info 的远程 GDB 保持等待；超时错误表示目标可能仍在运行。新停止增加 generation，使旧汇编、内存和实时采样代次失效。切核还有协调层 revision，避免后台核数据覆盖当前核。

17.4 Build/Download 与退出流程

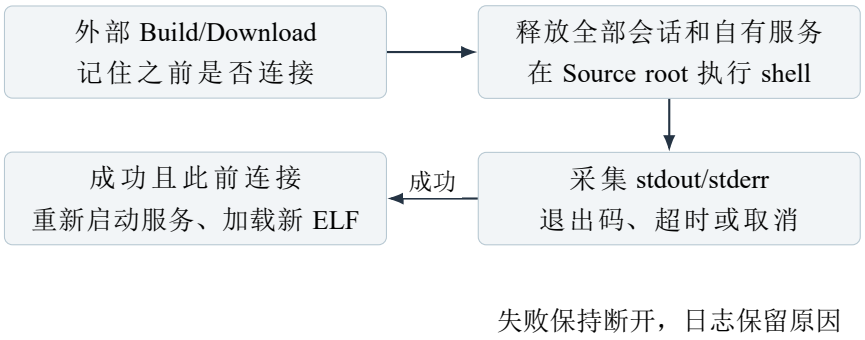


图 15: 外部任务流程；GDB actions.download 在当前会话/当前核执行，路径不同。

Windows 的外部任务通过 `cmd.exe /D /V:OFF /S /C` 执行；PowerShell 脚本须显式写解释器。任务字符串不是 GDB 命令数组；GDB actions 中以连字符开头的命令按 MI 发送，其他命令通过 GDB Console。原始配置命令有真实执行能力，应使用可信工程配置。

正常断开依次取消总线连接、必要时 Pause、查询并记住断点、删除本会话目标断点、执行 before_disconnect，再按策略 resume/detach/disconnect，退出 GDB，回收自有服务，保存 Watch 和断点定义。错误会被汇总并报告，不能把“进程消失”当作“目标状态正确”。

Windows Job 设置 KILL_ON_JOB_CLOSE，应用异常结束时回收自有进程树；这不保证硬件 detach 成功，也不替代板卡运行状态检查。非 Windows Job 实现目前是空壳，进程树清理与发布验证尚不具备同等保证。

实现依据：`src/session.rs`: connect、sync_target_state、wait_for_stop、disconnect、project_task；`src/process.rs`；`src/ui.rs`: SWITCH_QUIT。

18 多核工作区与跨核断点

STM32F429 是单核例子；本节配置用于多核平台的实现说明，不应向 F429 虚构第二个 CPU。每核有独立物理 GDB 进程和 endpoint，并可共享 Server。传统配置共享默认程序/描述；1.3.0 可通过 images 按核心绑定 Bao/Guest ELF，见第13.6节。

```
[[cores]]
name = "core0"
endpoint = "127.0.0.1:3333"
startup_order = 0
watch = ["counter"]

[[cores]]
name = "core1"
endpoint = "127.0.0.1:3334"
startup_order = 1
watch = []

[multicore]
scope = "all"
halt_peers = true
restart_core = "core0"
restart = ["monitor chipreset"]
```

`chipreset` 必须由目标板级脚本提供，上例不是通用 OpenOCD 命令。Connect/Run 按 startup_order 顺序等结果，只保证命令顺序，不保证前一核完成固件初始化。全局连接失败断开全部核并清理自有共享服务。重复 Connect 报错并保留现有连接，重建使用 Reconnect。

操作	作用范围
Connect / Reconnect / Disconnect / Exit	整个工作区。
Run All	每次连接/共享复位后先执行各核 run，之后继续；已运行核跳过。结构化 run 可用 scope=core。
Continue/Pause	根据 Scope All/Core；单请求可覆盖 scope，不改变会话默认范围。
Step/Next/Finish	只推进当前核；All 模式先暂停其他运行核。跨核等待/锁可能因此无法前进。
断点自动联停	All 且 halt_peers=true 时观察新停止，暂停其他运行核并切至触发核。
共享 Reset Chip	一次通过指定核执行复位，然后清每核寄存器缓存并刷新；即使 Core 范围也作用整片。
Watch/原始 GDB/新建断点	当前核；显式关联的断点编辑另按成员核分发。

18.1 协调器的响应和失败语义

Coordinator 将用户 id 与内部逐核 id 分离。一个 Batch 顺序等待子步骤，记录每核 result/error；汇总后回复用户。内部断点联停不会冒充用户 response。某核继续失败时尝试停住已运行核；暂停某核失败仍尝试剩余核；共享复位后的状态须逐核刷新。

18.2 关联断点不是相同地址推测

新建断点默认当前核。选中已有持久代码/硬件断点，Cores/c 勾选其他核，Ctrl+Enter 应用；当前核必须保留，临时断点和数据观察点不支持这种关联。每组保存稳定 group，但每核 GDB 编号独立，快照 cores 来自真实列表。

关联启停、修改条件/忽略数、删除会分发全部成员；Enable/Disable all 还包含当前列表中的关联外核。操作前要求受影响核均 READY/STOPPED，不会隐式 Pause。添加或编辑失败时反向执行 undo，连失败步骤也尝试恢复以覆盖“目标已改但配置保存失败”；回滚失败同样报告。软件断点会修改指令，若多个核共享可写代码内存，单核安装也可能影响其他核，应使用合适的硬件断点策略。

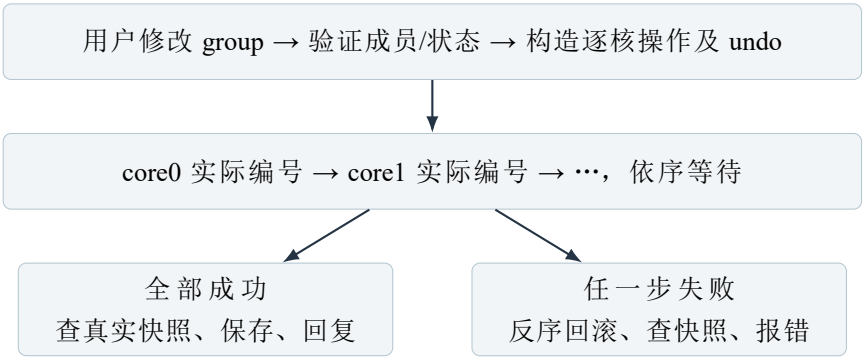


图 16: 跨核断点修改是有回滚的顺序协调，不是崩溃原子事务。

18.3 sync、CTI 和 AP 适配边界

第9.6节已说明 CTI/CTM 的硬件路径；这里对应的是其在工作区生命周期中的配置入口。`sync.open` 在共享 Server 就绪后、GDB 连接前执行，method=tcl 或兼容的 cti 都只发送明确配置的 TCL 字符串。TUI 不识别 CTI 拓扑、不配置 AP、不承诺硬件联停；CTI base、通道、核关联和 MEM-AP 路由均属板级脚本。多个通道可用不同端口，CPU target 名称也不等于固定 AP 编号。

本次已定位并复核 THA6206 双 Cortex-R52 的 2026-09-20 原始 result/events 记录，覆盖组控制、共享复位、运行中 Watch 和多核断点，见第12节。Bao 双 VM 的另一组证据见第13节；它不能替代双 VM 加逐核 Guest C 符号的完整 TUI 验收；9 月 29 日已新增 Bao smoke 的双 GDB 编译下载回归，见第13.14节。较早“无多核板卡”或“缺原始目录”的描述属于当时的资料范围。

实现依据：`src/coordinator.rs`；`src/coordinator/control.rs`；`src/coordinator/breakpoints.rs`；`src/ui/breakpoints/cores.rs`；`docs/examples/multicore-access.md`。

19 Headless JSONL 自动化接口

19.1 启动、请求与事件约定

```
debugtui --project ./debug.toml --headless --stdio
debugtui --project ./debug.toml --script ./commands.jsonl
```

stdin 每行一个 JSON 请求，空行忽略；stdout 每行一个事件，立即 flush。`--script` 自动启用 headless，顺序执行文件；请求失败停止后续脚本并清理，返回非零退出码。脚本 EOF 自动发 quit；交互 stdin EOF 也进入清理。`--stdio` 本身也启用 headless。`params` 可省略，但需要参数的方法会自行校验。

```
{"id":1,"method":"connect","params":{}}
{"id":2,"method":"watch","params":{"expression":"xTickCount"}}
{"id":3,"method":"continue","params":{}}
{"id":4,"method":"pause","params":{}}
{"id":5,"method":"evaluate","params":{"expression":"xTickCount"}}
{"id":6,"method":"quit","params":{}}
```

该示例假定远程 profile 连接后暂停。脚本没有内置 sleep；需要运行一段时间再采样时使用外部客户端控制请求时机，而不是把多行紧邻请求当作等待。

```
{"event":"response","id":5,"ok":true,
 "result":{"value":"12345"},"error":null}
{"event":"snapshot","snapshot":{"state":"STOPPED", ...}}
{"event":"log","channel":"gdb","text":"...",
 "timestamp":"...","elapsed_ms":1200}
{"event":"live_watch","sample":{"expression":"uwTick",
 "address":536870912,"bits":32,"value":12345,"error":null,
 "timestamp":"...","core":null,"generation":4}}
{"event":"exit"}
```

上段为事件结构示意，省略号不是合法 JSON，实际输出包含完整字段。客户端必须按 event 和

response.id 分类，不要把 stdout 每行当作一个请求结果；异步日志和快照可穿插。普通 `status` 返回缓存状态，不保证进行新的硬件读取。`continue/step` 成功可能只表示已发出执行动作；要等待停止使用 `wait_stopped` 或监听 snapshot。

19.2 方法参考

method	params	行为/条件
connect / reconnect	`{}`	连接/先清理再连接。
disconnect / quit	`{}`	断开/断开后终止 worker。
status	`{}`	当前 Snapshot。
continue / run / pause	可选 `scope:"core"|"all"`	scope 适用于多核协调。
wait_stopped	`timeout_ms`	单核默认 10000、最大 60000 ms；All 组默认 8000、限制 1–120000 ms。
step / next / stepi / finish	`{}`	源级进入/越过、指令进入、跳出；需要 STOPPED。
restart	`{}`	执行配置的复位，组策略另见前节。
evaluate	`expression`	停止求值，返回 value 文本。
watch / unwatch	`expression`	增删根表达式；非停止时新增先占位。
watch_expand	expression、path 数组、expanded；可选 more	path 为子索引路径；展开需停止，折叠本地处理。
watch_resolve	expression，可选 path	停止时返回 address、bits、signed、float、little_endian、type。
memory_read	数值 address、bits、little_endian；可选 channel	channel 空走默认 GDB；非空走命名 TCL。
peripheral_read	数值 address、bits、little_endian	停止读单寄存器，不修改 Memory 面板。
memory	文本 address，可选 count	停止；count 默认 256，限制 1–4096 字节，更新 Memory 快照。
disassemble	可选文本 address	默认 `$pc`，读取起点之后 128 字节范围。
frame	level	选择栈帧并刷新上下文。
files / refresh	`{}`	文件列表 / 刷新当前停止上下文。
break	location；可选 hardware、temporary、enabled、condition、ignore_count	代码断点可在 READY/STOPPED 创建。

method	params	行为/条件
data_break	expression；access=write/read/access；可选 enabled、condition、ignore_count	数据断点需 STOPPED。
enable_break	number 和 enabled；或 all=true、enabled	启停；number 是字符串。
update_break	number；可选 enabled、condition、ignore_count	修改现有定义。
delete_break	number	删除指定编号；单核缺失/空 number 会删除全部断点，应显式传入。
break_cores	number、cores 索引数组	多核关联持久代码断点，必须包含当前核。
select_core / cores	index 或 name / 空参数	多核切核/查看核列表。
control_scope	scope=all/core	多核改变本会话范围，不写回 TOML。
console	command	原生 GDB；一般要求 READY/STOPPED，部分执行别名特殊路由。
complete	text；可选 expression: bool	符号/命令补全，不执行输入。
images	`{}`	当前核的常驻镜像和识别状态；运行时身份为最后已知。
guest_context	image；可选 level、expressions	在有效的物理停止窗口只读查看 Guest 保存寄存器、栈和局部值。
return_guest / cancel_guest_return	image / 空参数	当前核完整 HVC32 返回与取消；物理状态/资源约束见第13.9节。
set_elf	path	仅单核且已断开；多 ELF 在 Setup 维护主 ELF 与 images 后重新 Start，不提供运行中热替换。
build / download	`{}`	执行配置动作；headless 无 UI 下载确认。
ui_preferences	Ui 对象	保存 UI 配置，无 MI；建议先读取工程并完整合并。

`break_apply`、`synchronize`、`restart_shared` 为协调实现内部使用的方法，客户端应使用上表公开操作，避免绕过组控制约束。当前接口是项目自定义 JSONL，不含 `jsonrpc:"2.0"`，没有网络多客户端服务或正式协商版本字段；自动化应固定程序版本并保留原始事件。

19.3 客户端实现注意事项

1. 使用不重复的非保留 u64 id；将 response.ok/error 与目标 state 分开判定。内部 EOF quit 使用 u64 最大值。
2. 持续消费 stdout，避免有界事件通道因客户端不读而阻塞；stderr 保存启动/脚本错误。
3. 先 connect 并确认状态；执行控制后根据需要等 wait_stopped；退出等 response 和 exit。
4. 数值 address/bits/value 使用 JSON 数字。JavaScript 对大于 $2^{53}-1$ 的整数不能精确表达，应使用保留整数精度的解析库或原始 JSON 文本处理；不能在解析为 Number 后再转 BigInt 修复。
5. 64 位内存值也不保证总线原子性。自动化调用 peripheral_read 是通用内存操作，核心不知道地址对应的 SVD 副作用；调用者仍应应用元数据访问规则。

实现依据：`src/main.rs`: run_headless；`src/session.rs`: Request/Event/execute；`src/session/memory.rs`；`src/coordinator.rs`。

20 将 DebugTUI 适配到其他芯片

新的芯片适配通常修改工具环境和板级脚本即可，无需重新编译 TUI。按以下顺序建立可验证的链路，比一次启用全部功能更容易定位错误。

1. **确定硬件信息**。型号、核架构、端序、Flash/RAM、调试接口、供电、复位、保护状态、DAP/AP/CTI 拓扑从该芯片资料和板卡原理图取得。
2. **独立验证 Server**。选其支持的 target/Flash 驱动，确认探针可连接、识别 CPU、暂停和读稳定内存。OpenOCD/J-Link 不能互换全部 monitor 命令。
3. **独立验证 GDB**。使用兼容目标架构和 ELF/DWARF 的 GDB，手动 remote/extended-remote，核对寄存器、PC、符号、断点、继续/暂停和退出。
4. **写环境 profile**。定义 GDB 和 Server 资源路径、ready、端口和 after_connect；仅加入已验证的 run/restart/download/before_disconnect。
5. **配置项目**。正确 ELF、source_root、源码映射、任务命令、SVD。核对 ELF 与烧录镜像，不因文件名相同而假定相同内容。
6. **增加运行读取**。根据实际 AP 路由建立 mem_ap target，声明 memory_access，先停止对照，再运行采样，验证无隐式停核及读后正常调试。
7. **增加多核**。每核唯一 endpoint、明确启动顺序和共享复位 owner；再验断点联停、单步隔离和部分失败恢复。CTI 必须单独做硬件验收。

移植项	验收标准
连接	重复连接/失败/重连行为清楚，外部服务不被误回收。
符号/源码	PC 所在函数和源行合理，compare-sections 或等效验证一致。
控制	连续 Continue/Pause、长时间运行后暂停、源级/指令单步均正确。
断点	代码、硬件、临时、支持的数据模式实际触发；资源耗尽给出错误。
数据	CPU/GDB 与总线稳定值一致，字节序、宽度、对齐、错误地址均有结果。
运行读	无隐式 halt/resume，旧地址生命周期清楚，隐藏逐项面板停止轮询。
下载	Flash 算法、SRAM 工作区、加载段、向量表/启动地址和校验明确。
退出	明确板上保持/恢复策略，无自有进程或监听残留。

对于 RISC-V 需要自备匹配的 GDB 和服务；1.3.0 的 images 允许按核绑定不同 ELF，但不代表已适配不同架构、不同 SVD 的全部异构核；本次隔离求值实测限 ARM32 小端。对于 Cortex-R/A，复位初态某些浮点寄存器不可读可配置 gdb.registers；总线值与核缓存之间的一致性由芯片/服务配置保证。

实现依据：`docs/examples/multicore-access.md`；`src/config.rs`。

21 日志、诊断与常见故障

21.1 日志组织

`--log-dir PATH` 或 session.log_dir 开启落盘。每次连接新建 `session-日期-时间-毫秒.log`，通过 create_new 防覆盖；时间在产生/读取日志时捕获，文件每行含绝对时间和单调会话耗时。多核日志在 `core-N/`，共享服务在 `coordinator/`。Console 是阅读过滤视图，不代替完整会话日志。

21.2 按故障层次查找

现象	先查什么	处理方向
无法找到 GDB / DLL	executable、data-directory、PATH、工具包完整性	单独运行 –version；按定义文件核对相对路径。
Server 启动超时	server 输出、ready 文本、USB 驱动、端口	确认真实监听文本；已运行服务用 –connect。
Could not connect to J-Link	探针驱动是否为 SEGGER 或 WinUSB/libwdi	历史本机存在驱动不匹配；用已验证的 OpenOCD profile 或正确安装相应驱动。

现象	先查什么	处理方向
J-Link 约 52–53 秒后失效	首个异常时间、monitor 读取、Server/DLL/固件与探针身份	已有绕过 TUI 的历史复现；clone 是待验证线索。详见第21.3节，勿仅增加超时或反复自动重连。
源码/变量与实际不符	ELF、Flash 校验、优化、路径映射	先确认镜像一致，再修 source_map；不要只换源码。
单步跳行、变量 optimized out	编译优化和调试信息	使用匹配的调试构建；本地固件为 -O0 -g3。
RUNNING 时旧值不变	是否普通停止快照；是否启用了合法通道	LIVE 只来自成功采样；先暂停解析地址，启用 ahb。
Live read unavailable	通道是否存在、cores 限制、while_running、地址类型	默认 GDB/core-tcl 停止限定；无地址表达式不能总线读。
运行读与 GDB 不相等	是否同一时间、DMA/其他核、缓存、类型/端序	全部相关核停止后 Refresh，再对稳定值比较。
Peripherals 无值	SVD 路径、展开/可见状态、访问属性、位宽/对齐	只写不读；副作用项只能显式手动操作；未知 endian 核对源码默认。
暂停超时	MI interrupt、thread-info、异步 stop、服务端状态	保留日志；FAULT 后显式 Reconnect；不要把重置当作暂停。
断点恢复 [!]	符号位置、可用硬件资源、数据类型、错误文本	暂停后重试启用或删除；记录未被静默丢弃。
下载进度 protocol error	是否 +download 匿名 tuple；最终 `^done` 和段校验	当前已知显示问题，核对实际下载结果，勿据进度条判定成功。
Download command 显示空	tasks.download 与 actions.download 的层级	空脚本可回退 GDB action，确认框显示实际动作。
Build 不找到 make/GCC	Source root、继承 PATH、Makefile 工具依赖	先在同一目录/环境独立构建。
终端功能键无响应	焦点、终端快捷键拦截、尺寸	使用可见按钮；检查宿主终端按键路由。
退出后板仍运行	before_disconnect 和 on_exit	本地 profile 明确 resume/go 后 detach，这是配置行为。

定位时记录程序版本、源码 SHA、profile、匹配 ELF 的哈希、探针/Server 版本、复现步骤、原始 MI 日志和最终目标状态。保存日志可包含变量和源码路径，应按工程资料的敏感程度管理。

实现依据：`src/logging.rs`；`src/session.rs`: log_at、start_server；`src/coordinator/server_log.rs`；本地历史报告。

21.3 案例：GDB + J-Link Server 在约 52–53 秒后失效

这一问题已在本地历史测试中复现，本版手册将它列为**尚未确认根因的长会话故障**。用户描述为“调试会话断开”，并怀疑所用 J-Link 为 clone 硬件。原始证据更精确地显示：目标访问先失效，出现寄存器读数异常、断点操作失败或停止等待超时；不能将每次异常一概等同为 TCP socket 断开或 Server 进程退出。

复现条件与时间证据

2026-09-16 的隔离实验直接启动 ARM GDB/MI 和 J-Link GDB Server V7.94e，绕过 DebugTUI。日志中 DLL 为 V7.94e，探针报告 `Hardware: V8.00`、`Firmware: J-Link ARM V8 compiled Jan 31 2018 18:34:52`；目标 STM32F429IG，SWD 4000 kHz。型号与固件字符串是设备自报信息，不是正品鉴定结论。

测试使用当时的 FreeRTOS ELF；六个只读段已与 Flash 匹配，随后在同一断点循环 Continue 和对应操作，不下载固件。每组短会话为 3 × 100 次，四组合计 1200 次通过；每会话约 18–23 秒。扩大单会话次数后出现如下结果：

分组	已完成轮数	首个异常时间	首个异常所在阶段/症状
Next	241	52.522 s	第 242 轮 Continue；寄存器失真，随后等待停止超时。
Step	249	53.325 s	第 250 轮 Continue；Server 报断点资源不足/插入失败，GDB 随后 Command aborted。
Stepi	268	52.575 s	第 269 轮 Continue；断点插入失败和 Command aborted。
Monitor Step	310	52.893 s	第 311 轮 Continue 后 PC/SP/LR/xPSR 同为 0xab78；monitor regs 为零。

时间基准为隔离脚本的会话开始、紧邻 Server 启动，**不是点击 UI Connect 后精确倒计时**。上表取自原始 trace 首次异常；`result.json` 的 elapsedMs 还包含后续等待/诊断，不能用它替代首次故障时间。四组异常都发生在返回断点的 Continue 阶段，不能据分组名称认定是四种单步算法各自有错。

静置实验进一步排除了“必须反复单步才能触发”：停在断点后 30 秒不发命令，读取正常，后续 10 次 monitor 单步通过；两次静置 60 秒后，在尚未执行单步时 `monitor regs` 已全零。刷新 GDB 寄存器缓存后，PC/SP/LR/xPSR 分别全部变成 `0x5428` 或 `0x9768`。CPUID（0xE000ED00）、DHCSR（0xE000EDF0）、CFSR（0xE000ED28）、HFSR（0xE000ED2C）读取也返回 Failed。

MI 的 `^done` 只说明 monitor 命令完成传递/处理，不保证其文本内的目标读取成功。Step/Stepi 日志还出现同一 token 先 `^running`、后 `^error,msg="Command aborted."`；只接受第一次响应会把真正的执行错误表现为等待超时。

clone 线索应如何解释

当前证据支持“异常不依赖 DebugTUI 的渲染与请求调度，也不依赖是否执行源码单步”。它尚不能在 J-Link Server/DLL、探针固件/硬件、USB/SWD 链路与目标调试状态之间做唯一归因。用户对 clone 探针的判断是重要排查线索，本手册保留这一信息；但本次核对的长会话原始日志没有发现明确的 clone/counterfeit 拒绝文本，也没有更换已知正常探针的同条件 A/B 结果。

因此不能把 52–53 秒直接写成“SEGGER 固定反 clone 超时机制”，也不能把 V8 自动等同为假货。SEGGER 对旧硬件的支持范围应按 J-Link BASE V8 官方说明核对；使用来源明确、受工具支持的探针与对应软件组合，才能使长期测试结果可重复。对于本地疑似 clone，优先用已知正常的同类探针复测，而不是把软件降级、改固件或周期重连当作根因修复。

本地 profile 中的 Server ready 超时为 15000 ms，普通 MI 请求超时默认为 8000 ms，GDB remote-timeout 为 5 秒；它们分别限制启动或单次等待，不是 53 秒会话寿命。`-singlerun` 表示 GDB 客户端断开后结束 Server，也不是按秒定时关闭。修改这些值之前，应先由日志确认究竟是哪一个等待超时。

历史中 SWD 降到 1000 kHz、关闭 Flash 断点并使用硬件断点，仍曾出现异常；刷新缓存也没有恢复正确读取。另一路 CMSIS-DAP/OpenOCD 的 90 秒运行通过是该组合的有效记录，但它同时更换了 Server、探针和连接参数，**不能反向证明原故障必定由 clone 检测导致**。

可复核的诊断顺序

1. **保存故障现场**。记录 Server/GDB 进程与端口是否仍在、首次错误时间、USB 是否重新枚举及目标电压/复位情况；保留原始 MI 与 Server 输出。不要先重连再采集唯一证据。
2. **确认是哪一层失效**。在已暂停的目标上对照 monitor 寄存器和稳定的系统地址；既看 MI 类别，也看 Failed、全零或重复异常读数。无法确认停核/访问状态时停止修改寄存器和内存，不把异常 PC 当作真实程序走飞位置。
3. **绕过 TUI 做持续时间对照**。使用同一 ELF/固件、Server/DLL、探针、线缆与 SWD 速率，分别做静置和操作循环；记录首次失效时间而非单纯记录操作次数。
4. **按单一变量替换**。首先用已知正常且受支持的探针保留其他条件对照；再分别排查 USB 线/端口、目标供电/复位、工具组合。驱动兼容性需保证一致可用，避免把“换探针后打不开 USB”混入结果。
5. **扩大验收窗口**。建议每个候选组合至少多次覆盖 30/60/120 秒的静置与持续操作，并继续到数分钟；这是排查建议，不是本次新增实测。检查寄存器、停核位置、内存和错误日志，不能只确认连接图标仍亮。

仓库已有 `scripts/test-step-isolation.cjs`。它硬编码了当时 `user_task.c:40` 断点及若干期望 PC，是**特定固件测试夹具**，不应直接拿最新 ELF 无条件运行。准备相符镜像，或先根据实际符号/反汇编调整测试预期后，才能使用以下命令模式；这些命令会复位和运行目标，本次文档修订未执行它们。

```
# ELF must match this test fixture's breakpoint and expected PCs.
node scripts/test-step-isolation.cjs <matching-ELF> 500 1
node scripts/test-step-isolation.cjs <matching-ELF> 10 1 monitor 30000
node scripts/test-step-isolation.cjs <matching-ELF> 10 1 monitor 60000
```

原始证据位置

- 四组长会话：`artifacts/step-isolation-2026-09-16T13-54-58-531Z/`，各组含 `trace.log`、`timeline.json`、`result.json`，根目录有汇总。
- 静置 60 秒：`artifacts/step-isolation-2026-09-16T13-59-37-549Z/`；重复：`artifacts/step-isolation-2026-09-16T14-01-45-671Z/`。
- 静置 30 秒：`artifacts/step-isolation-2026-09-16T14-01-10-862Z/`；1200 次短会话：`artifacts/step-isolation-2026-09-16T13-50-39-156Z/`。

初版手册已核对以上本地目录、长会话 trace 时间戳与三组静置 result；本轮保留该历史结论。历史 ELF SHA-256 为 `076B3D1D9CC4DC04BBBBB6269E816673BA78CBF9730E54BF8B9A4718D842FCDD`，不要与后续 CMSIS-DAP 报告的构建混用。资料汇总见 `TESTING.md` 的“追加隔离实验”。

22 构建、测试、打包与证据矩阵

22.1 开发环境和测试分层

工程使用 Rust 2024 edition；当前 Ratatui 0.30.2 依赖要求 Rust 1.88.0。主要直接依赖还有 Crossterm 0.29、Serde/serde_json 1、toml 0.9、unicode-width 0.2、svd-parser 0.14.10（expand）。具体解析版本固定在 Cargo.lock。release 采用 opt-level=s、thin LTO、单 codegen unit、strip。

```
./scripts/build.ps1 -Test
./scripts/build.ps1 -Release

# With the repository Rust environment on PATH:
cargo test --locked -- --test-threads=1
cargo clippy --locked --all-targets -- -D warnings
```

build.ps1 可使用仓库.dev/cargo 和.dev/rustup；GNU 链接器可通过 DEBUGTUI_GCC_DIR 加入 PATH。Node 仅用于开发测试脚本、npm 打包等，不在调试运行链路。`.dev`、target、bin、artifacts 均为忽略目录，Git 克隆不保证包含这些本地工具和证据。

验证层	主要脚本/覆盖
Rust 单元/集成	各模块内联 tests、tests/project_tasks.rs；解析、配置、状态/组控制、UI 单元格布局和实际 shell 生命周期。
模拟 GDB	tests/mock-gdb.cjs；test-gdb-environment、test-pause、test-lifecycle 等覆盖异常时序、架构边界和进程清理。
真实本机 GDB	test-native-gdb、completion、watch-tree、breakpoints、logs、svd；多核脚本启动多个真实 GDB。
内存路由	test-memory-access-gdb；本机测试使用真实 GDB + 模拟 TCL，可测试协议但不是 AP 硬件验收。
实板	test-hardware、test-pause-hardware、test-step-hardware、带 –hardware 的内存/断点/SVD 测试，要求明确 ELF/profile。
打包/发布	test-npm、test-release、release-assets、package；独立 tools/package 校验工具依赖哈希。

22.2 近期新增测试入口与验收方法

统一入口 `scripts/test-functional.cjs` 当前默认定义 20 个套件，显式硬件模式另加 3 个。可以用 –only 选择本轮要验证的功能；未执行、跳过、失败与通过分别记录，不能把选择性运行当作所有功能通过。`tests/functional-coverage.json` 维护功能到源码/套件的对应，`tests/README.md` 给出参数和夹具前提。

```
# Read-only host/isolated fixture acceptance; no --hardware.
node ./scripts/test-functional.cjs --binary ./target/release/debugtui.exe --only cli,devices-
    tui,devices-gdb,source-remap
# Real native GDB / GCC are required for devices-gdb and source-remap.
# Supply --gdb and --cc if they are not discoverable.
```

新增驱动（scripts 下）	覆盖与运行边界
`test-source-remap.ps1`、`test-source-remap-gdb.cjs`	真实终端目录选择与 GDB 映射；路径风格、取消/超时/扫描失败、保存和多核继承。工程 ELF 模式仅扫描，不连接板卡。
`test-devices.ps1`、`test-devices-gdb.cjs`	TUI 新增/选择芯片、核心子集、目录持久化；用多个真实本机 GDB 检查配置派生，不能当成所有芯片实板结果。
`test-devices-tha6206.cjs`	THA6206 的 core0/core1/双核连接和控制；要求配套 MCAL 固件，属于有板卡动作的专项。
`test-bao-setup.ps1`	Bao 隔离配置的无参数启动、核心选择与保存；不按 Start，不操作硬件。
`test-bao-tasks.cjs`	构建/下载脚本失败路径和退出码的隔离夹具；不烧录真实板卡。

新增驱动（scripts 下）	覆盖与运行边界
`test-bao-devices.cjs`、`test-bao-workflow.cjs`	前者带固件匹配门槛的连接验证；后者实际 Build/Download/回读/调试，只能在允许切换固件的板卡上执行。

22.3 源码测试与历史证据复核

以下按执行阶段记录结果，不把不同时点的测试数量相加。初版的 137 项记录属于 v0.8.3；9 月 21 日的 THA6206/Bao 文档补充曾做配置离线加载、Guest ELF 解析和产物哈希核对。9 月 23 日补充了已安装 v0.8.6 的 MCAL 回归；9 月 27–29 日新增功能验收、源码重映射、芯片选择及 Bao 构建下载证据。下面保留各自原日期；2026-10-03 的界面、分发验证不是重新跑过这些硬件测试。最新 1.3.0 的独立结果见第13.11节，不覆盖此前失败或未执行项。

阶段	已记录的结果及边界
v0.8.3 初版源码	137 项单元测试、1 项 CLI 集成测试通过，0 失败；保留为历史基线。
9 月 22 日源码选区	147 项单元测试、1 项 CLI 集成测试通过，2 项需显式环境的测试默认忽略；严格 Clippy、release 构建通过。Windows 原生剪贴板读回另测通过。
9 月 22 日 Setup 整合	`cargo test --all-targets`：154 项单元测试、1 项 CLI 集成测试通过，0 失败，2 项忽略；严格 Clippy 与 release 构建通过。包含项目切换、示例、相对路径、直接退出及布局回归。
9 月 22 日 THA6206	三种入口的源码操作/Watch 用例通过；附加 Reset/Run 与代码段比较有失败，保留为历史记录。
9 月 23 日 v0.8.6	168 项单元测试、1 项 CLI 集成测试通过，2 项外部环境测试默认忽略；MCAL 实板 16 个功能组通过，包含 21 个退出/重连场景，残留复位和深层栈问题见第12.8节。

近期阶段	已记录结果及未覆盖范围
9 月 27 日验收	对已安装 0.8.6 的当时 20 个套件通过；44 项新增用例通过、真正烧录 HW-16 跳过。与后来增加的套件数量不同。
9 月 28 日映射	0.8.7 的 174 单元 + 1 集成通过、2 忽略；0.8.8 为 175 + 1、2 忽略。验证四种路径风格、离线 GDB 与 THA/Bao ELF 存在性预览；非新的板上失败构建复现。
9 月 29 日芯片选择	0.9.0 为 181 单元 + 1 集成通过、2 忽略；10 种真实本机 GDB 组合通过。THA6206 实板 core0/core1/双核三项通过，允许 reset，无 Build/Download；未提供其他三款芯片实板结果。
9 月 29 日 Bao	smoke 单 GDB 与双 GDB 流程各 5 项通过，外部任务夹具 6 项通过。详细边界和产物见第13.14节。

近期阶段	已记录结果及未覆盖范围
9 月 29 日界面	0.9.1 为 181 单元 + 1 集成通过、2 忽略；严格 Clippy、Release、Bao Setup 选择/保存通过。EXE 已本地安装，芯片目录哈希未改变。
10 月 3 日 v0.9.3	185 单元 + 1 集成通过、2 忽略；CLI 12/12、隔离 npm 生命周期 7/7 通过。全局 EXE 与构建哈希一致，15 份工程及芯片目录文件不变；Bao 隔离项目 core0/core1/双核选择与保存通过，无实板操作。
10 月 3 日终端	键盘、搜索、配置页、缩放、退出 8/8 通过；独立事件探针未收到该宿主注入的 SGR 鼠标记录，Files 原生用例改用 Tab。此结果不包含真实终端鼠标的端到端验收；鼠标命中保留 App 测试覆盖。
10 月 28 日 1.3.0	全局安装版 THA6206 Bao/VM 实板与终端 74/74；Rust 606 库测试 + 7 集成通过、3 忽略，软件 10、分发 14、本机 GDB 21 组分别通过。EXE/固件摘要及限制见第13.11节与 `artifacts/validation-1.3.0.json`。
保留的失败	旧 ConPTY 通用终端 7/8 及 10 月 3 日首次 TERM-03 鼠标注入失败报告仍保留，不改记为通过；应用内部鼠标命中测试不能替代此端到端结果。

实现依据：`TESTING.md` 的对应日期记录；`artifacts/devices-tha6206-1790653885250-7b45250a/report.json`；`artifacts/ui-controls-20260929/install.json`；`artifacts/terminal-20260929-203508-1db44c3a/report.json`。

源码选区阶段的详细记录在 `artifacts/source-selection-20260922/result.json` 及各入口事件文件中；其中 EXE 哈希只代表该阶段的构建，不代表之后加入 Setup 修改的二进制。`artifacts/setup-ux-20260922/` 的 12 份文本快照覆盖空配置、示例列表、应用示例和字段帮助在 45×12、80×24、120×36 下的布局，属于 Ratatui 单元格输出，不是实板连接记录。

硬件测试可通过环境变量 `DEBUGTUI_TEST_PROFILE` 选择工具配置，未设置时沿用 OpenOCD 默认 profile。此覆盖入口适用于 `test-breakpoints-gdb.cjs` 和 `test-memory-access-gdb.cjs`。源码选区实板用例在 `src/ui/source_text/hardware_tests.rs` 中显式忽略，需专门配置项目及日志环境后调用；普通测试通过不等于这类用例已执行。THA6206/Bao 历史结果见第12、13节，J-Link 长会话故障见第21.3节。

本地证据（artifacts 下）	历史结果	本次核对方式
`dap-full-1789916855036/verification.json`	122 请求，Build、DAP 实际下载/可加载段校验、寄存器、外设、栈、Watch、90 s 运行后恢复。	读取 JSON、project.toml 和总报告。
`breakpoints-stm32-1789916988826/verification.json`	75 请求，代码/硬件/临时/三类数据断点、启停和恢复。	读取 JSON，与脚本语义核对。
`memory-stm32-1789917004955/verification.json`	70 请求；20 次 100 ms 运行读，零新增 GDB 命令；之后暂停/单步可用。	读取 JSON、profile 引用及测试脚本。

本地证据（artifacts 下）	历史结果	本次核对方式
`singlecore-0.8.3-1789917013498/verification.json`	93 请求直接单核；41 请求单核 + 旧 Live Watch，24 次实时事件。	读取 JSON，与线程路径核对。
`dap-ui-20260920-2307/REPORT.md`	汇总 401 个实机请求，11 个离线补全请求；记录实际 TUI 操作和下载进度缺陷。	阅读报告和原始 GDB/OpenOCD 日志。
`dap-ui-20260920-2307/raw-openocd.stderr.log`	CMSIS-DAP、1 MHz、Cortex-M4、AP0、6/4 断点资源、1024 KiB Flash。	直接核对设备识别文本。
`memory-stm32-1789825648786/verification.json`	较早 OpenOCD + J-Link/F429 内存通道回归。	结合 TESTING.md 历史范围。

401=122+75+70+93+41，是请求数而非独立功能数。历史实板报告的源码 SHA 为 `8c74f72e4139d62cc3b3034266d22bbfe1a2a14e`，与当前文档基线不同；本次以源码确认相关实现，不能声称历史二进制就是当前重新构建的 EXE。

22.4 分发和许可证

package.json 的 files 包含 EXE、profiles、docs、内置 tools、说明与许可证，开发脚本及构建缓存不进入 npm 包。TUI 以 Apache-2.0 分发；GDB、OpenOCD、SEGGER 工具及 SVD 遵从各自许可证，不能用应用许可证替代第三方条款。

`scripts/package.ps1` 核对 npm/EXE 版本并拒绝开发/工具依赖泄入 TUI 包；`scripts/package-tools.ps1` 检查 GDB/OpenOCD 主程序存在，按实际 `bin/` 内容与配置目录生成 `debugtui-tools-arm-win-x64.zip`；`openocd-adapter/` 源码不进入该包。安装器同样按实际工具目录复制，无需依赖清单。

实现依据：`Cargo.toml`、`Cargo.lock`、`package.json`；`scripts/`；`scripts/package-tools.ps1`；`NOTICE`、`THIRD_PARTY_NOTICES.md`、`licenses/`。

23 已知问题、实现限制与维护建议

环境侧另有尚未关闭的 J-Link 长会话故障，见第21.3节。它与下述已确认的 UI/解析实现问题分开管理；当前手册未把“疑似 clone”作为已证实的软件定时限制，也没有以其他探针的通过记录关闭该故障。以下旧专题保留 0.9.3 手册当时的缺陷、设计限制和历史资料偏差，未在本次多 ELF 文档增补中逐项重新验收。1.3.0 多 ELF 的当前边界集中于第13.10节，不能用旧专题推定后来功能仍未实现；本文只更新手册，不改动产品行为。

23.1 已确认的下载进度解析缺陷

本地原始 GDB 报文含匿名 tuple：

```
5+download,{section=".isr_vector",section-size="428",total-size="1311248"}
5+download,{section=".isr_vector",section-sent="428",section-size="428",total-sent="428",total-
    size="1311248"}
5^done,address="0x0800604c",load-size="27396",transfer-rate="120952",write-rate="3424"
```

当前 `mi::parse` 在 class 后遇逗号直接调用 result，期待 identifier=value，因此拒绝 `{...}` 并记录 expected identifier。Session 虽有 download 进度处理分支，但这些状态记录无法抵达它。后续 token 匹配的 `^done` 仍可成功，历史所有加载段 matched。修复应增加这类实际状态记录的兼容解析和回归测试，同时保持拒绝截断报文的边界。

23.2 当前代码与旧说明的差异

项目	本手册采用的当前结论
SVD 字节序	Device::parse 对未知/缺失 endian 使用 unwrap_or(true)；默认小端。不能沿用“未知只展示”的旧说法。
SVD 是否实现	已实现解析和 Peripherals，README 部分旧段落“尚未实现”已过时。
环境表数量	八类普通 section 加 backend/backends/core_targets 元数据；新字段由芯片选择层处理。
tools 自动发现	Setup 有工程目录发现逻辑；“完全不会自动寻找 tools”只适合旧版本描述。
单核 Server 日志	Session 将全部 stderr 标为 server-error；Coordinator 的新分帧器区分普通 stderr/真正错误。不能仅凭单核通道名断言发生错误。
下载栏空值	tasks.download 空仍可能回退 actions.download；应查看确认框/实际动作。
多核硬件范围	现已复核 THA6206 多核原始记录及 Bao 双 VM 记录；保留历史证据、本次离线检查与待验证配置的区别，见第12、13节。

23.3 实现资源和可观测性限制

- 停止刷新最多取栈帧 0–31、128 个局部变量；Memory 每请求最多 4096 字节。Watch、源码、MI 和 SVD 另有限制，见前文。
- refresh 的若干栈/locals/寄存器查询使用容错分支，个别子查询失败可能保留旧列表；错误日志和停止代次比单纯“面板有值”更可靠。
- 可见项轮询与旧 live_watch 的调度不同；旧采样队列容量 64，使用 try_send，高负载下可能丢采样，不能用作无损数据记录仪。
- 旧 live_watch 的配置只要求正间隔，实际循环 clamp 到 10–60000 ms；逐项 UI 则要求 50–60000 ms，二者不能混用。
- 软件组控制不是硬件同步；运行时总线读取不是跨多个值的一致事务；工程偏好写入不是跨进程原子提交；用户芯片目录另有跨进程锁和同目录临时文件发布。

- 通用终端 Files 用例 TERM-03 改用 Tab 后通过；该 ConPTY 宿主未交付注入的 SGR 鼠标事件，旧失败报告保留。真实终端鼠标仍未完成端到端验收，应用内部鼠标命中测试不能替代该结果。
- 当前没有专门的变量/寄存器写入 UI；原始 GDB 可写，用户应理解实际目标副作用。默认面板读取不等于整个调试连接过程零修改。
- Source 选区改变 Ctrl+C 的优先级：聚焦源码且有选区时复制，确定暂停使用 F6。加入 Watch 仍受当前核、当前帧和运行时读取通道限制，不自动启用实时采样。
- THA6206 修正后的脚本已有 9 月 23 日及 29 日功能回归，不能再将 9 月 22 日旧失败描述为所有当前流程均失败；CSE 新旧值的专项硬件 A/B 和偶发复位/深栈限制仍按第12.8节单列。

优先维护方向是修复真实下载状态解析、统一两条服务日志路径、对缺省 SVD 字节序给出明确策略或提示，并同步过期 README。若扩展到异构核、线程/RTOS 视图或网络 API，应同时定义数据归属、状态代次、清理及错误契约，再扩展 UI。

实现依据：`src/mi.rs`: parse；`src/svd.rs`: Device::parse；`src/session.rs`: refresh/start_server；`src/live_watch.rs`: spawn；`artifacts/dap-ui-20260920-2307/REPORT.md`。

A 全工程源文件与职责索引

2026-10-03 核对的 0.9.3 发布前工作树中，`src` 共 48 个 Rust 文件（含模块测试），下表覆盖全部文件。近期新增模块包括芯片目录及选择、离线源码目录扫描和搜索相关模块。

文件（相对工程根）	职责与关键入口
`src/main.rs`	main/run、CLI 路由、run_headless，串行 stdin/script 请求与事件输出。
`src/lib.rs`	库模块导出，供二进制和测试共享。
`src/cli.rs`	Options::parse/document，参数解析、CLI 覆盖、路径绝对化。
`src/launch.rs`	Document、Setup、相对路径、字段帮助、文件浏览器、保存冲突/草稿、直接退出和启动动作。
`src/launch/choices.rs`	调试项目 TOML 筛选、Projects 列表和可选 Examples；不保存或启动进程。
`src/launch/devices.rs`	Chip 列表、Add chip 表单、核心复选、Legacy 和草稿选择。
`src/launch/remap.rs`	ELF 目录选择、父子层级、匹配预览、重新扫描和取消。
`src/launch/images.rs`	1.3.0 的 Symbol images 草稿编辑器；不连接目标，高级声明保存时保留。
`src/clipboard.rs`	Windows 原生 Unicode 剪贴板、失败清理与其他平台工具回退。
`src/config.rs`	全配置模型、默认值、递归合并、校验、路径解析、偏好写锁。
`src/devices.rs`	用户目录初始化/追加锁、能力校验、backend 合并、模板展开与所选核派生。
`src/source.rs`	跨主机路径拼写、目录边界、GDB 映射别名、离线 ELF 源文件扫描与取消。
`src/search.rs`	文件模糊匹配评分、大小写处理及 GDB 符号正则构造。
`src/session.rs`	Engine/Gdb/Snapshot、命令执行、异步 record、停止刷新、任务/连接/清理。
`src/session/breakpoints.rs`	MI 断点编解码、创建/修改/恢复、失败处理及立即持久化。
`src/session/watch.rs`	GDB variable object 树、分页、展开映射和变化标记。
`src/session/memory.rs`	watch_resolve、命名内存读、宽度/对齐/端序校验、TCP 缓存。
`src/session/symbols.rs`	停止态 GDB 符号索引请求、来源/位置与查询范围。
`src/images.rs`、`src/session/images.rs`	多 ELF 定义、可执行地址/域识别、内容版本、连接前装载与失效检查。
`src/session/image_navigation.rs`	按镜像限定文件/符号/ELF 反汇编，保留来源和独立源码映射。

文件（相对工程根）	职责与关键入口
src/session/image_values.rs	单 ELF 类型与实板内存通道结合，隔离同名全局和混合表达式。
src/session/guest/、src/ui/guest/	真实 Guest 保存 ABI、只读 RSP、Guest 帧视图及 HVC 返回/取消的资源管理。
src/coordinator.rs	多 Session 工作区、共享 Server、Batch/Step 队列、核选择、派生配置。
src/coordinator/control.rs	Scope、顺序 Run、共享 reset、断点联停和单步隔离。
src/coordinator/breakpoints.rs	group 注解、成员路由、真实编号映射、undo 队列。
src/coordinator/server_log.rs	共享服务 UTF-8 分帧、无换行 ready、stderr 分类。
src/mi.rs	Record/Value、字符串 quote/parse、嵌套与转义、语法边界测试。
src/process.rs	Windows Job Object 所有权和子进程树回收；其他平台回退。
src/logging.rs	Stamp 捕获时间、Trace 排他创建、逐行绝对时间与单调耗时。
src/live_watch.rs	TCL 连接/RPC/回显标记，ELF 符号解析和旧式采样器。
src/svd.rs	SVD 展开为紧凑模型，访问属性、位域、寄存器解码。
src/theme.rs	色板、边框、选择/焦点、核心关联色及遮罩样式。
src/ui.rs	App 主状态、命令分发、键鼠、事件循环、按需数据、退出/重建协调。
src/ui/render.rs	面板布局、工具栏、Help、源码选区/横向裁剪与固定断点栏、寄存器/内存绘制。
src/ui/breakpoints.rs	断点列表和编辑弹窗、开关、条件/忽略次数、批量操作。
src/ui/breakpoints/cores.rs	关联断点的核心复选 UI 和应用动作。
src/ui/console.rs	Console 历史视口、跟随、新消息计数和拖动。
src/ui/completion.rs	Console/Watch 补全调度、焦点和过期响应校验。
src/ui/cores.rs	核心按钮、颜色、状态和溢出导航。
src/ui/effects.rs	有界动画、值变化高亮、真实停止轨迹和绘制调度。
src/ui/formats.rs	整数进制转换、偏好键、格式弹窗和保存。
src/ui/highlight.rs	C 类源码着色、跨行注释状态缓存、可见行 Span 及选区所需的文本范围。
src/ui/monitor.rs	逐项刷新设置、可见项公平调度、LIVE 缓存、错误退避。
src/ui/peripherals.rs	SVD 树、展开/滚动、寄存器读取及位域显示。
src/ui/source_tabs.rs	文件路径去重、独立浏览位置、标签溢出/搜索列表、源码切换时清理选区。

文件（相对工程根）	职责与关键入口
src/ui/search.rs	Files/Symbols 筛选输入、键鼠导航、延迟查询与结果选择。
src/ui/search/tests.rs	搜索输入、焦点、过期结果及宽窄终端布局测试。
src/ui/source_text.rs	UTF-8 选区、显示坐标、键鼠移动、横向滚动、Copy 与当前核 Add to Watch。
src/ui/source_text/tests.rs	选区、中文/Tab、复制、Watch 路由、焦点/代次、横向视口和渲染回归。
src/ui/source_text/hardware_tests.rs	显式启用的真实 GDB/THA6206 源码操作测试，普通测试默认忽略。
src/ui/watch.rs	Watch 树扁平化、成员选择、展开/删除/分页、数值格式键。
src/ui/live_watch_tests.rs	实时事件在实际 Watch 渲染器中的回放与过期保护测试。
src/ui/visual_tests.rs	真实 Ratatui Buffer 布局/主题断言、按环境变量导出预览。

A.1 其他目录的工程角色

目录/文件	内容及分析边界
profiles/	默认芯片目录、npm 初始化脚本及 THA 项目/环境示例；不附带调试驱动或工具二进制。
tests/	mock-gdb、SVD/ELF fixture、ConPTY 终端宿主、覆盖矩阵及实际 shell CLI 集成测试。
scripts/	构建、许可证生成、打包、发布资产及模拟/本机/硬件验证。
tools/	可选调试环境、BAT、三个 F429 profile、板级 cfg、独立打包与依赖锁。
tools/bin/	第三方 GDB、OpenOCD 程序/依赖/脚本/许可证；未宣称分析其全部实现。
resources/svd/stm32/	STM32F429 原始 SVD、catalog、出处和许可。
licenses/	Rust 标准库和 SVD/相关依赖许可资料。
artifacts/	本地历史测试和发布产物，Git 忽略；不是自动化测试全部成功的推定依据。
.dev/、target/、bin/	本地工具缓存、Rust 输出及分发 EXE，不是自研模块。
根目录 Markdown	README/ARCHITECTURE/TESTING/PUBLISHING；需按版本和源码交叉核对。
docs/	LaTeX 手册、对应 PDF、重建脚本及专题文档；Bao 多 ELF 使用说明与分阶段验收分别在 bao-multi-elf.md、bao-multi-elf-development.md，原始证据在 artifacts。

B 命令行速查与术语

CLI 参数	行为
无参数	进入 Setup，不启动调试链路。
–project PATH	工程目录或配置文件；显式启动选项。
–setup	强制先检查启动表单。
–chip ID / –cores IDS	覆盖芯片及物理核心子集，例如 –chip tha6206 –cores 0,1；切换为格式 3，仍需匹配 profile/端口。
–init-profiles	初始化或校验用户芯片目录，输出其路径并退出，不连接板卡。
–gdb PATH / –gdb-arg ARG	指定 GDB/追加参数；gdb-arg 可重复。
–environment FILE	指定工具环境 TOML。
–tools-dir PATH	简写为 PATH/debug-env.toml。
–connect ENDPOINT	指向外部服务并禁用自有 service。
–target-mode MODE / –local	remote/extended-remote/local；local 禁用 service。
–elf FILE / –svd FILE	可选程序符号文件 / 外设描述。
–log-dir PATH	会话日志目录。
–headless / –stdio	使用 JSONL，任一均启用 headless。
–script FILE	串行执行 JSONL 并退出，隐含 headless。
–demo / –snapshot FILE	无 GDB 的 UI 演示 / 导出演示文本快照。
–version / -V；–help / -h	版本或帮助。未知选项报错。

术语	含义
MI / MI2	GDB Machine Interface 的行文本机器接口及本项目使用的解释器模式。
RSP	GDB Remote Serial Protocol，在本例通过 TCP 承载。
ELF / DWARF	可执行/链接格式及调试信息；符号/类型/源码位置主要由 GDB 处理。
SVD	CMSIS System View Description，外设寄存器元数据。
DAP / DP / AP	ARM Debug Access Port 体系、Debug Port、Access Port；不要与编辑器 Debug Adapter Protocol 混淆。
CoreSight / ADI	片上调试追踪架构 / ARM 调试访问接口架构；详见第9节。
MEM-AP	内存映射访问端口模型；CSW/TAR/DRW 用于描述和执行访问。
AHB-AP / APB-AP	面向 AHB / APB 的 AP；APB 外设也可经 AHB-AP 和总线桥访问。
AXI-AP / JTAG-AP	面向 AXI 总线 / 芯片内部 JTAG 结构的访问端口。

术语	含义
CMSIS-DAP / J-Link	标准探针命令协议与固件规范 / SEGGER 探针及其软件体系。
HID / bulk / WinUSB	USB 设备类 / USB 传输类型 / Windows USB 驱动机制；不是目标调试接口。
SWD / JTAG	探针到目标的调试传输；本地 F429 配置选择 SWD。
CTI / CTM / ECT	交叉触发接口 / 通道分配互连 / 嵌入式交叉触发设施统称。
FPB / DWT	指令断点等资源 / 数据观察点、计数及事件资源。
ETM / ITM	执行流追踪来源 / 插桩与事件来源；ETM-M4 为指令追踪。
ATB / TPIU	片上 Trace 数据总线 / 片外 Trace 输出接口组件。
ETB / TMC	片上 Trace 缓冲 / 可配置追踪缓冲与内存路由组件。
SWO / SWV / RTT	单线 Trace 输出 / 常见串行观察能力称呼 / 基于目标 RAM 缓冲的数据传递。
generation	用于区分停止上下文和缓存有效性的代次；不是 CPU 周期或采样计数。
Scope / active core	运行控制范围 / 当前检查数据的核心，二者独立。
LIVE / cached	来自实时采样的有效值 / 保留的历史值；标记不构成跨变量同步保证。
watch / watchpoint	Watch 表达式观察 / 硬件或 GDB 数据断点，删除前者不删除后者。

C 外部参考与文档复现

主要架构资料于 2026-09-21 核对；2026-09-22 补充 Cortex-R52 EDRCR.CSE 与本地复位脚本核对。实现细节仍以本地源码和本地工具版本为准；网页当前版本可能新于分发工具。

1. GDB/MI Interface：MI 的用途和接口边界。
2. GDB/MI Output Syntax：结果、异步、流和 token 语义。
3. GDB Remote Protocol Overview：RSP 包框架。
4. GDB Connecting to a Remote Target：remote/extended-remote 连接。
5. OpenOCD Tcl Scripting API：RPC 和 0x1a 帧边界。
6. OpenOCD CPU Configuration：target、mem_ap、显式内存访问。
7. SEGGER J-Link GDB Server：服务选项与 remote monitor 命令。
8. ST STM32F429IG：具体器件资料入口，含数据手册和 RM0090。
9. Arm CMSIS-DAP Overview 与 CMSIS-DAP USB 配置：主机探针接口、HID/bulk 和可选 SWO 端点。
10. OpenOCD Debug Adapter Configuration：适配器驱动、后端和传输选择；以本地版本实际能力为准。
11. Arm IHI 0029D，CoreSight Architecture v2.0：组件分类、可见接口与拓扑。
12. Arm IHI 0031G，ADIv5.0–ADIv5.2：DP、MEM-AP、寄存器和访问语义。

13. Arm CoreSight basics Part 1、Part 2、Part 3：交叉触发、访问路径与 Trace 采集解释。
14. Arm Cortex-M4 TRM 与 ETM-M4 TRM：核调试、DWT/ITM、指令追踪和输出边界。
15. ST RM0090，第 38 章 Debug support：F4 的 SWJ 引脚、AHB-AP、FPB/DWT/ITM/ETM/TPIU。引脚仍需结合具体封装和板卡核对。
16. SEGGER J-Link BASE V8 与 J-Link/J-Trace User Guide：硬件代际与工具边界；本手册不据此声称存在固定 53 秒 clone 限制。

C.1 从 LaTeX 重建 PDF

本文是单一、可独立编译的 LaTeX 源文件，流程图全部使用 TikZ 矢量绘制，不依赖本地 artifacts 截图。需要 XeLaTeX、ctex、fontspec、TikZ、listings 等包，字体为 Windows 的 SimSun、SimHei、Microsoft YaHei、Times New Roman、Arial、Courier New。等宽字体选用可区分 ASCII 连字符的 Courier New，避免复制命令后参数中的减号变成 Unicode 连字符。其他平台需替换字体设置并核对命令复制结果。

```
# From repository root:
./docs/build-usermanual.ps1

# Equivalent manual build, from docs:
xelatex -interaction=nonstopmode -halt-on-error usermanual.tex
xelatex -interaction=nonstopmode -halt-on-error usermanual.tex
xelatex -interaction=nonstopmode -halt-on-error usermanual.tex
```

三轮编译稳定目录、页码和交叉引用。随附脚本在 docs 下创建独立临时目录，成功后仅复制 PDF 并清理该次中间文件；失败时保留临时日志以便定位。没有系统 XeLaTeX 时，也可使用支持 XeTeX 的 Tectonic：例如 `tectonic --keep-logs --reruns 2 --outdir <临时目录> docs/usermanual.tex`，首次运行需要下载其 TeX 资源，Windows 字体要求相同。2026-10-28 增补使用隔离缓存的 Tectonic 0.17.0 构建；编译器来源和摘要保留在本轮文档验证记录，未将工具加入 DebugTUI 运行依赖。需要图形化质量检查时，将最终 PDF 全页渲染为 PNG，检查裁切、遮挡、缺字、表格续页、箭头与标注，然后删除本次临时渲染产物。

本文中的实板日志不因 PDF 编译而重新生成；复现硬件行为必须另行准备相同芯片、探针、工具、固件和 profile。文档构建结果不替代源码测试和实板验证。

D 流水线产物的本地源码映射

D.1 适用场景与现有能力

本地提交代码后，流水线编译测试失败；下载该次构建的 HEX 和 ELF，在本地调试时却因 ELF 中的绝对源码路径指向构建机而无法打开 C 文件。DebugTUI 0.8.3 已支持通过工程 TOML 中的 `[[source_map]]` 解决此问题，无需修改 DebugTUI 源码或为了改路径重新编译 ELF。

配置同时用于源码窗口查找和 GDB 的 `set substitute-path`，并由各核继承。映射只改变文件查找位置，不修改 ELF、HEX、指令地址或目标内存。v0.8.7 增加 Setup 重映射开关、ELF 目录选择和本地文件预览；流水线产物下载及源码版本选择仍由用户完成。配置步骤与示例统一在本附录维护。

D.2 推荐方式：在 Setup 选择目录层级

1. Program / ELF 选择服务器构建产物；Source root 选择相同版本的本地源码根。
2. 将 SVD file 后面的 Source remap 改为 Yes。为 No 时，下一行 ELF path prefix 灰显且不可选择或修改。DebugTUI 在后台使用已配置的 GDB 读取 ELF 源文件表，忽略 profile 中的启动参数、初始化与板级动作，关闭自动加载脚本，不启动 OpenOCD、不连接或复位板卡。
3. 在 ELF directories 面板选择与 Source root 对应的旧目录。上下键或鼠标选择；左右键选择父/子层级。列表给出本地存在数 / 覆盖文件数，下方预览具体映射。目录层级选错会保留错误的后缀，通常表现为匹配数为零；文件存在也不能证明源码版本一致。
4. Enter 或 Apply 确认，Ctrl+S 保存，再 Start debugging 使用新映射。ELF path prefix 可重新打开选择；R 重新扫描，Esc 取消并保留原选择。缺失 ELF、无源码记录或扫描失败时显示原因，取消后修正配置。

选择写入工程 TOML 的以下表，目标始终跟随当前 Source root；无需另外填写 to。相对 Source root 仍按 TOML 所在目录解释。

```
[source_remap]
enabled = true
from = "/builds/team/THA6XXX_MC_AS440"
```

目录匹配支持正反斜杠与混用、Windows 盘符、UNC 路径和空格。若 ELF 保留多种分隔符拼写，扫描自动将它们保存为 aliases，GDB 与 TUI 都应用对应规则。部分 GDB 仍会解析旧网络路径，不可达的 UNC 主机可能使扫描超时；此时可以用下文的手工 source_map 配置绕过扫描。关闭 Source remap 时保留但停用该选择和已有 source_map；旧配置未设置开关时仍沿用原规则。额外手工规则先应用，Setup 选择覆盖手工规则中相同的前缀。自行写在 gdb.init 或 Console 中的 substitute-path 命令不受此开关管理，建议迁移至工程映射配置。前缀的大小写须与 ELF 记录一致；不会自动折叠 Windows 盘符大小写，也不按同名文件猜测路径。具体扫描边界及映射顺序见第15.2节。

配置项	含义与相对路径基准
`program.elf`	GDB 加载的 ELF；相对工程 TOML 所在目录。
`program.source_root`	本地源码查找基础目录、Build/Download 工作目录；相对 TOML。不能代替旧绝对路径映射。
`source_remap.from`	Setup 选择的 ELF 目录前缀；目标为 Source root，enabled 为开关。
`source_map.from`	ELF 中实际记录的旧目录前缀；保留旧路径，不转为本地目录。
`source_map.to`	对应的本地源码目录；相对 TOML，而非 source_root。
`live_watch.elf`	已配置旧式 Live Watch 时使用的符号文件；应同步指向该次流水线 ELF。

D.3 步骤一：准备同次构建的镜像、符号和源码

1. 下载失败构建的 HEX 和包含调试信息的配套 ELF；HEX 不提供 C 行号和类型信息，被剥离的调试信息也不能靠路径映射恢复。
2. 记录流水线实际编译的 Git SHA，包括可能的合并测试提交。准备相同的子模块、依赖、生成源码、编译器版本和构建配置；同一提交不保证未归档生成代码相同。
3. 在独立 checkout/worktree 中准备对应源码，保留正在修改的工作区；先复现原构建，不先点击 Build 覆盖下载产物。

以下命令在固件 Git 仓库执行，替换实际提交号；不在 DebugTUI 仓库执行。worktree 不自动复制当前目录的未跟踪工具、配置和生成文件，需要另行准备。

```
$ciCommit = "REPLACE_WITH_CI_COMMIT_SHA"
git worktree add --detach ../THA6206-ci-debug $ciCommit
```

D.4 步骤二：查出 ELF 中的旧路径

在已有会话的 Console 输入原始 GDB 命令 `info sources`，查看记录的源码路径；`info source` 显示当前源码及编译目录，没有当前源码上下文时可能无结果。也可在固件工程根目录离线读取，不连接板卡：

```
& ".\.vscode\gdb\arm-none-eabi-gdb.exe" -q -nx -batch `
  -ex "info sources" ".\ci-artifacts\THA6206_Demo_Prj.elf"
```

相对文件名需结合编译目录判断；必要时检查 DWARF 的 `DW_AT_comp_dir`、`DW_AT_name` 和行号表。不要把 ELF 在本地的存放目录当作旧源码根。

例如旧源码根为 `/builds/team/THA6XXX_MC_AS440`，本地根为 `D:/LiBo/Files/DATA/FreeRTOS/vscodegdb/THA6XXX_MC_AS440`；两边后续均为 `DemoWorkspace/CodeProject/.../Demo_App.c`，则替换根目录即可，保留其下目录结构。本附录中的流水线路径都是示例，不代表已经拿到某次失败构建。

D.5 步骤三：修改工程 TOML

将可用的 `debug-core0.toml`、`debug-core1.toml` 或 `debug-multi.toml` 复制为同目录的 `debug-ci.toml`，保留 GDB、服务、端口、SVD、多核等设置。下面是合并片段，不是完整板级配置；已有表应修改原表，不能重复声明。

```
[program]
elf = "./ci-artifacts/THA6206_Demo_Prj.elf"
source_root = "."
# Keep the existing svd field here, if configured.

[[source_map]]
from = "/builds/team/THA6XXX_MC_AS440"
to = "."
```

本例 TOML 位于本地源码根。`to = "."` 相对 TOML 解析，与终端启动目录无关；若 TOML 位于 `debug/` 子目录，源码根在上一层，则可用 `to = ".."`，并分别调整 ELF 与 source_root。Windows 构建机可填写 `from = "C:/agent/_work/42/s/firmware"`。采用实际路径拼写和大小写，推荐正斜杠。

已启用旧式 Live Watch 时，将原有 `[live_watch]` 表的 `elf` 同步改为 `./ci-artifacts/THA6206_Demo_Prj.elf`；保留 bus_target、tcl_endpoint、采样间隔等其他字段。仅更改 Program/ELF 不会自动更新这个独立字段。

多个源码根可配置多条规则，嵌套前缀的具体规则放在前面：

```
[[source_map]]
from = "/builds/team/firmware/generated"
to = "./ci-artifacts/generated"

[[source_map]]
from = "/builds/team/firmware"
to = "."

[[source_map]]
from = "/opt/vendor/sdk"
to = "../vendor-sdk"
```

匹配按目录前缀与边界进行，不支持任意子串替换、通配符或按同名文件递归猜测。GDB 使用第一个匹配规则；DebugTUI 还检查结果文件是否存在，缺失时可能继续其他查找。应保证目标文件存在，不用重叠规则掩盖缺失文件，以免两层查找指向不同源码。

D.6 步骤四：烧录配套 HEX，重新启动并验收

修改 program.elf 不会自动改写外部下载脚本的 HEX 路径。当前 THA6206 工程的 `.vscode/Load.bat` 固定使用 `DemoWorkspace/CodeProject/Debug/<chip>_Demo_Prj.hex`。流水线镜像若存入 ci-artifacts，应先让脚本明确接受并使用该文件，或按项目既定流程烧录该文件。此调整属于固件工程配置，不是 DebugTUI 源码修改。

保存 TOML 后，结束旧会话并重新启动，确保从磁盘重新读取配置；旧会话内 Reconnect 不保证重读外部修改。烧录时按项目流程释放调试器对探针的占用。

```
# From the firmware checkout:
debugtui --project ./debug-ci.toml

# In the paused session's GDB Console:
show substitute-path
info source
list DemoApp_MainCore1
```

示例函数需替换为 ELF 中实际存在并有调试信息的函数。在 Source 中设置断点，运行命中后核对文件、函数、行号和栈，再单步。多核逐核检查规则和源码。Console 临时设置 substitute-path 只改变当前核 GDB，不替代 TOML 中供界面与各核共用的持久映射。

D.7 故障排查与验收边界

现象	检查与处理
Source not found	从当前帧或 info sources 获取实际旧路径；核对前缀、后缀和目标文件存在性，重启加载配置。
GDB 能 list，窗口找不到	检查是否只在 Console 临时设置；改为共享 source_map，并核对重叠规则。
文件打开但行号不符	核对提交、生成代码、依赖、ELF 与烧录镜像。映射不会验证源码内容。
某一核仍报旧路径	检查是否仅设置了当前核 GDB，以及是否启动了正确多核配置；逐核查看规则。
Watch 地址或值异常	检查 live_watch.elf 与 program.elf；另查内存通道、缓存一致性。
没有 C 行号或符号	获取含调试信息的配套 ELF；分离调试信息时取得所需文件。映射无法补回已剥离的信息。
跳行、变量 optimized out	可能是原构建优化行为；调试友好重编译版本是另一份产物，不能代替原故障复现。
SDK/生成文件缺失	取得同版本文件并映射其独立根目录；同名不代表相同源码。

验收至少包括正确的烧录文件和 ELF、正确的映射、对应源码与行号、断点命中和单步；多核分别检查。仅成功打开一个同名 C 文件不能证明三者匹配。本次文档修订未获取新的失败构建，也未连接板卡进行该场景实测。

D.8 流水线长期规范与参考

每次归档 HEX、ELF、Git SHA、编译器版本、配置、子模块/依赖版本、实际源码根和不在 Git 中的生成代码。固定 checkout 根目录或记录旧根目录，均可降低映射维护成本。GCC 可在编译阶段

对相关源文件一致使用以下参数，旧根目录由流水线展开：

```
-fdebug-prefix-map=<CI_SOURCE_ROOT>=/debug-src/firmware


# Local debug-ci.toml:
[[source_map]]
from = "/debug-src/firmware"
to = "."
```

THA6206 当前使用 GHS，不能直接套用 GCC 参数；编译时路径重写需查对应 GHS 版本文档。现有 source_map 不依赖 GHS 提供同名参数，也不要求修改流水线 ELF。

参考：GDB Source Path（前缀替换、目录边界和规则顺序）；GCC Debugging Options（调试信息目录重写）。

实现依据：`src/config.rs`：Project::from_document / source_path；`src/session.rs`：加载 ELF 后设置 substitute-path；`src/coordinator.rs`：derive_project；`src/ui/source_tabs.rs`：read_source。